

TECNOLÓGICO NACIONAL DE MÉXICO

**Instituto Tecnológico de Cuauhtémoc
Instituto Tecnológico de Cd. Jiménez**



FUNDAMENTOS DE PROGRAMACIÓN

Una Guía Teórico-Práctica



**Carlos Alonso Camacho Ramírez
Alba Juassu Ogaz Vasquez
Bertha Ivonne Sánchez Luján**



FUNDAMENTOS DE PROGRAMACIÓN

Una Guía Teórico-Práctica



Carlos Alonso Camacho Ramírez
Alba Jyassu Ogaz Vasquez
Bertha Ivonne Sánchez Luján



Camacho Ramírez, C.; Ogaz Vasquez, A. y Sánchez Luján, B.I.
(2023). *Fundamentos de Programación: Una Guía Teórico-Práctica*. México.

Primera edición, 2023

Este libro fue evaluado por pares académicos integrantes del Cuerpo Académico ITCHID-CA-2 - Educación Matemática y Computación, del TecNM/ Instituto Tecnológico de Chihuahua II.

Edición, diseño de portada y maquetación: Alba Jyassu Ogaz Vasquez y Bertha Ivonne Sánchez Luján

Derechos reservados, 2023

©Carlos Alonso Camacho Ramírez

Alba Jyassu Ogaz Vasquez

Bertha Ivonne Sánchez Luján

© Plaza Editores

El contenido de este libro es responsabilidad exclusiva de sus autores (as).

ISBN: 978-607-59872-4-8

Se autoriza el uso del contenido de esta obra con fines no comerciales, siempre y cuando se cite la fuente y se respeten los derechos patrimoniales de los autores y titulares de los derechos de autor de la misma. Atribución-NoComercial 4.0 Internacional (CC BY-NC 4.0)

Editado en Chihuahua, México.

Impreso en Guadalajara, Jalisco, México.

Introducción

La educación es un pilar fundamental para el progreso de la sociedad, y resulta crucial atender cuidadosamente la forma en que se transmiten los conocimientos a otras personas, para que estos adquieran una formación específica y puedan aplicarla en áreas concretas. En este proceso de aprendizaje, intervienen dos partes esenciales que deben trabajar en conjunto. Por un lado, está la metodología de enseñanza empleada por el instructor, y por otro, la manera de aprender del estudiante, buscando alcanzar un nivel de competencia adecuado.

En la actualidad, el uso de computadoras digitales se ha convertido en una necesidad básica en diversos ámbitos laborales, lo que hace esencial mantenerse constantemente actualizado en esta área tecnológica. Específicamente, es crucial saber cómo dar órdenes a estas máquinas para que ejecuten tareas de manera correcta; esta actividad se conoce como programación. De ahí la importancia de capacitarse mediante cursos relacionados con la creación de programas informáticos.

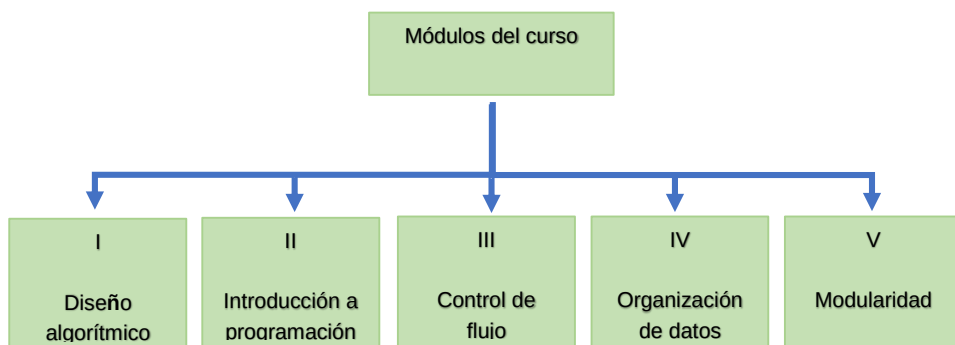
El material presentado en este libro es una propuesta para ayudar a los estudiantes a utilizarlo como una herramienta de apoyo en sus cursos de programación de computadoras. El contenido se presenta de manera teórico-práctica, ya que busca complementar el conocimiento adquirido en las aulas con la aplicación práctica, sin descuidar los fundamentos teóricos necesarios para lograr un aprendizaje sólido.

El documento está diseñado especialmente para aquellos estudiantes que se inician en el área de la programación de computadoras, utilizando la metodología estructurada. Se exponen las ideas fundamentales necesarias para la resolución de problemas mediante algoritmos, utilizando los softwares DFD y PSeInt. Posteriormente, se enseña a crear programas computacionales con el lenguaje de programación Dev-C++. Además, este libro puede ser de gran apoyo para los docentes que imparten materias básicas en el área de programación.

El material está organizado en cinco capítulos que guiarán paso a paso en el conocimiento de la programación de computadoras, se inicia desde conceptos básicos hasta la capacidad de resolver problemas de la vida real a través de programas computacionales. La propuesta de trabajo incluye una combinación de explicaciones teóricas, ejemplos prácticos y ejercicios para garantizar una comprensión profunda y aplicable del contenido.

El enfoque propuesto es avanzar de manera ordenada y gradual, desde el primer hasta el último capítulo, comprendiendo los conceptos fundamentales y aplicándolos mediante ejemplos y ejercicios utilizando el lenguaje de programación.

A continuación, se muestran los temas o capítulos que se abordarán en este libro:



En el primer capítulo, denominado Diseño algorítmico, se abordan los fundamentos de algoritmos creando diagramas de flujo con el software DFD y pseudocódigos con el software PSeInt.

En el segundo capítulo, llamado Introducción a la programación, se inicia con la creación de programas básicos con el lenguaje de programación de Dev-C++.

En el tercer capítulo, denominado Control de flujo, se da continuidad con el uso de diferentes tipos de instrucciones de control denominadas condicionales y cíclicas usando el lenguaje de programación de Dev-C++.

En el cuarto capítulo, llamado Organización de datos, se trabaja con la forma de organizar los datos usando arreglos y estructuras con el lenguaje de programación de Dev-C++.

En el quinto capítulo, denominado Modularidad, se enfoca a la división de un programa en partes llamados módulos a través de la creación de funciones en el lenguaje de programación de Dev-C++.

Propuesta de trabajo

La propuesta de trabajo incluye una explicación detallada de cada tema y subtema, con ejemplos específicos para facilitar su comprensión y aplicación. Además, se sugiere resolver ejercicios de manera conjunta entre el instructor y los estudiantes para abordar posibles dudas, y luego, para reforzar lo aprendido, se alienta a los estudiantes a realizar actividades de manera individual que serán entregadas al instructor(a). Para esto, se identifican los siguientes elementos:

- **Ejemplos.** Se utilizan para la explicación y entendimiento de cada uno de los temas tratados en clase (diagramas, pseudocódigos o programas).
- **Ejercicios.** Son ejercicios o programas que se resuelven en conjunto instructor/ estudiantes para resolver posibles dudas.
- **Actividades.** Son tareas para realizar por parte del estudiante para completar lo aprendido.

Necesidades para trabajar

- ✓ Los programas que aquí aparecen se pueden trabajar en distintos sistemas operativos, en nuestro caso usamos Windows versión 10 o posterior.
- ✓ Para la creación de algoritmos se proponen las herramientas de software Free-DFD para diagramas de flujo y PSeInt para pseudocódigos.
- ✓ Para la elaboración de programas se usa Dev-C++. Esto por su fácil adquisición y sencillo entendimiento para todos aquellos que inician en el área de la programación de computadoras.

Objetivos generales del libro

- Servir como instrumento central, en el proceso de enseñanza del primer curso de programación de la carrera de Ingeniería en Sistemas Computacionales.
- Ser una herramienta de trabajo dentro de un proceso de aprendizaje, guiado por un profesor(a) y donde el estudiante sea el protagonista.
- Encontrar mejores formas de enseñar, resolviendo problemas de la vida real usando un lenguaje de programación de alto nivel.
- Mostrar al estudiante los conceptos necesarios del área de programación, de manera tal que los pueda implementar en trabajos, tareas y prácticas de programación.
- Conocer distintas herramientas computacionales que ayudan en la creación de programas (Editores, Compiladores, Depuradores, etc.).
- Conforme se avance en cada tema, aplicar los conocimientos tratados, construyendo programas utilizando el *lenguaje* de programación C++.

Cada capítulo ha sido diseñado para apoyar a los estudiantes a aprender o mejorar sus habilidades en programación. ¡Esperamos que disfruten este recorrido!

Carlos Alonso Camacho Ramírez. TecNM/ ITCd Cuauhtémoc
Alba Jyassu Ogaz Vasquez. TecNM/ ITCd Jiménez
Bertha Ivonne Sánchez Luján. TecNM/ ITCd Jiménez

Contenido

Capítulo 1: Diseño algorítmico	1
1.1 Computación y programación	1
1.2 Conceptos básicos de algoritmos	9
1.3 Diseño de algoritmos.....	19
1.3.1. Diagramas de flujo	19
1.3.2. Pseudocódigo	32
1.4 Diseño de funciones.....	49
Capítulo 2: Introducción a la programación	55
2.1 Introducción	56
2.2 Entornos de trabajo	59
2.3 Proceso de compilación	74
2.4 Estructura de un programa en Dev-C++	76
2.5 Instrucciones básicas en programa de C++.....	78
2.5.1 Instrucciones de entrada/salida	78
2.5.2 Tipos de datos en C++.....	79
2.5.3 Expresiones en C++.....	85
2.5.4 Cadenas.....	89
2.6. Complemento: Uso de system	95
Capítulo 3: Control de flujo	103
3.1 Introducción	104
3.2 Estructuras secuenciales	104

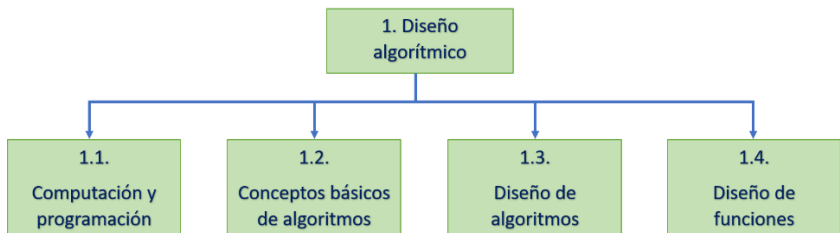
3.3 Estructuras selectivas	106
3.4 Estructuras de repetición	122
3.4.1 Instrucción for	122
3.4.2 Instrucción while	126
3.4.3 Instrucción do-while	129
3.5. Complemento: Instrucciones de salto en estructuras de control	133
3.6. Complemento: Buffer en C++	140
Capítulo 4: Organización de datos	147
4.1 Arreglos	148
4.2 Arreglos unidimensionales	149
4.3 Arreglos Bidimensionales	154
4.4 Estructuras o registros	158
4.4.1 Estructuras anidadas	164
4.4.2 Arreglo de estructuras	168
Capítulo 5: Modularidad	179
5.1 Módulos o funciones en C++	180
5.2 Funciones definidas por el programador en C++	181
5.2.1 Variables locales y globales	192
5.3 Funciones existentes en C++	198
Referencias	209
Resumen.....	211

Capítulo 1: Diseño algorítmico

Objetivos de capítulo 1:

- Conocer el concepto general de computadora y sus posibles aplicaciones para la solución de problemas, así como los elementos que la componen.
- Entender el concepto de algoritmo y su creación por medio de diagramas de flujo y de pseudocódigos utilizando herramientas de software.
- Aplicar en los algoritmos los conceptos de estructuras de decisión, repetición y funciones por medio de ejemplos en software PSeInt.

Subtemas a trabajar en capítulo 1:



1.1 Computación y programación

En el mundo actual escuchamos comúnmente el uso de la **computadora**, que nos ayuda en casi todos los campos de

nuestra vida diaria, por ejemplo, la usamos en: educación, industria, gobierno, medicina y negocios particulares, entre otros. Por lo que es casi imposible pensar en el funcionamiento de una empresa, una aerolínea, un banco, una compañía de seguros o una institución educativa sin recurrir a estas máquinas las cuales ayudan a ordenar y manipular datos.

En la mayoría de las tareas que se realizan existe una gran dependencia a las computadoras, esto porque desde un microondas, un celular, una lavadora automática y hasta un automóvil, necesitan de programas computacionales o informáticos para poder funcionar.

Una **computadora** es una máquina electrónica capaz de resolver problemas mediante la ejecución de programas computacionales. Está formada físicamente por circuitos y componentes electrónicos, que en conjunto pueden ejecutar tareas diversas de forma rápida. Estas máquinas se apoyan en un conjunto de programas (software) para controlar las actividades que se realizan. De acuerdo con esta definición, podemos decir que la computadora se divide fundamentalmente en dos partes:

- El **hardware** que es la parte física de un computador, es decir, la parte que realiza los cálculos.
- El **software** es la parte lógica del computador, es decir, la parte que le dice al hardware que debe realizar.

Algunos autores utilizan la siguiente metáfora: “Un computador es como un ser humano: el hardware es el cuerpo y el software es la mente”.

El **hardware**, es un conjunto de componentes físicos o dispositivos que están divididos de acuerdo con la tarea que realizan: dispositivos de entrada (teclado), de salida (monitor), de

almacenamiento (disco duro, RAM) y de cómputo (procesador). Observemos la figura 1.1.

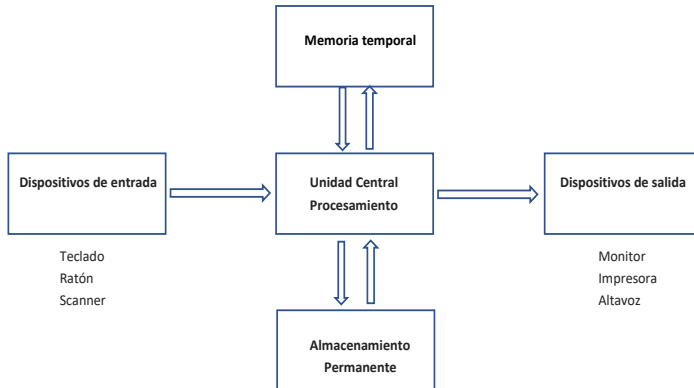


Figura 1.1. Hardware general de computadora.

El **Software** es un conjunto de programas de cómputo, así como datos, procedimientos y actividades que permiten realizar tareas en una computadora. Prácticamente le dicen al hardware que es lo que debe de realizar. Puede estar constituido por:

- Un sistema operativo.
- Herramientas de software.
- Lenguajes de alto nivel.

El **Sistema Operativo** es el conjunto de programas y datos que administra los recursos tanto de hardware (dispositivos) como de software (programas y datos) de un sistema de cómputo y/o comunicación. Además, funciona como interfaz entre la computadora, el usuario y las aplicaciones.

Algunos ejemplos de sistemas operativos pueden ser: Windows, Linux, Mac OS entre otros. Para el caso de dispositivos móviles se

encuentran Android, IOS, Windows Phone, entre otros. Cada uno de ellos tiene diferentes versiones y distribuciones que se ajustan a los diversos equipos de cómputo en los que trabajan.

La **herramienta de software** es un programa diseñado para permitir a un usuario realizar diferentes tipos de tareas, ejemplos, el procesador de texto Word, la hoja de cálculo Excel y base de datos Access.

Los **lenguajes de alto nivel** son aquellos que permiten al programador crear programas por medio de un conjunto de instrucciones, para que puedan ser ejecutados por una computadora. Por ejemplo, C++, Java, Perl, PHP, Python, entre muchos más.

Estos lenguajes normalmente se clasifican en **compiladores** e **intérpretes**, los cuales nos ayudan a traducir un programa (archivo fuente) escrito en un lenguaje de alto nivel hacia un programa que pueda ser entendido y ejecutado por la computadora (archivo ejecutable o aplicación).

Como se dijo, un programa debe pasar por un proceso para que pueda ser convertido y entendido por la computadora, por lo que es importante conocer el funcionamiento general y la relación que tienen los lenguajes con el hardware de la computadora, lo cual observamos en la figura 1.2.

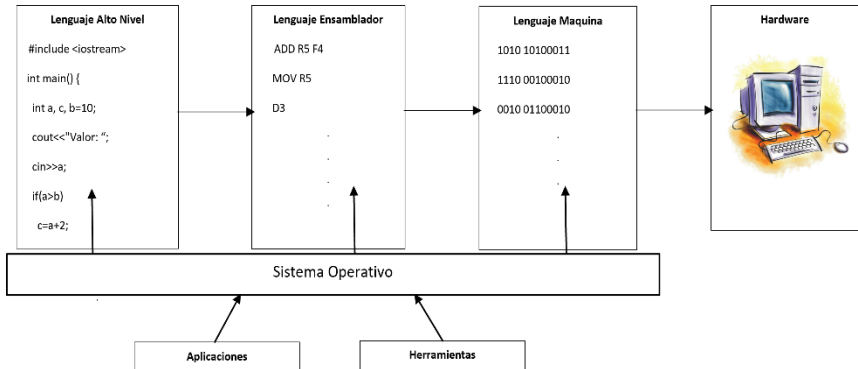


Figura 1.2. Arquitectura de software.

En figura 1.2 se muestra la arquitectura de **software** y las relaciones entre los diferentes lenguajes participantes en la traducción de un programa, los cuales mencionamos enseguida:

- Lenguaje de Alto Nivel. Permite escribir instrucciones para crear un programa computacional y traduce estas instrucciones a lenguaje ensamblador.
- Lenguaje Ensamblador. Se encarga de traducir las instrucciones en ensamblador a instrucciones en lenguaje máquina.
- Lenguaje Máquina. Es el único que puede entender la computadora, formado por 0 y 1.

En la figura 1.2 también observamos que la administración de los recursos del sistema de cómputo es realizada por el **sistema operativo**, el cual nos permite trabajar con las aplicaciones.

El objetivo aquí es aprender a construir programas de computadora, entonces es importante entender de manera general los pasos a seguir para lograrlo, y además llamar por su

nombre a las personas que participan en este proceso. Por lo que revisaremos los siguientes conceptos:

Un **programa** de computadora es un conjunto de instrucciones que le indican a la computadora las tareas que debe realizar. De acuerdo con la definición, un programa es un software.

El **cliente** es la persona u organización que pide la solución de algún problema de la vida real por medio de programas de computadora.

Los **programadores** son las personas que entienden las necesidades del cliente para después diseñar y construir programas de computadora que resuelvan los problemas planteados.

El **usuario** es toda persona que utiliza los programas desarrollados por los programadores. En algún momento todos nosotros somos usuarios al utilizar un navegador de internet, un procesador de texto, una aplicación matemática, o cualquier otro software.

En la figura 1.3 se muestran las personas que pueden iniciar, construir y usar un programa de computadora. Observemos que aquí se separa al cliente y al usuario debido a que no siempre son las mismas personas.

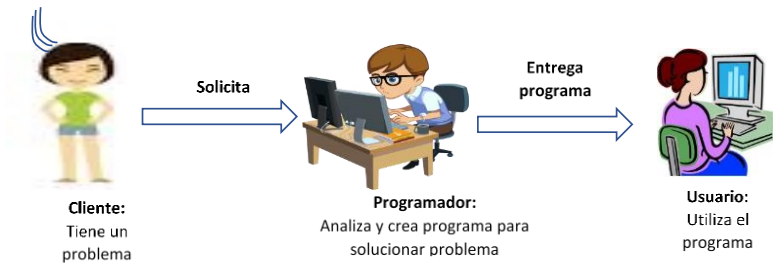


Figura 1.3. Proceso general para solución de problemas.

El análisis y diseño de sistemas computacionales

Para resolver algún problema utilizando un lenguaje de programación se requiere de ciertas actividades, desde el entendimiento del problema observando cómo trabaja y la obtención de información para proponer un diseño para su mejora o solución y por último creando los programas computacionales necesarios. Más específicamente, el **análisis y diseño de sistemas** el desarrollo de sistemas computacionales se divide en:

- **Definición.** Aquí se especifica y entiende cuál es el problema que se pretende resolver.
- **Análisis.** Se recaba y analiza la información para proponer posibles mejoras. Se especifica qué es lo que el sistema debe de hacer.
- **Diseño.** Se especifica las características del producto terminado, esto es, se establece como alcanzar la meta.
- **Programación.** Se escriben las instrucciones en un lenguaje de programación, para construir los programas.
- **Prueba.** Aquí se verifica y prueba que el programa funciona correctamente.

Dentro del análisis y diseño de sistemas informáticos aparece una actividad fundamental llamada **programación**, de aquí lo importante que son los **Fundamentos de programación**.

Lenguajes de programación y programas

Observamos que el tratar de entender el lenguaje nativo del equipo (lenguaje máquina), es más fácil de aprender y entender un lenguaje de programación para dar instrucciones al equipo de cómputo. Por ejemplo, un software llamado **compilador** toma las instrucciones escritas en el lenguaje de programación y participa para convertirlas en un lenguaje máquina.

La programación en su concepto más básico consiste en: decirle a un equipo qué debe hacer. Por medio de las instrucciones escritas en un lenguaje de programación se le dice, por ejemplo, que muestre algo en la pantalla, que emita un sonido o que realice cálculos e imprima resultados. Algunos ejemplos de lenguajes de programación que se pueden utilizar son: Pascal, Fortran, Cobol, Ada, C, Java, Python, PHP, Visual Basic, C++ y Ruby, estos tienen muchos puntos en común, y con estas bases te resultará muy fácil aprender y elegir el lenguaje de programación a utilizar.

Los **lenguajes de programación** nos permiten construir **programas** y las características generales de un programa son:

- Persiguen resolver un problema.
- Tiene una secuencia de instrucciones.
- Son instrucciones que entiende la computadora.

Hablando de los programas, también podemos observar la figura 1.4 el proceso normal que estos realizan: Reciben datos de entrada, procesan los datos y generan datos de salida:

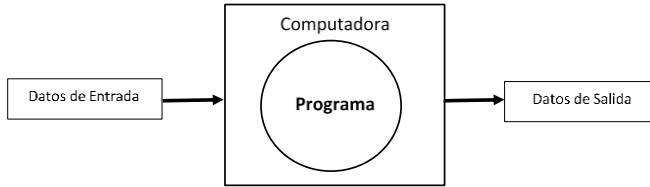


Figura 1.4. Tareas básicas de un programa computacional.

Por todo lo anterior, podemos decir que el papel de los **programas** es importante, ya que sin ellos la computadora es virtualmente inútil. Los lenguajes de programación nos permiten escribir estos programas y por consiguiente comunicarnos con las computadoras.

1.2 Conceptos básicos de algoritmos

Un **algoritmo** es una secuencia de pasos a seguir para alcanzar un objetivo o resolver un problema, por ejemplo, cuando quiero utilizar un videojuego, podría realizar los siguientes pasos:

1. Elijo un videojuego de mi colección.
2. Compruebo si Tv y consola de videojuego están conectados (y procedo).
3. Si la Tv y/o consola de videojuego esta(n) apagada(s) la(s) enciendo. Si no, pues no.
4. Introduzco el videojuego en la consola de videojuegos. Dejo el estuche sobre la consola.
5. Si la Tv no está en el conector adecuado, lo elijo, si no, pues no.
6. Tomo los controles a distancia (el del Tv y el del videojuego).
7. Me pongo cómodo.
8. Elijo las condiciones iniciales del videojuego.
9. Pulso PLAY para comenzar a jugar.

Observemos que, en el algoritmo anterior, la descripción de cada paso no me lleva a ambigüedades, los pasos son absolutamente explícitos y además el número de pasos es finito.

De lo anterior, podemos decir que las **características fundamentales** que debe cumplir un **algoritmo** son las siguientes:

- Un algoritmo debe ser preciso e indicar el orden de realización de cada paso.
- Debe estar totalmente definido.
- Debe ser finito. Se debe terminar en algún momento o tener un número finito de pasos.

Además, para la creación de algoritmos es indispensable contar con: Entrada, Proceso y Salida, por ejemplo, para realizar un platillo siguiendo una receta de cocina se tendría:

- **Entrada:** ingredientes y utensilios empleados.
- **Proceso:** elaboración de la receta en la cocina.
- **Salida:** platillo terminado y listo para comer.

Observemos que en la solución de problemas por medio de algoritmos lo expresamos en **palabras entendibles para nosotros**, aun no se está especificando algún lenguaje de programación.

En la figura 1.5 se muestran los pasos a seguir para la solución de un problema por medio de un algoritmo, para que posteriormente se convierta en un programa de computadora:

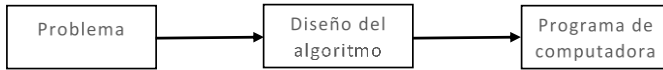


Figura 1.5. Solución de problema con algoritmo.

En la figura 1.5 observamos que, para obtener un programa de computadora, se recomienda primero el diseño de su algoritmo, por lo que enseguida aparecen algunos **conceptos básicos**, necesarios en la construcción de algoritmos y programas:

Variable. Es un espacio donde se almacena un dato o valor. A este espacio se le asigna un nombre de variable para referirse al valor almacenado en la memoria de la computadora.

El valor del dato almacenado en una variable puede cambiar tantas veces como sea necesario, pero los valores siempre serán de un mismo tipo.

A continuación, se muestran ejemplos de variables y la asignación de un dato a cada una de estas:

Nom = "Carlos Alonso"

Se usa una variable llamada **Nom** y almacena el texto **Carlos Alonso**.

Edad = 42

Se usa una variable llamada **Edad** que almacena el dato numérico **42**.

Pais = "Mexico"

Se usa una variable llamada **Pais** y almacena el texto **Mexico**.

Estatura = 1.68

La variable se llama **Estatura** y almacena el dato numérico **1.68**.

Normalmente antes de poder hacer uso de una variable es necesario definirla o declararla y una forma básica podría ser:

[Tipo de dato] [Nombre de variable(s)]

Primero se especifica el tipo de valor que va a almacenar (Numérico, carácter, etc.). Por ejemplo, la declaración de una variable para almacenar la edad de una persona podría escribirse:

Entero edad

Es importante conocer que no se pueden declarar variables con el mismo nombre, pero sí pueden existir varias variables del mismo tipo. También puedo declarar distintas variables al mismo tiempo, por ejemplo, para almacenar número de hijos, se puede escribir de la siguiente manera:

Variables declaradas por separado	Variables declaradas usando la coma: ,
<i>Entero edad</i> <i>Entero numero_de_hijos</i>	<i>Entero edad, numero_de_hijos</i>

Opcionalmente, al declarar variables, se les puede asignar un valor inicial. Por ejemplo, si desea declarar variable para almacenar un número entero y que inicialmente tenga el valor 35, escribiría:

Entero NumEnt = 35

Constante. Una constante también almacena datos, la diferencia es que el valor no puede cambiar. En programación es una buena práctica escribir los identificadores de constante en mayúsculas, de esta forma es más fácil localizarlos. Ejemplos, para almacenar un año de nacimiento el cual nunca va a cambiar o el valor de la constante matemática Pi, que se muestran a continuación:

Constante ANACIMIENTO = 1968

Constante PI = 3.1416

Constante TEMPERATURA = -5

Tipo de dato. Cuando creamos algoritmos que usan una o más variables es necesario especificar el tipo de valor que estas pueden almacenar. Los tipos de datos básicos en algoritmia pueden ser:

Entero, Real, Lógico y Caracter. A partir de estos se pueden crear otros tipos de datos (compuestos) como por ejemplo el tipo Cadena que se forma por un conjunto de caracteres.

Entero. Estos pueden almacenar valores enteros positivo y/o negativo, sin punto decimal. Ejemplos: 123, 25, -12, -5689, 30285 o 1968.

Real. Almacenan un valor con punto flotante. Ejemplos: 145.89, 0.5559, 9.0, 3870.8888 o 195.56.

Lógico. Pueden almacenar solo dos valores (0 o 1). El 0 representa falso y 1 representa verdadero.

Caracter. Almacena un solo caracter, ejemplos: 'a', '5', 'R', 'P', '6', '2', 'y' o 'z'. La comilla nos especifica que los números se toman como un caracter y no como un valor numérico.

Cadena. Es un conjunto de caracteres. Ejemplos: “casa”, “panfilo”, “Es prueba:7654”, “R5589” y “este es correo: ab@e25”.

Identificador. Es el nombre que se le da a un elemento del algoritmo y estos se pueden clasificar como:

Palabras reservadas. Son identificadores que ya están predefinidos y forman parte de un lenguaje algorítmico o de programación (tienen un significado especial).

Identificadores definidos. Son los identificadores que nosotros podemos definir, por ejemplo, asignar un nombre a un algoritmo, o poner nombre a variables (Nom, Pais y edad) y a constantes como ANACIMIENTO o PI. Algunas reglas de sintaxis para la asignación de un nombre a un elemento de un algoritmo son:

1. Consta de uno o más caracteres.
2. El primer caracter debe ser una letra o el caracter subrayado (_), mientras que, todos los demás pueden ser letras, dígitos o el carácter subrayado.
3. Las letras pueden ser minúsculas o mayúsculas del alfabeto inglés.
4. No está permitido el uso de las letras 'ñ' y 'Ñ'. Las vocales no pueden llevar diéresis ni acentos. Además, no se permiten espacios en blanco.
5. No pueden existir dos identificadores iguales (dos elementos de un algoritmo no pueden nombrarse de la misma forma).

Ejemplos de identificadores válidos que pueden ser definidos por el desarrollador:

numero
dia_del_mes
PINGUINO1
_ciudad

Los siguientes identificadores no son válidos por incumplir alguna regla:

_Día
*numero**
Lugar de nacimiento
año

Según los tipos de datos mencionados, las siguientes palabras no pueden ser utilizados como identificadores debido a que ya están predefinidos: **Entero** o **Caracter**.

Además, los identificadores son sensibles a minúsculas y mayúsculas, por ejemplo, **Mes** y **mes** son considerados identificadores distintos.

Operadores y expresiones. La expresión nos ayuda a realizar diferentes operaciones sobre los datos y/o variables. Se construyen bajo reglas de sintaxis ya determinadas y puede combinar operadores con uno o más operandos. El **operador** define las operaciones que se realizaran dentro de una expresión, por ejemplo, los operadores de suma o multiplicación dentro de una expresión. Y los **operandos** pueden ser valores, constantes o variables, por ejemplo, con los datos numéricos se pueden realizar operaciones aritméticas:

$111 + 6$	<i>Operación suma</i>
$19 - 72.25$	<i>Operación resta</i>
$24 * 3$	<i>Operación multiplicación</i>

Al evaluar una expresión, ésta toma un resultado que depende del valor asignado previamente a las variables/datos y de los operadores utilizados. El resultado de una expresión también será de un determinado tipo de dato. Las últimas tres expresiones se conforman de dos operandos y un operador, pero el número de éstos pueden variar, por ejemplo:

$111 + 6 - 8$	Tres operandos y dos operadores
$-((+19) + 72)$	Dos operandos y tres operadores
$-(-72)$	Un operando y dos operadores

Observe que los caracteres más (+) y menos (-) tienen dos usos:

- Los operadores de más (+) y menos (-) se usan para realizar la operación aritmética correspondiente.
- Signo de un número, pueden actuar como operadores unarios (aplican a un solo operando).

Los caracteres para abrir paréntesis "(" y cerrar paréntesis ")" se utilizan para establecer el orden en el que los operadores actúan sobre los operandos. Por ejemplo, dadas las declaraciones:

```
Constante PI = 3.141592
Entero numero = 2
Real radio_circulo = 3.2
```

Con los valores asignados a las variables, podemos escribir las siguientes expresiones:

```
2 * PI * radio_circulo  
( PI * PI )  
numero * 5
```

De las evaluaciones anteriores se obtienen los siguientes resultados:

Resultado	Tipo dato	Operaciones
20.106189	(valor real)	(2 * 3.141592 * 3.2)
9.869600	(valor real)	(3.141592 * 3.141592)
10	(valor entero)	(2 * 5)

Las expresiones pueden estar clasificadas de la siguiente manera:

- Aritmética (devuelve un número entero o real).
- Lógica (devuelve un valor lógico: verdadero o falso)
- De caracter (devuelve un caracter representable por el ordenador).
- De cadena (devuelve una cadena).

Una vez entendidos los conceptos básicos de algoritmos podemos decir que existen diferentes formas de representarlos, estas son:

- Lenguaje natural (ambiguo y extenso).
- Pseudocódigo (reduce ambigüedad y estructurada).
- Diagrama de flujo (reduce ambigüedad y estructurada).
- Lenguaje de programación (en computación).

En el área de la programación las representaciones más usadas son el “Diagrama de flujo” y el “Pseudocódigo”. Pero antes de iniciar realicemos la siguiente actividad usando un lenguaje natural.

Actividad 1.1: Realizar los siguientes algoritmos escribiendo los pasos con sus propias palabras:

1. Escriba algoritmo que especifique todas las actividades que se siguen regularmente para trasladarse a la escuela (desde que se levanta hasta que llega a la escuela).
2. Escriba algoritmo que especifique los pasos a seguir para hacer la compra de alimentos a consumir durante la siguiente quincena o semana.
3. Escriba algoritmo que solicite un número y nos diga si es positivo o negativo.
4. Escriba algoritmo que nos ayude a realizar la suma de tres números y nos diga cuál es el resultado.

1.3 Diseño de algoritmos

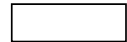
1.3.1. Diagramas de flujo

Es la representación gráfica de un algoritmo usando un conjunto de símbolos. Cada símbolo representa una acción en concreto y cada instrucción del algoritmo se visualiza dentro del símbolo correspondiente. Los símbolos se conectan con líneas de flujo o flechas para indicar el orden en que se ejecutan las instrucciones. Las formas de los símbolos podrían variar según el autor o software con el que se trabaje. Un conjunto de símbolos generales se muestra en figura 1.6.

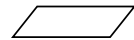
Se usa para indicar el inicio y el fin del diagrama de flujo:



Símbolo de proceso que representa una operación sobre los datos:



Símbolo de entrada de datos (Lectura):



Símbolo que representa la salida de datos (Escritura):



Símbolo Subproceso. Es la llamada a un módulo cuyo diagrama esta independiente:



Símbolo de decisión. Ayuda a cambiar el curso del diagrama basado en la condición:



Las flechas nos indican cual es el flujo dentro del diagrama:

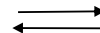


Figura 1.6. Símbolos generales en diagrama de flujo.

El diagrama de flujo de la figura 1.7 representa el algoritmo para calcular el área de un rectángulo.

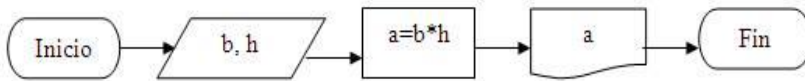


Figura 1.7. Ejemplo de diagrama de flujo.

El **óvalo** rotulado con **Inicio** representa el comienzo del algoritmo y el rotulado con **Fin** nos indica el final del algoritmo. El paralelogramo es el símbolo de **entrada** de datos e indica que se capture el valor de la base (b) y el valor de la altura (h), el rectángulo nos indica que se realiza un **proceso** sobre los datos de entrada para calcular el área (a) multiplicando la base por la altura (se usa el * como operador de multiplicación), enseguida el símbolo de documento representa la **salida** indicando que se muestra el valor obtenido, y enseguida se finaliza el algoritmo.

Reglas para la construcción de diagramas de flujo:

1. Todo diagrama de flujo debe tener un inicio y un fin.
2. Las líneas de flujo nunca deben cruzarse, para evitarlo debe utilizarse el símbolo conector.
3. Las líneas de flujo (flechas) deben terminar siempre en un símbolo.
4. No puede llegar más de una línea de flujo a un símbolo.
5. Todos los símbolos en un diagrama deben estar conectados mediante una flecha; todo símbolo debe tener una flecha entrando y otra saliendo salvo el símbolo que indica inicio o fin del diagrama.
6. no escribir mucho en los símbolos ya que el propio símbolo indica la operación a realizar.

Un algoritmo puede incluir una cantidad de instrucciones, también llamadas sentencias, por ejemplo:

Instrucción de asignación. Almacena un valor al identificador indicado, La variable puede tomar diferentes valores durante la ejecución del algoritmo. Puede ser expresada como **b = 5**, o bien, como **b ← 5**.

Instrucción para Leer (entrada). Indica que el usuario tecla un dato y este se almacena en la variable indicada. Veamos el ejemplo en figura 1.8:

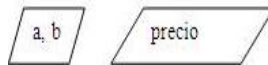


Figura 1.8. Símbolo de lectura.

Instrucción Escribir (salida). Muestra el valor contenido en una variable o algún mensaje en un dispositivo de salida como la pantalla de computadora, observe el ejemplo de figura 1.9:



Figura 1.9. Símbolo de salida.

Los pasos para la **construcción de algoritmos** por medio de diagramas de flujo sería el siguiente:

Entender el problema: Esto es importante para determinar la secuencia de instrucciones que nos ayude a constituir el diagrama. Entender cuál es el problema encontrado y cómo se va a solucionar.

Declaración de datos: Ahora debe hacerse la declaración de datos necesarios, asignando nombre a los datos a utilizar, así como el tipo de valor que pueden almacenar. Estos datos podrían ser:

Datos de Entrada: Los datos que se suministran al algoritmo.

Datos de Salida: Los datos o resultados que se esperan.

Datos de trabajo: Los que se usan para la obtención de los resultados intermedios en un proceso.

Especificar las fórmulas a utilizar: Las fórmulas deben escribirse utilizando los nombres asignados a las variables, así como los operadores necesarios (Aritméticos, lógicos, etc.).

Diseño del algoritmo: Se desarrolla o crea la alternativa de solución para el algoritmo, el cual puede desarrollarse utilizando el diagrama de flujo, verificando los distintos flujos posibles.

Prueba de escritorio: Se debe ejecutar el algoritmo con diferentes datos de prueba para verificar si se obtienen los datos de salida esperados. Si después de las pruebas se demuestra que el algoritmo cumple con el objetivo, felicidades. Pero si no, se revisa el algoritmo y se corrige.

Ejemplo 1.1: Diseñar un diagrama de flujo para convertir una temperatura dada en grados Fahrenheit a la correspondiente temperatura en grados Centígrados.

Entender el problema: Cada grado Fahrenheit es igual a $100/180$ o $5/9$ grados en la escala Celsius. Para convertir Fahrenheit a Celsius se resta 32° a los grados Fahrenheit y el resultado se multiplica por $5/9$.

Datos de entrada: **f**: Grados Fahrenheit a convertir.
Tipo numérico.

Datos de salida: **c**: Los grados Celsius (resultado
obtenido). Tipo numérico.

Formula a utilizar: **$c = (f - 32) * 5/9$**

El diseño del diagrama de flujo se muestra en figura 1.10:

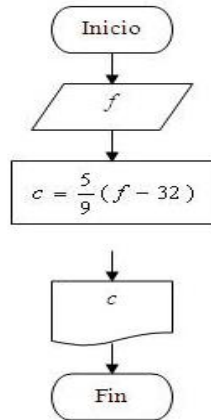


Figura 1.10. Diagrama de flujo para conversión de grados

Las pruebas de escritorio aparecen en figura 1.11:

Grados Fahrenheit (Entrada)	Grados Celsius (Salida)
88	31
95	35
-13	-25

Figura 1.11. Prueba de diagrama de flujo para conversión.

Ejemplo 1.2: Diseñe un diagrama de flujo para calcular el área de un círculo cualquiera y la longitud de su circunferencia.

Entender problema: El área del círculo se obtiene con el radio al cuadrado y el resultado se multiplica por el valor de pi. La longitud de circunferencia es la multiplicación del diámetro por valor de pi.

Datos de entrada: **r**: Representa el radio del círculo.
Tipo numérico.

Datos de salida: **a**: Es el área del círculo. Tipo numérico.
l: Longitud de la circunferencia. Tipo numérico

Fórmulas a usar: **$a = 3.1416 * r^2$**
 $l = 2 * 3.1416 * r$

El diseño del diagrama de flujo se muestra en la figura 1.12:

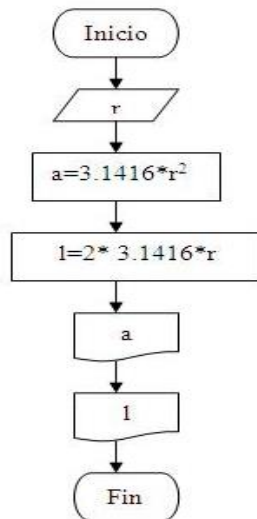


Figura 1.12. Diagrama que calcula área y longitud de círculo.

Las pruebas de escritorio aparecen en la figura 1.13:

Radio (Entrada)	Área: a (Salida)	Longitud: (Salida)
15	706.86	94.24
6	113.1	37.70
3	28.27	18.84

Figura 1.13. Prueba de diagrama de flujo para calcular área de círculo y longitud.

Actividad 1.2: Resolver los siguientes problemas construyendo los diagramas de flujo correspondientes:

1. Calcular el porcentaje de hombres y mujeres que hay en un grupo de personas.
2. Calcular el salario que recibe por mes un empleado que trabaja por horas, reteniéndole el 7 por ciento por concepto de seguro de trabajo.
3. Dado el precio de un artículo y la cantidad de artículos comprados, calcular el total a pagar considerando que se retiene el 12% por concepto de impuesto sobre ventas (ISV).
4. Calcular calificación final de un estudiante del curso de computación, para ello se debe tomar en cuenta lo siguiente: se realizaron tres exámenes valorados en 80% cada uno y tres prácticas de laboratorio valoradas en 20% cada una.

Uso de software DFD para diagramas de flujo

En la actualidad existe diferente software que nos puede ayudar a construir y analizar diagramas de flujo, como por ejemplo el programa llamado **DFD**. Este nos permite crear diagramas de flujo de datos por medio de herramientas de edición para este propósito. Ya que se crea el algoritmo representado de forma gráfica, podrá ejecutarse, analizarse y depurarse en un entorno fácil de usar. Enseguida vamos a construir diagramas de flujo con DFD aplicando los comandos más usados. Una vez que ingresamos al DFD 1.1, nos aparecerá la ventana principal mostrada en figura 1.14:

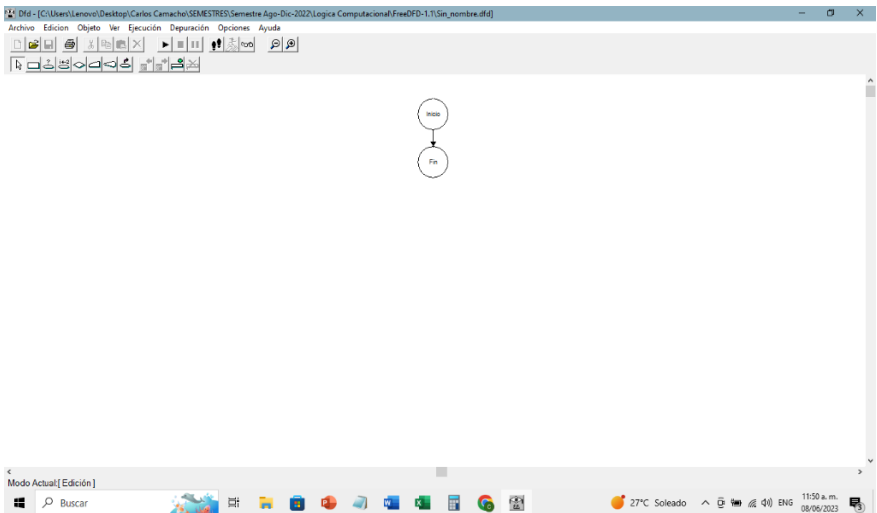


Figura 1.14. Ventana inicial de software DFD.

En la parte superior de la ventana inicial del software DFD se muestra una barra menús con distintas opciones y una barra de herramientas.

Los principales menús y opciones del DFD que podemos utilizar se mencionan a continuación:

Archivo: Menú que se encarga de las acciones del archivo que estamos trabajando.

Nuevo: Se usa para crear un nuevo diagrama DFD.

Abrir: Es para abrir un diagrama DFD ya guardado.

Guardar: Guarda el diagrama que se está trabajando.

Guardar como: Para guardar con un nombre diferente el diagrama que se está trabajando.

Imprimir: Es para imprimir el archivo abierto.

Salir: Para terminar y salir de DFD.

Edición: Menú para editar el diagrama que se está trabajando.

Cortar: Para quitar un bloque del diagrama y guardarlo en el portapapeles.

Copiar: Para copiar el bloque marcado en el diagrama y guardarlo en el portapapeles.

Pegar: Para pegar en el diagrama el bloque guardado en el portapapeles.

Eliminar: Para borrar el bloque marcado en el diagrama.

Eliminar Subprograma: Para borrar el diagrama correspondiente a un subprograma.

Editar Objeto: Edita el contenido del objeto seleccionado.

Objeto: Aquí aparecen todos los comandos para la inserción de objetos en el diagrama.

Cursor: Permite seleccionar objetos del diagrama en edición.

Asignación: Permite asignar valores a las variables usadas en el diagrama.

Ciclo Mientras: Ejecuta un bloque de instrucciones mientras la condición sea verdadera.

Ciclo Para: Ejecuta un bloque de instrucciones hasta que el contador alcance el límite especificado.

Decisión: Selecciona el flujo a seguir de acuerdo al valor lógico de una condición.

Lectura: Permite dar valores a las variables desde el teclado.

Llamada: Se invoca o se hace la llamada a un subprograma.

Salida: Para insertar la presentación de datos en la pantalla.

Nuevo Subprograma: Se abre ventana de edición para crear diagrama de un subprograma.

Ver: Para la visualización y movimiento de ventanas.

Zoom: Para cambiar tamaño de los objetos en pantalla.

Anterior Subprograma: Visualiza el subprograma anterior.

Siguiente Subprograma: Visualiza el siguiente subprograma.

Depurador: Muestra o esconde ventana donde se evalúa expresiones y tipos de datos.

Ejecución: Este menú activa la ejecución del diagrama.

Ejecutar: Ejecuta el algoritmo representado por el diagrama.

Pausar: Realiza una pausa en la ejecución del diagrama.

Detener: Termina la ejecución del diagrama.

Depuración: Controla las actividades de depuración del diagrama.

Paso Simple: Ejecuta el siguiente objeto.

Ejecutar Hasta: Permite realizar la ejecución del diagrama hasta el objeto que se marque dentro del diagrama.

Detener: Termina la ejecución del diagrama.

Evaluar: Muestra el dialogo para evaluación de expresiones y datos.

Ayuda: Nos muestra las características y explicación básica de los comandos que podemos utilizar en DFD.

Opciones: Controla las opciones de trabajar los ángulos en grados para las funciones trigonométricas. Y ángulos en radianes para las funciones trigonométricas.

Debajo de la barra de menús podemos observar algunos botones que sirven para construcción y ejecución de los diagramas de flujo, ver figura 1.15.



Figura 1.15. Botones y símbolos de DFD.

En tabla 1.1 se indica cual es el trabajo de algunos de los botones que aparecen en figura 1.15.

Botón	Acción
	Permite seleccionar objetos del diagrama en edición.
	Permite asignar valores a variables.
	Ejecuta instrucciones mientras la instrucción sea verdadera.
	Ejecuta instrucciones hasta alcanzar el valor límite.
	Instrucción de decisión o condicional.
	Instrucción de lectura o entrada de datos.
	Instrucción de salida de datos.
	Llamada a un subprograma.
	Ejecución del diagrama de flujo.
	Ejecutar paso a paso.
	Muestra o esconde la ventana de depuración.
	Aumenta o disminuye tamaño de los objetos en pantalla.
	Para crear un nuevo subprograma.

Tabla 1.1. Función de algunos botones de DFD.

Ejercicio 1.1: Usando el software DFD, vamos a resolver los problemas planteados anteriormente en la actividad 1.2 construyendo los diagramas de flujo correspondientes.

1.3.2. Pseudocódigo

Pseudocódigo se define como un lenguaje simplificado para describir un algoritmo utilizando una mezcla de frases y palabras claves, indicando el inicio y el fin del algoritmo y las instrucciones específicas a realizarse en el algoritmo. Por ejemplo, el siguiente pseudocódigo corresponde al algoritmo para calcular el área del rectángulo: el algoritmo calcula el área (a) si se le suministra la longitud de la base (b) y la longitud de la altura (h).

Proceso Area

Leer b;

Leer h;

*a=b*h;*

escribir a;

FinProceso

Aquí podemos observar que el algoritmo se llama **Area** y utiliza tres variables (b, h y a) para realizar las operaciones necesarias. Aparecen palabras reservadas como **Proceso** (indica el principio del algoritmo), leer (usada para capturar un valor y guardarlo en la variable especificada), **escribir** (para mostrar en salida estándar, como pantalla, algún resultado obtenido), y **FinProceso** (indica el final del algoritmo). Veamos que también que existe una operación aritmética la cual realiza el cálculo necesario y el resultado lo almacena en la variable **a**. Las instrucciones en pseudocódigo son fáciles de entender y escribir para crear nuestros pseudocódigos.

Uso de software PSeInt para construcción de pseudocódigo

PSeInt es una herramienta de software que nos ayuda a construir pseudocódigos, el cual utiliza un pseudolenguaje que nos permite centrar la atención en los conceptos fundamentales de la

algoritmia. Proporciona un entorno de trabajo con distintos recursos como pseudocódigos y diagramas de flujo; es gratuito y lo encontramos en el siguiente [url: http://pseint.sourceforge.net/](http://pseint.sourceforge.net/) Después de instalarlo e ingresar, nos mostrara la pantalla principal indicada en figura 1.16:

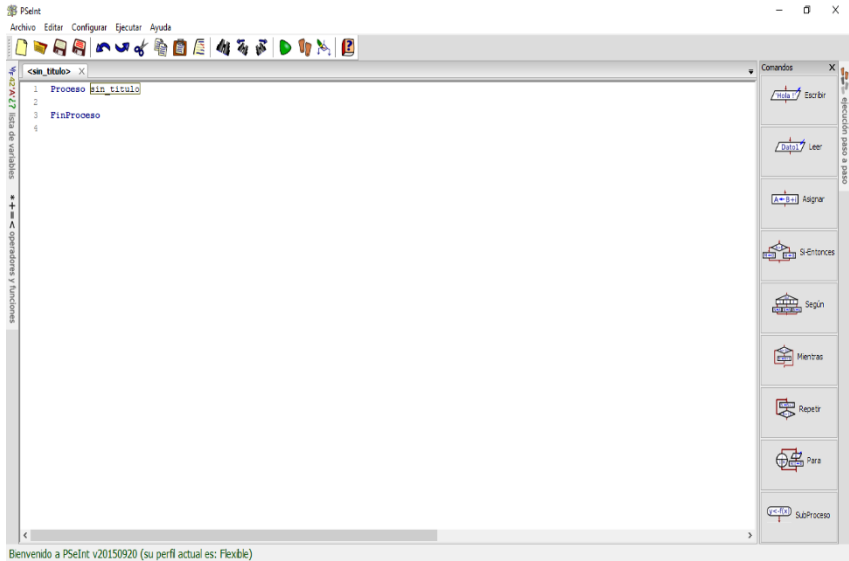


Figura 1.16. Ventana inicial de software PSeInt

Podemos observar que aparece una ventana donde muestra la palabra **Proceso** “sin_titulo”. En la parte de “sin_titulo” podemos escribir el nombre que le queremos asignar al pseudocódigo. También, el final del pseudocódigo se especifica con la palabra **FinProceso**. Entre **Proceso** y **FinProceso** es donde vamos a escribir las instrucciones a realizar por el pseudocódigo. En la parte superior aparece una barra de herramientas, las cuales utilizaremos con algunos ejemplos.

Configurando PSeInt

Es necesario configurar el entorno de trabajo que vamos a usar en PSeInt, por lo que haremos lo siguiente:

- Vamos al menú **Configurar**.
- Seleccionar **Opciones de Lenguaje** (perfiles).
- Aparecerán diferentes perfiles ya preestablecidos por distintas universidades, por defecto viene configurado como **“Flexible”**, la cual puede ser usada.
- También podemos personalizar el perfil si vamos a perfil **<Personalizado>** - Clic en opción **Personalizar** – Aparece ventana **Opciones del lenguaje** – Aquí se puede seleccionar las opciones necesarias, enseguida se muestra una configuración que podríamos usar:

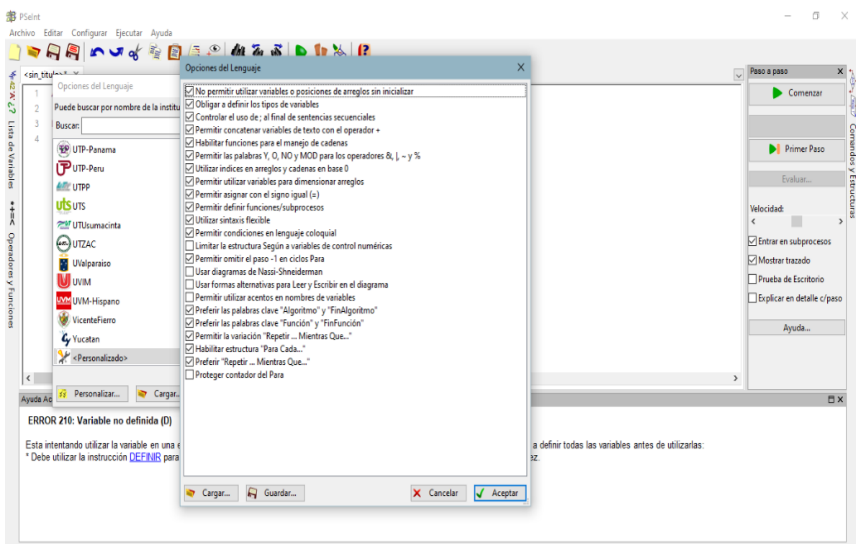


Figura 1.17. Personalizando PSeInt

Después daremos clic en **Aceptar**.

Ejemplo 1.1: Se quiere mostrar en pantalla el nombre de nuestra institución y mi nombre completo, para esto tendríamos que escribir el pseudocódigo siguiente:

Proceso Ejercicio1

Escribir "Instituto Tecnologico de Cd. Cuauhtemoc"

Escribir "Carlos Alonso Camacho Ramirez"

FinProceso

Una vez escrito el pseudocódigo, lo podemos **ejecutar** con la opción correspondiente, lo que nos llevaría a lo mostrado en figura 1.18:

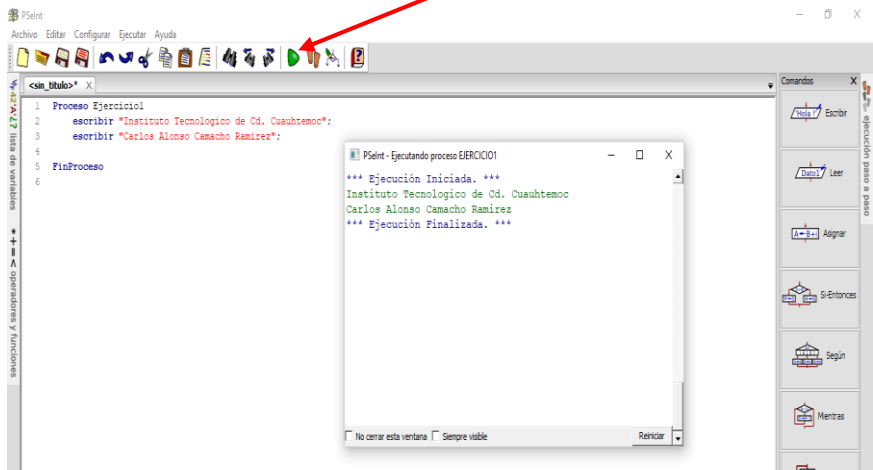


Figura 1.18. Escritura y ejecución de pseudocódigo

En figura 1.18 apareció la **ventana de ejecución** del pseudocódigo donde se muestra resultado. El pseudocódigo se llama **Ejercicio1** y está conformado por dos instrucciones de salida (escribir).

Ejemplo 1.2: Se muestra pseudocódigo donde se utiliza una variable llamada **edad** de tipo entero y se le asigna el valor de 25, para después mostrar un texto seguido por el valor de edad.

Algoritmo Ejemplo2

edad Es Entero;

edad = 25;

Escribir "Mi edad es: ", edad;

FinAlgoritmo

El resultado de este algoritmo se muestra en la figura 1.19:

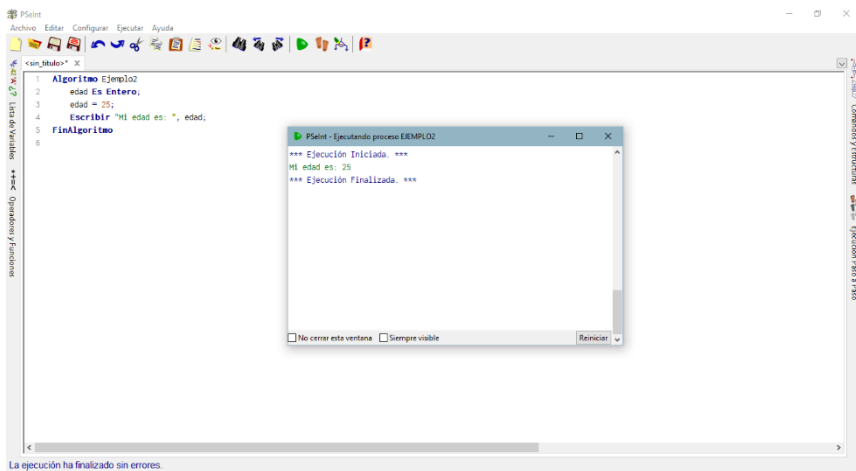


Figura 1.19. Usando declaración y asignación en PSeInt

Ejemplo 1.3: Pseudocódigo para calcular el área de un rectángulo (a), pidiendo los valores necesarios base (b) y altura (h). Se usa la instrucción **leer** para capturar un valor. Observando el ejemplo podemos ver que a diferencia de la palabra **Algoritmo** se usa la palabra **Proceso**, que también es válido.

Proceso Area

```
    Escribir "Dame la base: ";  
    Leer b;  
    Escribir "Dame la altura: ";  
    Leer h;  
     $a = b * h$ ;  
    Escribir "El área es: ", a;  
FinProceso
```

El resultado se muestra en la figura 1.20:

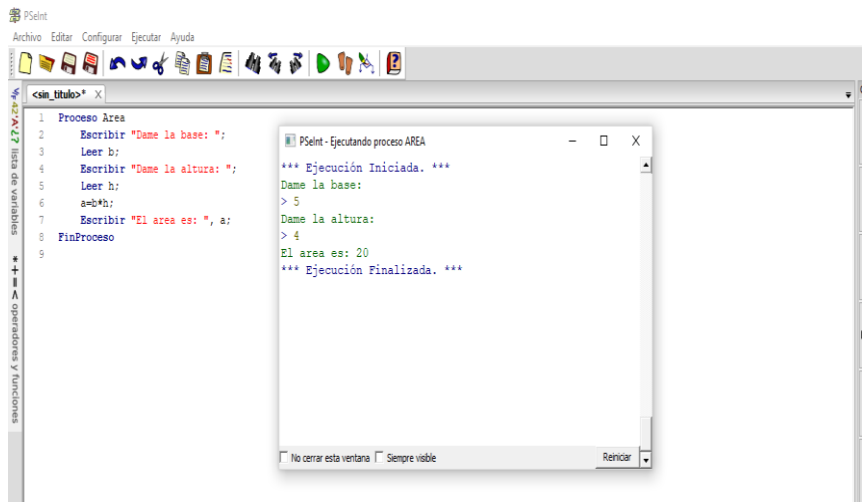


Figura 1.20. Pseudocódigo que calcula el área de rectángulo

Enseguida se mencionan algunas de las instrucciones y herramientas más utilizadas en **PSeInt**.

- Instrucción **leer**, permite que el usuario escriba algo y esto se guarde en la variable indicada.
- En la parte superior observamos las opciones comunes que se usan como nuevo, abrir, guardar, deshacer, cortar, copiar, pegar, ayuda, que son muy comunes.
- En el lado derecho podemos observar “comandos y estructuras” que se pueden utilizar, que aparecen como diagrama de flujo. Debajo aparece “ejecución paso a paso” nos permite ejecutar paso por paso cada instrucción.
- Del lado izquierdo, vemos “lista de variables”, que son todas las variables que se utilizan en el pseudocódigo.
- También aparece “operadores y funciones” donde encontramos operaciones matemáticas (+, -, /, *) y operadores lógicos que se pueden utilizar (mayor que, igual, etc.).
- Para observar el diagrama de flujo del pseudocódigo escrito damos clic en icono **Dibujar diagrama de flujo...**



Figura 1.21. Convertir a diagrama de flujo

Para el ejemplo de calcular el área de un rectángulo mostrado anteriormente, nos quedaría como se muestra en la figura 1.22:

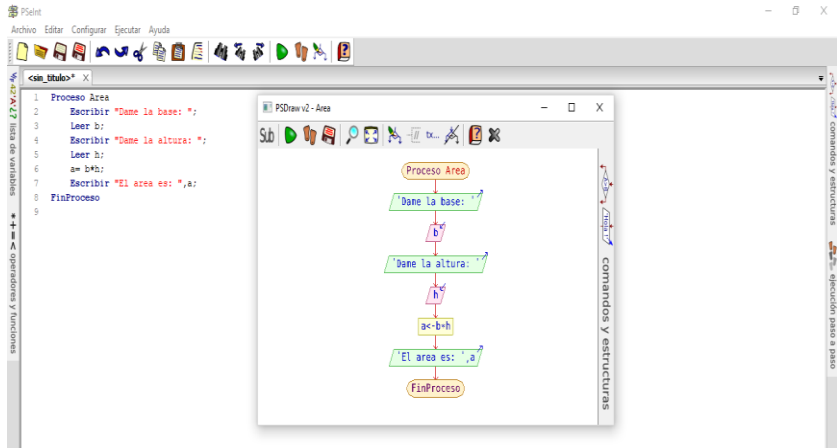


Figura 1.22. Pseudocódigo y diagrama que calcula el área de rectángulo

Actividad 1.3: Realizar los siguientes ejercicios de pseudocódigo usando el Software **PSelnt**:

1. Mostrar el resultado de la fórmula: $xyz = 4 - 2 + 20 * 3 - xy$. Dato de salida: xyz .
2. Sumar 4 números dados, a la suma restarle 10, y a este último resultado multiplicarlo por 5.
3. Solicitar el peso de seis estudiantes, calcular el peso promedio y mostrar resultado.
4. Calcular el perímetro de un cuadrado, el cual se obtiene haciendo la suma de la longitud de sus cuatro lados iguales, que equivale a $p = 4 * longitud$. Dato de entrada: $longitud$. Dato de salida: p .
5. Calcular el área de un triángulo rectángulo. La fórmula a utilizar sería: $\text{Área} = (b * h) / 2$. Datos de entrada: b y h . Dato de salida: Área .
6. Convertir cantidad de grados Fahrenheit a grados Centígrados. Solicitar los grados Fahrenheit, restarle 32 a los grados Fahrenheit y el resultado se multiplica por $5/9$.
7. Buscar cual es la fórmula para calcular el volumen de un cilindro. Crear pseudocódigo que solicite datos necesarios y mostrar el resultado.

Instrucciones de decisión y repetitivas

La forma en que se ejecutan las instrucciones que aparecen en un algoritmo es de manera **secuencial**, esto es, una después de otra según van apareciendo (de primera a la última). Además, también se pueden incluir instrucciones que ayuden a tomar una **decisión**, las cuales nos permiten seleccionar si se ejecuta un conjunto de instrucciones u otro. También existen las instrucciones **repetitivas**, que se utiliza cuando es necesario repetir varias veces la ejecución de un grupo de instrucciones.

Estructuras de decisión (condicionales)

Aquí se evalúa una condición para determinar si se cumple o no, si se cumple (si es verdadera) se ejecuta un conjunto de instrucciones, si no cumple (si es falsa) se ejecutará otro conjunto de instrucciones o simplemente no se realiza nada. Observemos que en la evaluación de la condición el resultado nos puede arrojar un valor lógico (Falso o Verdadero). Por ejemplo:

- Si la calificación es mayor a 70 el alumno aprobará, de lo contrario no acredita la materia.
- Si quiero realizar la operación $c=b/a$ primero hay que verificar que **a** sea diferente de cero, ya que la operación se puede realizar únicamente si **a** es diferente de cero.

La condición se puede especificar usando operadores relacionales o lógicos como los siguientes: > (Mayor que), < (menor que), <= (menor o igual que), >= (mayor o igual que), = (igual que), != (diferente que), **Y** (AND lógico) y **O** (OR lógico). Estos pueden variar en cada lenguaje de programación y herramienta de software que se esté utilizando.

En los siguientes ejemplos observamos que dependiendo del valor de cada variable solo nos dará como resultado un falso o un verdadero:

Calif > 70 Evalúa si el valor de variable **Calif** es mayor a 70.

Sueld > 180 y Sueld < 220 Evalúa si el valor de **Sueld** es mayor a 180 y menor a 220.

B < A Evalúa si valor de B es menor que valor de A.

Ejemplo 1.4: Se evalúa una condición y si el resultado es **verdadero** se ejecuta el conjunto de instrucciones indicado, y si la condición es **falsa** no se ejecuta nada. Se quiere realizar la operación: **c = b/a**, pero primero verificamos que **a** sea diferente de cero, si es diferente entonces se realiza la división, sino pues no pasa nada.

Proceso Deci_Simple

Escribir "Dame valor de b: ";

Leer b;

Escribir "Dame valor de a: ";

Leer a;

Si (a != 0)

c = b/a;

Escribir "El resultado es: ", c;

FinSi

FinProceso

Ejemplo 1.5: Queremos que cuando el resultado de la condición sea verdadero realice un segmento de instrucciones, y cuando es falso se realicen otras operaciones. Con el mismo ejemplo (**$c = b/a$**), primero verificamos si la variable **a** es diferente de 0, si lo es se realizará la operación, y si no es diferente de cero nos muestra un mensaje. Observar que, se muestra el Verdadero y el Falso.

Proceso Deci_Doble

Escribir "Dame valor de b: ";

Leer b;

Escribir "Dame valor de a: ";

Leer a;

Si ($a \neq 0$)

$c = b / a$;

Escribir "El resultado es: ", c;

Sino

Escribir "Error: No se puede realizar division";

FinSi

FinProceso

Actividad 1.4: Usando **PSelInt**, construir el pseudocódigo de los siguientes ejercicios de **Decisión**:

1. Pseudocódigo que solicite un número y nos diga si este es par o impar.
2. Pedir altura de dos estudiantes y nos diga si tienen la misma altura, sino que nos diga quién es más alto.
3. Solicitar tres números, y que nos diga si el tercer número capturado es igual a la suma de los otros dos números.
4. Pseudocódigo que solicite un número y si este es mayor o igual a 5 muestre el triple de este, de lo contrario que nos muestre la cuarta parte del número.
5. Pseudocódigo que solicite tres números y los muestre en pantalla ordenados de menor a mayor.
6. Encontrar el costo de un viaje solicitando la distancia a recorrer y el número de días de estancia. Sabiendo que si la distancia a recorrer es superior a 1500 Km y el número de días de estancia es superior a 7 se otorgara un descuento del 30%. El precio por km. es de \$40.00.
7. Elabore pseudocódigo que lea el Nombre, Edad, Sexo y Estado civil de una persona, se imprimirá la Edad si es una mujer casada, sino que imprima el Nombre.
8. Preguntar si se desea calcular el área de un Cuadrado, un Triángulo o de un Círculo y al final mostrar en pantalla el valor del área de la figura seleccionada. El número de variables necesarias para el Cuadrado son 2, para el Triángulo son 3 y para el Círculo son 2.

Estructuras repetitivas o iterativas

En ocasiones es necesario que el mismo conjunto de instrucciones se ejecute cierto número de veces, y esto depende del valor que tome la variable de control durante la ejecución del pseudocódigo.

Instrucción Para en PSeInt

La instrucción **Para** ejecuta a un grupo de instrucciones durante un determinado número de veces, se usa cuando ya conocemos el número de repeticiones. El formato para escribir esta instrucción en PSeInt es el siguiente:

```
Para variable ← inicio Hasta final Con Paso Incre Hacer  
    bloque de instrucciones;  
FinPara
```

En *variable* se guarda el valor de *inicio* y en *final* se especifica hasta que valor llegará. El *Incre* nos indica como irá cambiando el valor de *variable*. Luego aparece el bloque de instrucciones el cual se repetirá hasta que *variable* alcance el valor de *final*. Si se omite la cláusula **Con Paso**, la variable se incrementará automáticamente en 1. Además, el *Incre* no solo puede aumentar, también puede disminuir.

El siguiente ejemplo muestra doce veces el texto “Ejemplo de repetición usando Para”:

Proceso Ejem_Para1

Escribir "Ejemplo de repetición usando Para";

Para a ← 1 Hasta 12 Con Paso 1 Hacer

Escribir "Sistemas Computacionales";

FinPara

FinProceso

Con el mismo ejemplo anterior, observemos que al cambiar el incremento a 2 el texto solo aparece 6 veces:

Proceso Ejem_Para_2

Escribir "Ejemplo de repetición con Para";

Para a ← 1 Hasta 12 Con Paso 2 Hacer

Escribir "Sistemas Computacionales";

FinPara

FinProceso

Instrucción Mientras en PSeInt

Ejecuta un conjunto de instrucciones mientras la condición sea verdadera. Aquí la condición es evaluada al principio y si es verdadera se ejecuta el bloque de instrucciones, lo cual repetirá hasta que la condición sea falsa.

Si al evaluar por primera vez la condición resulta ser falsa las instrucciones del ciclo no se ejecutarán nunca; también si la condición siempre es verdadera se produce un ciclo infinito, por lo que es importante tener cuidado con las variables de control en este tipo de estructuras.

Ejemplo 1.6: Se quiere imprimir un mensaje mientras que el valor de la variable x sea menor o igual a 20. Observe como x es inicializada antes de evaluar la condición y además dentro del ciclo se especifica cómo debe ir cambiando (aquí va incrementando en uno).

Proceso Ejem_Mientras1

Escribir "Ejemplo repetición usando Mientras";

$x=4$;

Mientras $x \leq 20$ Hacer

Escribir "Grupo de ISC de primero";

$x=x+1$;

Fin Mientras

FinProceso

Actividad 1.5: Realizar siguientes ejercicios con **PSelInt**, (con estructuras **Para** y **Mientras**).

1. Pseudocódigo que lea 8 números e indicar cuánto es la suma de todos ellos.
2. Pseudocódigo que solicite cuantos números enteros quiere capturar (**n**), enseguida teclear esos **n** valores y al final nos muestre cuantos fueron mayores a 200.
3. Pseudocódigo que permita leer el valor de 20 números, y nos indique cuantos de estos fueron positivos y cuantos fueron negativos.
4. Pseudocódigo que lea 10 números e indicar cuál es la suma de los números pares.
5. Leer **n** valores enteros hasta que se ingrese el número 250, y mostrar el número de valores que fueron ingresados.
6. Construya un pseudocódigo que permita ingresar 15 calificaciones que pueden ser entre 0 y 100, y nos muestre finalmente cuántos alumnos aprobaron (70 o más).
7. Pseudocódigo que permita leer una cantidad variable de números hasta que se ingresen 3 números negativos. Indicar cantidad total de números leídos.
8. Leer la edad y el peso de 15 alumnos del grupo. Mostrar cual es la edad promedio y el peso promedio.

1.4 Diseño de funciones

Cuando un algoritmo es más complejo o requiere de un gran número de instrucciones, se recomienda descomponerlo en partes más pequeñas llamadas **funciones** (módulos o subrutinas). Cada una de las funciones puede tener sus entradas y salidas para contribuir a la solución del proceso principal. A cada función se le asigna un identificador y está conformada por un bloque de instrucciones, y cada vez que queramos usarlas solo tendremos que "invocar" o "llamar" a la función por medio de su identificador.

Algo muy importante aquí son los argumentos o parámetros, los cuales permiten la entrada de valores a la función y trabajar con ellos. Para invocar (llamar) a una función solo tendremos que escribir el nombre de ésta incluyendo sus argumentos.

Características generales de funciones:

- Una función puede o no recibir argumentos.
- El tipo de dato de los argumentos en la función debe coincidir con el tipo de dato enviado.
- Puede o no regresar un valor, denominado resultado o valor de retorno.

Por lo pronto, nos interesa sólo conocer el concepto general de función ya que el tema es tratado de una manera más extensa en el capítulo 5. Por lo tanto, analicemos los siguientes ejercicios con PSeInt:

Ejemplo 1.7: Se quiere un pseudocódigo que contenga una función que realice la división de dos números capturados por el usuario.

Funcion $r1 \leftarrow \text{Division}(a, b)$

$r1 = a/b;$

FinFuncion

Proceso Principio

Escribir "Dame valor de a: ";

Leer a;

Escribir "Dame valor de b: ";

Leer b;

Escribir "----- ";

Escribir "El resultado es: ", Division(a, b);

FinProceso

En el ejemplo 1.7 aparece el proceso principal llamado **Principio** y ahí es donde se capturan los dos datos que se desea dividir, enseguida se invoca a la función llamado **Division** enviándole los dos valores capturados (**a** y **b**). Cuando **Division** recibe estos valores los asigna respectivamente a las variables **a** y **b** para trabajar con ellos. Al realizar la operación se guarda el resultado en **r1** y es regresado al proceso **Principio**, específicamente al lugar donde fue invocada la función **Division**, y por último el valor es mostrado. Cada vez que en el algoritmo nos encontremos el nombre de función y sus argumentos, la función se ejecutará y nos regresará un resultado.

Ejemplo 1.8: Se necesita capturar 2 valores y estos deben ser sumados, restados y multiplicados, mostrando el resultado obtenido. Enseguida estos tres resultados serán sumados y se mostrará el resultado final.

Funcion $r1 \leftarrow \text{Suma}(a, b)$

$r1 = a + b;$

FinFuncion

Funcion $r2 \leftarrow \text{Resta}(a, b)$

$r2 = a - b;$

FinFuncion

Funcion $r3 \leftarrow \text{Multiplica}(a, b)$

$r3 = a * b;$

FinFuncion

Proceso Principio

Escribir "Dame valor de a: ";

Leer a;

Escribir "Dame valor de b: ";

Leer b;

Escribir "La suma es: ", Suma(a,b);

Escribir "La resta es: ", Resta(a,b);

Escribir "Y multiplicacion: ", Multiplica(a,b);

SumaTotal = Suma(a, b) + Resta(a, b) +

Multiplica(a, b);

Escribir "La suma total es: ", SumaTotal;

FinProceso

En el ejercicio 1.8 aparece el proceso principal llamado **Principio** el cual captura valores en dos variables (**a** y **b**), y participan otras tres funciones llamadas **Suma**, **Resta** y **Multiplica** a las cuales se les envían 2 argumentos y cada una nos regresa un resultado que será usado en la suma total.

Actividad 1.6: Para aplicar el uso de funciones usando **PSelInt**, resolver los problemas planteados a continuación:

1. Algoritmo que contenga una función para calcular la edad de una persona, solicitando en el proceso principal el año actual y el año de nacimiento.
2. Algoritmo que contenga una función que calcule la calificación final de un estudiante donde el 70% equivale al examen y 30% a prácticas. En proceso principal se capturan las calificaciones y se envían a la función para que nos regrese la calificación final.
3. Existen dos tipos de empleados (1 y 2), calcular el total a pagar de un empleado solicitando las horas trabajadas y el sueldo por hora. Deberá aparecer una función para el empleado tipo 1 (se le descuenta el 7%) y otra para el empleado tipo 2 (se descuenta el 10%).
4. Dado el precio de un artículo y la cantidad de artículos comprados, calcular el total a pagar considerando que se retiene el 12% por concepto de impuesto. Deberá existir una función que calcule y regrese el total de la compra, y en el proceso principal se obtenga total a pagar.

Rúbrica de evaluación ejercicios del capítulo 1

	Excelente 10-8	Bueno 7-4	Deficiente 3-1	No realizado 0	Total
Estructura	Emplea de forma correcta la estructura de diagrama de flujo y/o pseudocódigo. -Inicio -Cuerpo -Simbología adecuada -Fin	Emplea la estructura de diagrama de flujo y/o pseudocódigo. -Inicio -Cuerpo -Simbología adecuada -Fin (omite al menos uno de los puntos anteriores)	Emplea parcialmente la estructura de diagrama de flujo y/o pseudocódigo. -Inicio -Cuerpo -Simbología adecuada -Fin (omite al menos dos de los puntos anteriores)	No presenta estructura de diagrama de flujo y/o pseudocódigo	
Solución a problemática	Propone soluciones haciendo uso de razonamiento e interpretación lógica. -Se entienden claramente las ideas de solución. -El proceso de solución genera los resultados deseados. -Sigue un orden establecido.	Propone soluciones, parcialmente, omite al menos uno de los puntos siguientes: -Se entienden claramente las ideas de solución. -El proceso de solución genera los resultados deseados. -Sigue un orden establecido	Propone soluciones deficientes, omite dos de los puntos siguientes: -Se entienden claramente las ideas de solución. -El proceso de solución genera los resultados deseados. -Sigue un orden establecido	No da solución a las problemáticas presentadas	
Presentación	-Respeta la sintaxis de diagramas de flujo y/o pseudocódigo. -La presentación	-Respeta, parcialmente, la sintaxis de diagramas de flujo y/o pseudocódigo. -La presentación	-Respeta, parcialmente, la sintaxis de diagramas de flujo y/o pseudocódigo. -Debe mejorar la	No presenta el diagrama y/o pseudocódigo	

Capítulo 1. Diseño algorítmico

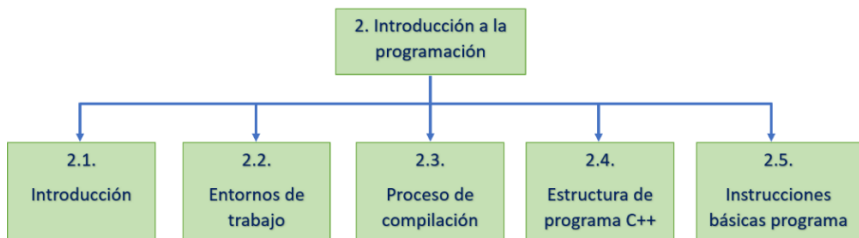
	es limpia y ordenada.	es limpia y ordenada.	presentación respecto a orden y limpieza.		
Comentarios					

Capítulo 2: Introducción a la programación

Objetivos de capítulo 2:

- Conocer los diferentes elementos necesarios para la construcción de programas, utilizando el lenguaje de programación C++.
- Presentar las diferencias entre procesador de texto y editor de texto, para conocer cuál es el que nosotros necesitamos para la construcción de programas.
- Mencionar distintos entornos de trabajo que nos ayudará a la construcción de programas, y seleccionar alguno de éstos para trabajar.
- Presentar los procesos de compilación desde un programa fuente hasta convertirse en un programa que se pueda ejecutar en la computadora.
- Conocer la estructura fundamental de un programa en el lenguaje C++. Así como las instrucciones básicas (entrada, salida, proceso) que se pueden incluir en un programa.

Subtemas a trabajar en capítulo 2:



Comentarios y recomendaciones:

Analicemos los siguientes comentarios y recomendaciones relacionados con la programación, para alcanzar nuestro objetivo de una mejor manera:

- Cualquier aprendizaje se logra paso a paso, de ahí la importancia de los conceptos básicos planteados en los temas anteriores.
- El peor error que se comete es empezar directamente con un lenguaje de programación, ya que primero debes entender los conceptos que existen en el ambiente de la programación.
- La mayoría de los lenguajes de programación trabajan con los mismos conceptos, esto significa que, si ya conocemos las bases podremos aprender a programar.
- La programación consiste en organizar o manipular datos, por lo que al programar nosotros le decimos a la computadora que es lo que debe realizar con ellos.
- La programación al igual que otras especialidades tiene sus dificultades, por lo que depende mucho del tiempo que el estudiante le dedique a esta práctica.
- Recuerda que primero debes intentar tú sólo encontrar una posible solución al problema a resolver, si después de varios intentos no lo logras, entonces pide ayuda.

2.1 Introducción

Como hemos mencionado hasta ahora, de manera general la programación consiste en manipular datos o información, y que normalmente un **dato** es un valor que recibe la computadora y que este es procesado para generar un resultado como información completa y útil para el usuario. Entonces podemos decir que el

objetivo principal de los sistemas computacionales es saber tratar y organizar correctamente a los datos, para que se cumpla con las necesidades que el cliente esté solicitando.

Cualquier problema que se puede solucionar por medio de computadora creando aplicaciones informáticas, nos llevará a la tarea de **programación**. Pero para esto es necesario tener definidos los datos de entrada que serán ingresados al programa, las operaciones que se van a realizar con estos datos (proceso) y, los datos que el programa nos arrojará como salida. De manera general, después de realizar un análisis del problema a resolver, este nos llevará al entendimiento de qué es lo que el cliente está solicitando; entonces podremos identificar los datos necesarios, procediendo con el diseño de la solución por medio de un programa.

Ya hemos trabajado la forma de solucionar problemas aplicando los conceptos de algoritmos y sus características, así como la utilización del software **Free-DFD** para la construcción de diagramas de flujo y **PSelnt** para crear pseudocódigos. Ahora vamos a empezar a construir **programas** usando algún lenguaje de programación que nos permita expresarlo por medio de instrucciones (código). Por mencionar algunos ejemplos de lenguajes de programación, tenemos a Visual Basic, Pascal, Logo, C++, JavaScript, Python, entre muchos más.

En primer lugar, es necesario seleccionar un entorno de trabajo adecuado para la construcción de programas. Aquí nos referimos a que es importante conocer los ambientes y herramientas para el desarrollo y ejecución de programas, como pueden ser los **editores**, **compiladores** y **sistemas operativos**, que nos ayudarán a crear, modificar, compilar y ejecutar programas.

El **sistema operativo**, es un conjunto de programas que administran los recursos de un sistema de cómputo y, funciona como interfaz entre la computadora y el usuario o aplicaciones. Existen diversos sistemas operativos, por ejemplo, para equipos de cómputo se pueden mencionar Windows, Linux y Mac OS, y para los dispositivos móviles están IOS, Android y Windows Phone.

Hablando de **ambientes de trabajo**, podemos decir que existen de tipo texto y de tipo gráfico o visual, donde esta última es más utilizada.

En cuanto a los **editores** para crear programas, primero vamos a diferenciar los conceptos de editor y procesador de texto para evitar confusiones.

Editor de texto, es un editor de programas que se centran casi exclusivamente en el propio texto. Están diseñados para mantener a sus usuarios centrados en el contenido puro de lo que están escribiendo, ofreciendo poco o ningún medio para cambiar el formato o el estilo visual de los contenidos; es conocido como texto “sin formato” o “texto plano”. Los programadores aprecian los editores de texto simples, ya que la falta de formato es propicio para escribir en múltiples lenguajes de programación. Algunos ejemplos son: Notepad++, Atom, Brackets, Sublime Text, Vim o Emacs.

Un **Procesador de texto** está destinado como herramienta primaria para escribir y dar formato a un texto. Se permite trabajar con formatos específicos para ser impresos o leídos por otros programas de procesamiento de textos. Las herramientas de formateo en procesadores de texto incluyen opciones para impresión y salida digital, con soporte para insertar hipervínculos, imágenes, tablas, etc. Algunos ejemplos pueden ser: Microsoft

Word, Word Perfect, Word Pad, Lotus Word Pro, Quick office, Docs to Go, Open office.

Entonces, un archivo fuente debe ser escrito en un **editor de texto** para que después el compilador genere un programa que sea ejecutable en la computadora. Y los factores que se deben de tomar en cuenta a la hora de elegir el entorno para desarrollar programas son: el editor de texto, el compilador, el sistema operativo y si posee o no interfaz gráfica.

El **compilador** es un programa que participa en la traducción de un programa escrito en algún lenguaje de programación (instrucciones) a otro lenguaje distinto (lenguaje maquina), para que sea entendible y ejecutado por la computadora.

2.2 Entornos de trabajo

Al hablar de un entorno de trabajo de programación (IDE) nos referimos a un conjunto de herramientas de software que nos ayudan y facilitan la actividad de programar. Este entorno nos ofrece un soporte en la preparación de programas, tanto en la codificación como en las pruebas de estos. En la actualidad existen diferentes entornos de trabajo que nos ofrecen otras funciones además de sólo manipulación del código fuente, y esto nos facilita la tarea de programación.

Debido a la existencia de distintas metodologías de programación, esto también nos ha llevado a diversas técnicas y herramientas que nos ayuden a escribir los programas. Algunas de ellas se han incorporado a los entornos de programación integrados, y otras están soportadas mediante herramientas independientes. Algunos entornos de programación son específicos para un lenguaje de

programación, mientras que otros combinan piezas de código fuente escritas en diferentes lenguajes de programación.

Ya que en nuestro curso trabajaremos con el lenguaje de programación **C++**, enseguida mencionamos algunos de los entornos para poder trabajar y crear programas:

Eclipse: Es uno de los entornos más conocidos y utilizados por los programadores. Sirve para Java, C++, PHP, Perl y otros.

Netbeans: El lenguaje que mejor soporta es Java, ya que fue creado por Oracle y su creación fue para ser el IDE de Java. Aunque soporta JavaScript, HTML5, PHP, C/C++, entre otros.

Microsoft Visual Studio: Se trabaja con el sistema operativo Windows y además soporta múltiples lenguajes de programación, tales como C++, C#, Visual Basic .NET, F#, Java, Python, Ruby y PHP, al igual que entornos de desarrollo web.

JetBrain: Crean entornos para varios lenguajes como Java, Ruby, Python, PHP, SQL, Objective-C, C++, JavaScript y C#.

CodeLite: Es para diversos sistemas operativos, soporta los lenguajes C/C++, PHP y Node.js

Dev++

Es un entorno de desarrollo integrado (IDE) para programar en lenguaje C/C++. El Entorno está desarrollado en el lenguaje Delphi de Borland.

Borland C++

Es un entorno de desarrollo integrado de C y C++ para sistemas operativos MS-DOS y Microsoft Windows. Fue sucesor de Turbo C++ e incluyó un mejor depurador, el Turbo Debugger.

Otros entornos de desarrollo para C y C++ que se pueden obtener de forma gratuita son: Ultimate++, Zinjai, CodeLite, Code Blocks, Adjunta DevStudio, Geany, Kdevelop, Monodevelop, Qt Creator, Sun Studio, WxDev-C++, Borland C++, Xcode

Forma de trabajar durante el curso:

En cuanto al sistema operativo, sabemos que uno de los más utilizados es Windows, por lo que la mayoría de los clientes estarán solicitando aplicaciones para trabajar en Windows. Aunque existen otros sistemas operativos que se pueden utilizar.

Hablando de los IDE, se dice que no existe uno perfecto. Normalmente éste se selecciona de acuerdo con el tipo de aplicación a desarrollar, pero en nuestro caso no es imprescindible, ya que, con un editor de texto, simple o potente, más el compilador y enlazador, podríamos programar.

Hablando de compiladores para C++, en el curso podemos trabajar con cualquiera de siguientes:

- **El Visual Studio:** Es un entorno de desarrollo que soporta múltiples lenguajes de programación.
- **Borland C++:** De manera general, el Borland C++ tiene más prestaciones, pero se necesita licencia (no es gratuito).
- **Dev C++:** Este es totalmente gratuito y nos da un entorno de trabajo fácil y amigable.

Para poder empezar a trabajar con la construcción de programas, a continuación, presentamos una vista general de cada uno de estos tres.

Introducción a Microsoft Visual Studio

Cuando se requiere crear algún programa o aplicación, ya sea de escritorio, Windows Phone o aplicaciones web, podemos utilizar **Microsoft Visual Studio**, el cual es un entorno de desarrollo que soporta múltiples lenguajes de programación, tales como C++, C#, Visual Basic .NET, F#, Java, Python, Ruby y PHP, al igual que ASP.NET MVC y Django.

Algunas de las **características generales** de Microsoft Visual Studio pueden ser:

- Se pueden crear diferentes tipos de aplicaciones.
- Soporta varios lenguajes de programación.
- Permite crear aplicaciones para Windows en poco tiempo.
- Permite un desarrollo eficaz y menor inversión en tiempo y dinero.
- Se pueden generar librerías dinámicas de forma activa.

Soluciones y proyectos en Visual Studio

Una **solución** es un contenedor que se usa para organizar uno o más proyectos de código relacionados, por ejemplo, una biblioteca de clases y archivos, así como un proyecto correspondiente. Cuando se abre una solución en Visual Studio, esta carga automáticamente todos los proyectos que contiene.

Cuando se crea una aplicación en Visual Studio, debe comenzar con un **proyecto**. Un proyecto contiene todos los archivos de

código fuente, iconos, imágenes, archivos de datos y cualquier otra cosa necesaria para realizar la compilación en un programa ejecutable o sitio web. Un proyecto también contiene toda la configuración del compilador y otros archivos de configuración que podrían ser necesarios en diversos servicios o componentes con los que el programa se comunicará. Al crear un **proyecto**, Visual Studio crea automáticamente una **solución** para hospedar ese proyecto, esto si es que no existe una solución ya abierta.

Microsoft Visual Studio para lenguaje C++

Aquí mencionamos la secuencia de **pasos** a seguir para construir un primer **proyecto** en Microsoft Visual Studio con C++. Primero especificamos que se creará un proyecto con el nombre: Escuela, y con nombre de programa: Sistemas1.

Ejercicio 2.1:

1. Abra Visual Studio con cualquiera de las siguientes opciones:
 - Dar doble clic en **Icono** que aparece en el **Escritorio** de la pantalla: .
 - En **Inicio** → **Todos los programas** → **Microsoft Visual Studio** → Y seleccionamos **Microsoft Visual Studio 2010**.

Al momento de iniciar nos muestra el IDE general de desarrollo, que observamos en figura 2.1.

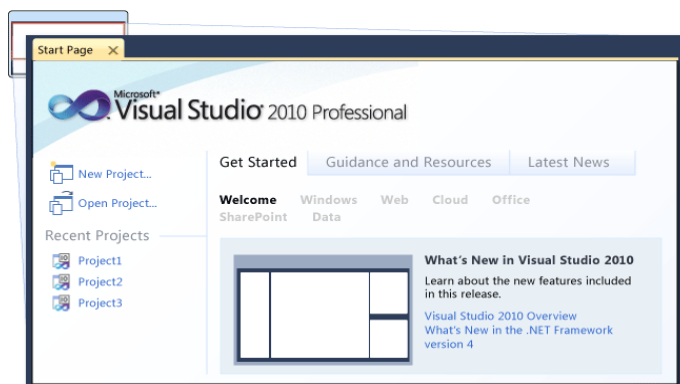


Figura 2.1. IDE con configuración general de desarrollo aplicado

Esta ventana puede variar, dependiendo de los ajustes que se hayan aplicado y las personalizaciones realizadas anteriormente. Observe que en la figura 2.1 nos da la opción de crear un proyecto nuevo (New Project) o de abrir uno ya existente (Open Project). Después de crear un proyecto nuevo, podremos observar un menú de opciones que aparece en la parte superior (ver figura 2.2).

Ejemplo - Microsoft Visual Studio

File Edit View Project Build Debug Team Data Tools Architecture Test Analyze Window Help

Figura 2.2. Menú de herramientas

En español: Archivo, Editar, Proyecto, Construir, Depurar, Grupo, Datos, Herramientas, Arquitectura, Prueba, Analizar, Ventana y Ayuda.

- Ahora iniciemos con el proyecto: Seleccionamos **File → New → Project** y entonces aparecerá el cuadro de dialogo **New project** (Nuevo proyecto).

3. En la ventana de New Project podemos observar **Installed Templates** (Plantillas Instaladas); enseguida seleccionamos el lenguaje (**Visual C++**) → **General** → **Proyecto vacío** (Empty Project).
4. En **Examinar** (Browse) podemos seleccionar la carpeta donde se va a guardar el archivo → **Seleccionar** y escribimos nombre del proyecto → **Aceptar**.

Después de crear el proyecto, aparecen varias ventanas de herramientas, así como en el espacio del editor. Las ventanas de herramientas, menús y barras de herramientas disponibles dependen del tipo de proyecto o archivo en el que esté trabajando

5. En ventana de **Explorador de soluciones**, escogemos con clic derecho: **Archivos de código fuente** (Source Files). En **Agregar** (Add) → **Nuevo elemento** (New Ítem). Y de las opciones seleccionamos C++ File (.cpp). Esto nos indica que vamos a crear un programa con extensión **.cpp**. Vea figura 2.3.

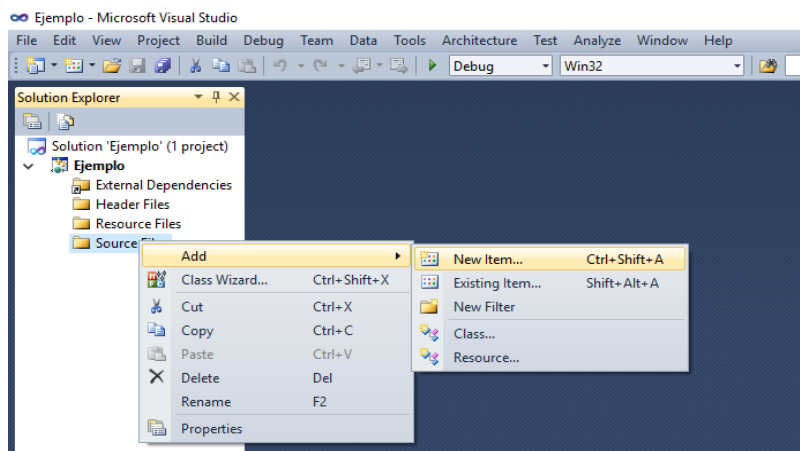


Figura 2.3. Ejemplo de Explorador de soluciones

6. Escribimos el nombre del archivo y clic en **Agregar** (Add). Observe figura 2.3.
7. Aparece una ventana en blanco para escribir las instrucciones de nuestro programa, además aparece en parte superior izquierda el nombre del programa con extensión .cpp.
8. Una vez escrito, lo vamos a compilar y ejecutar presionando **F5** y después seleccionar **Sí**.

Notas:

- Cuando ya tenemos programas existentes los podemos cargar de la siguiente manera:

En **File** → **Open** → **Project/Solution**, Seleccionamos la carpeta del proyecto y aparecen algunas carpetas y el archivo de Tipo **Microsoft Visual Studio Solution** → Damos doble clic en este último y aparecerá en la ventana de **Solution Explorer** → Seleccionamos el programa y se carga en la ventana de **Global Scope**.

- Los proyectos y archivos que son creados se almacenan por default en la siguiente carpeta:

c:\users\laptop\documents\visual studio 2010\Projects\.

Por ejemplo:

Un archivo de código fuente se almacena en:

c:\users\laptop\documents\visualstudio2010\Projects\Ejercicio2\Ejercicio2\Ejercicio2.cpp

Y el archivo ejecutable o identificado como la aplicación se almacena en:

c:\users\laptop\documents\visualstudio2010\Projects\Ejercicio2\Debug\Ejercicio2.exe

A continuación, se presentan ejemplos sólo para ver cómo trabaja el Visual Studio 2010, no importa que no se entiendan por completo, eso lo detallaremos más adelante.

// Ejemplo 2.1: Programa escrito en Visual Studio C++

```
#include<iostream>
using namespace std;
int main()
{
    cout<<"-----"<<endl;
    cout<<"    Sistemas    "<<endl;
    cout<<"    Hola Mundo    "<<endl;
    cout<<"-----"<<endl;
    cout<<endl<<endl;
    system("pause");
    return 0;
}
```

Una vez escrito el programa (Archivo fuente) lo podemos verificar presionando la tecla **F5** la cual nos sirve para compilar o detectar posibles errores y posteriormente ejecutar el programa en la computadora.

// Ejemplo 2.2: Programa escrito en Visual Studio C++

```
#include<iostream>
#include<conio.h>
using namespace std;
int main()                // funcion principal
{                          // Inicio de funcion
    cout<<" \t \t Insttuto Tecnologico \n ";
    cout<<" \t \t Pulsa tecla ..... ";
    getch();
    cout<<" \n \n \t Estamos en el capitulo 2 "<<endl;
    cout<<"-----"<<endl;
    cout<<endl;
    system("pause");
    return 0;
}                          // Final de funcion
```

Introducción a Borland C++

Este IDE es derivado de C++ y desarrollado por la compañía Borland Software Corporation. Cuando iniciamos nos aparecerá una ventana parecida a la que se muestra en figura 2.4 (puede variar según la configuración y versión utilizada).

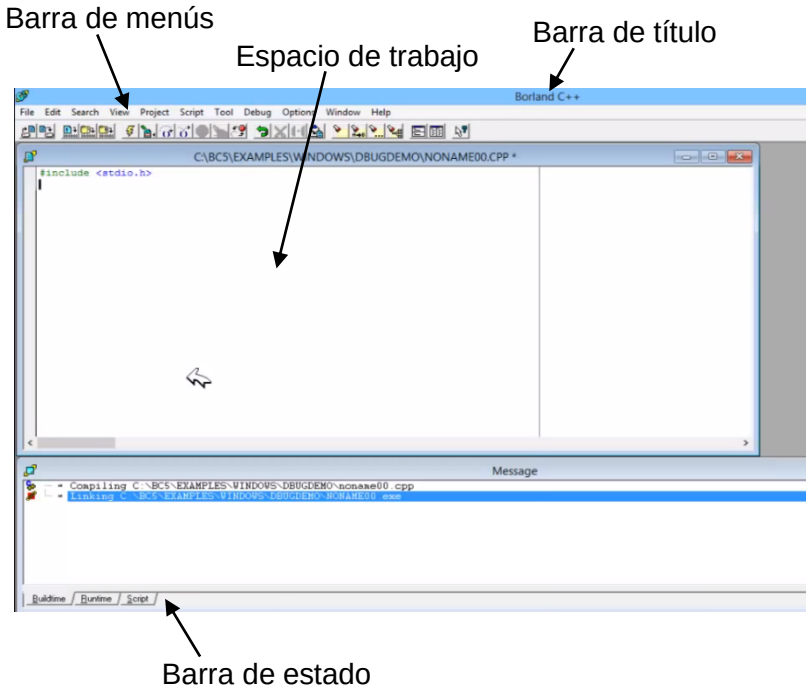


Figura 2.4. Ventana inicial de Borland C++

Al igual que otros, nos permite crear programas en el menú **File**, seguida la opción **New** y **Text Edit** (observe la siguiente figura 2.5). Al realizar estos pasos se creará un archivo nuevo en el cual se escribirán todas las instrucciones del programa. Una vez terminado es recomendable guardar o almacenar el programa (archivo fuente) para después poderlo compilar.

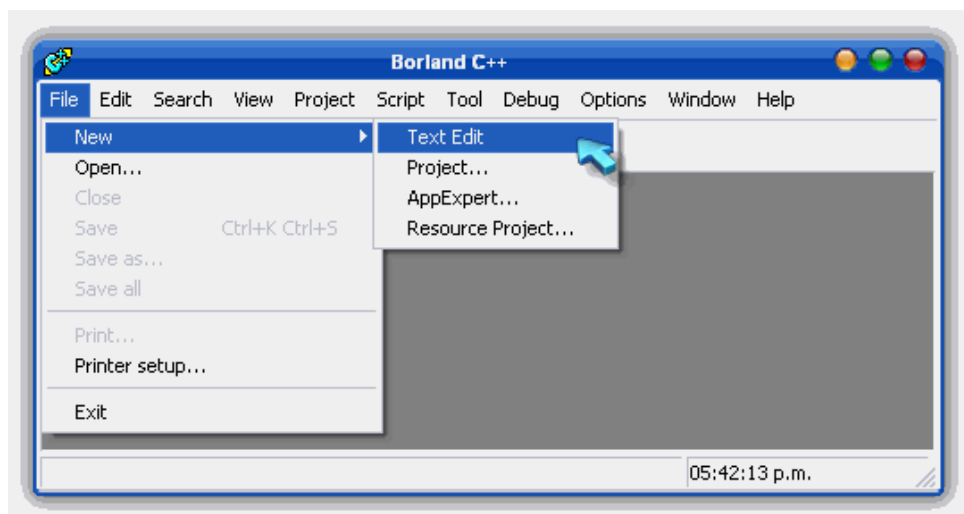


Figura 2.5. Ventana al crear archivo nuevo en Borland C++

La compilación se realiza con **Alt+F9** o con el botón . Si el compilador no encontró ningún problema a la hora de compilar, entonces se genera el programa ejecutable con la extensión **exe**. Para ejecutar el programa solo necesitamos utilizar el botón de **Run** o bien con **Ctrl+F9**. Veamos el siguiente ejemplo para probar cómo funciona:

/ Ejemplo 2.3: Programa escrito en Borland C++ que muestra dos mensajes */*

```
#include<iostream.h>
#include<conio.h>
void main(void)
{ cout<<"\n Se parece al Hola mundo \n";
  cout<<"\n Vamos a pulsar una tecla para terminar";
  getch();
}
```

Enseguida se muestra otro ejemplo para probar cómo funciona:

```
/* Ejemplo 2.4: Programa escrito en Borland C++ que
muestra mensajes */
#include<iostream>
#include<conio.h>
using namespace std;
int main(){           // Inicio de funcion main()
    cout<<" \t \t Instituto Tecnológico \n ";
    //Instruccion de salida cout
    cout<<" \t \t Pulsa tecla ..... ";
    getch();
    cout<<" \n \n \t    Estamos en la Unidad 2 "<<endl;
    cout<<"-----"<<endl;
    getch();
    return 0;
}                    // Final de funcion main()
```

Introducción a Dev C++

Al momento de arrancar con Dev C++ nos aparecerá una ventana en la cual podemos observar un menú de opciones y algunos botones auxiliares. Para crear un programa podemos acceder al menú **Archivo** y después seleccionar **Nuevo**. Entonces aparecerá el cursor en la ventana donde podremos escribir las instrucciones que formaran parte de nuestro programa, en la siguiente figura 2.6 lo podemos observar.

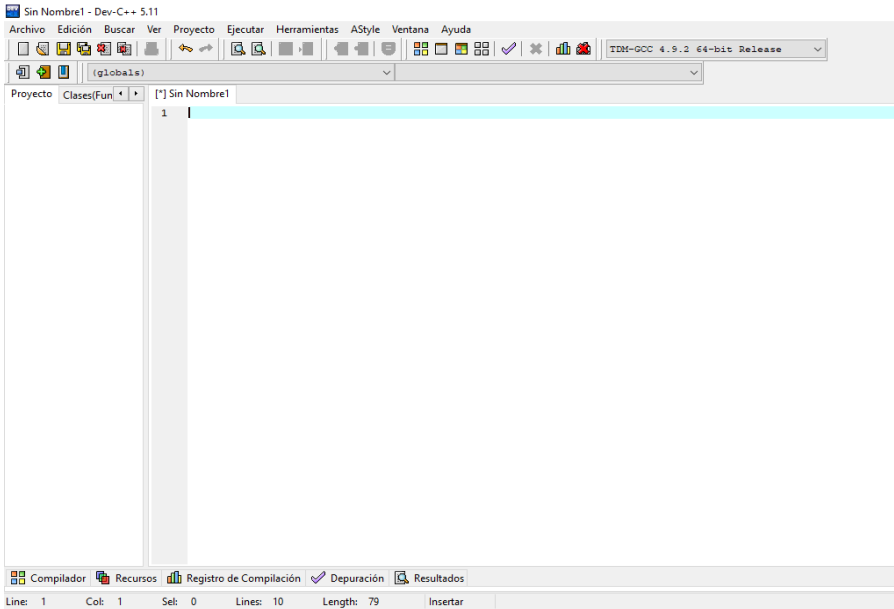


Figura 2.6. Ventana al crear archivo nuevo

Aquí podremos escribir cada una de las instrucciones de un programa, siguiendo las reglas y sintaxis que nos indica el lenguaje de programación.

Una vez terminado el programa, lo podremos compilar pulsando la tecla **F9** y después con la tecla **F10** lo podemos ejecutar. Recuerda que aquí solo estamos probando de manera general cómo funciona el Dev C++, después explicaremos las instrucciones de programa de forma detallada.

Para iniciar vamos a escribir los siguientes ejemplos de programa:

```
// Ejemplo 2.5: Programa que muestra dos mensajes
#include<iostream>
using namespace std;
int main(){
    cout<<" Hola Mundo"; //Instruccion de salida
    cout<<" \n Hasta pronto.... ";
    return 0;
}
```

```
/* Ejemplo 2.6: Solicita un valor y despues lo muestra
en pantalla en Dev C++ */
#include<iostream>
using namespace std;
int main()
{
    int x;
    cout<<"Dame un valor entero: ";
    cin>>x;
    cout<<"\n El valor tecleado es: "<<x<<endl;
    return 0;
}
```

Notas:

- Con **Ctrl-N** se crea un archivo nuevo fuente.
- Con **Ctrl-O** se abre un archivo fuente ya existente.
- Del lado izquierdo de cada línea aparece el número de línea.

- Al momento de pulsar **F9** se pueden detectar errores, si estos aparecen es necesario localizarlos y realizar los cambios necesarios para volver a compilar.
- Una vez que no existen errores, con la tecla **F10** se puede ejecutar el programa y aparecerá una ventana mostrando los resultados del programa escrito.

Otras opciones para crear programas en línea

También se puede trabajar con compiladores en línea para crear programas en C++ como los siguientes:

- <https://www.onlinegdb.com/>
- <https://www.programiz.com/cpp-programming/online-compiler/>
- <https://www.jdoodle.com/online-compiler-c++/>

2.3 Proceso de compilación

Hasta ahora solo probamos de manera general con tres entornos de desarrollo (IDE) diferentes, pero ¿En realidad qué sucede cuando compilamos o ejecutamos un programa?

Las instrucciones de un programa se escriben en un archivo de texto de acuerdo con las reglas definidas por el lenguaje utilizado, este archivo con extensión **.cpp** es conocido como el archivo fuente.

Posteriormente las instrucciones del archivo fuente serán revisadas y analizadas por un **compilador**, si las instrucciones están escritas de manera correcta se obtendrá como salida un archivo ejecutable con extensión **.exe**, el cual tiene instrucciones que ya pueden ser procesadas por la computadora.

En la siguiente figura 2.7 se muestra este proceso para convertir un archivo fuente (.cpp) a un archivo ejecutable (.exe).

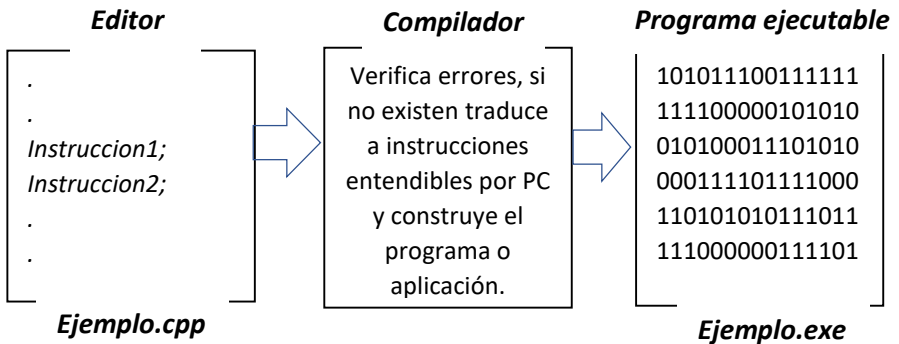


Figura 2.7. Proceso de Edición, compilación y ejecución de programa

Se pueden cometer errores al escribir las instrucciones, la mayoría de estos errores provocan que el compilador no pueda generar el programa ejecutable (mostrándonos los errores e indicando de qué tipo de error se trata).

La **ejecución** es la etapa que continua después de haber compilado un programa, y es donde podremos ver los resultados esperados del programa escrito, por ejemplo, solicitar algún dato al usuario o mostrar mensajes en pantalla. Entonces la ejecución es el intervalo de tiempo en el que un programa de computadora se lleva a cabo (se ejecuta). Un archivo ejecutable es un archivo

binario cuyo contenido se interpreta por el computador como un programa.

2.4 Estructura de un programa en Dev-C++

Ahora vamos a conocer la estructura de un programa en C++ y como deben ser escritas sus instrucciones, lo que nos ayudará a crear nuestros propios programas. Todos los programas tienen un **formato** específico, el cual no debe cambiar a la hora de escribir el programa. Cada lenguaje de programación establece su formato y el nombre de instrucciones válidas (palabras reservadas) para poder construir un programa de manera correcta. Un programa en C o C++ se compone de una o más funciones, de las cuales una se llama **main ()**, que es la función principal y es la primera que se ejecuta en el programa.

Entonces podemos decir que los programas en C++ deben seguir el siguiente **formato general**:

```
#include< >
variables y constantes globales;
tipo de dato main ( )
{
    variables locales;
    conjunto de instrucciones del programa;
    <llamadas a funciones;>
}
```

Explicación de elementos de la estructura de un programa:

- Hablando de la directiva de preprocesador **#include** esta se usa para incluir las declaraciones de otro archivo.

Cuando se encuentra una línea **#include <archivo>**, entonces reemplaza esta línea por el archivo especificado.

- Las llaves { } son los delimitadores para indicar cuál es el contenido del programa (instrucciones).
- En la parte de **variables** se pueden declarar el tipo y el nombre de las variables que vamos a utilizar, por ejemplo:
int a;
- Los puntos y comas (;) sirven para indicar el final de una instrucción.
- Todas las instrucciones se escriben en minúsculas.
- Al momento de imprimir en pantalla, existen las **secuencias de escape** que ayudan a darle formato a la información mostrada. Algunos ejemplos que podríamos usar son:

\n	Inserta un salto de línea.
\b	Carácter de retroceso.
\t	Tabulación horizontal.
\v	Tabulación vertical.
\\	Mostrará una barra invertida.
\"	Mostrará comillas dobles.
\a	Pita bocina.

Se pueden agregar comentarios en los programas, que sólo es información que no se ejecuta:

// Comentario en una sola línea

/* Estamos probando como se escriben los comentarios en programas del lenguaje C++, aquí estoy usando dos líneas para escribir este comentario ***/**

En un programa se pueden usar funciones existentes en el lenguaje o crear nuestras propias funciones, donde podremos hacer las declaraciones necesarias y escribir las instrucciones de ésta. Para agregar **funciones** en programas de C++, es necesario seguir un formato ya establecido, donde se especifique lo siguiente:

- El tipo de valor que va a regresar.
- El nombre de la función.
- Lista de argumentos que pudiera recibir entre paréntesis, si es necesario.

2.5 Instrucciones básicas en programa de C++

2.5.1 Instrucciones de entrada/salida

El lenguaje C++ cuenta con dos instrucciones básicas que nos sirven para introducir datos al programa y para mostrar los resultados obtenidos en el programa. Estas son “**cout<<**” y “**cin>>**”.

Lectura o entrada de datos en C++

A **cin** se le conoce como flujo de entrada estándar que normalmente es el teclado. La entrada de datos es cuando se le solicita un dato al usuario para que ese valor sea almacenado en alguna variable. Para poder realizar esto, se escribe la instrucción **cin** seguida de los caracteres **>>** y luego el **nombre** de la variable que estamos utilizando y al final el caracter punto y coma (;). Ejemplo:

```
cin>>nume;
```

Salida de datos en C++

El **cout** es el flujo de salida estándar que por lo general es la pantalla. Para poder usarla necesitamos escribir la instrucción llamada **cout** seguida de los caracteres << seguida de un mensaje, del *nombre* de variable o de alguna llamada a función, terminando con el caracter punto y coma (;). Para imprimir o mostrar en pantalla un mensaje se hace de la siguiente manera:

```
cout<<"Estoy estudiando";
```

Para que estas instrucciones se puedan utilizar es necesario incluir el archivo de cabecera de entrada/salida llamado *iostream*. Es decir, si voy a utilizar *cin* o *cout* en mi programa debo incluir este archivo:

```
#include<iostream>
```

Regresemos a los ejemplos contruidos en tema 2.2 para observar que las instrucciones cumplan con la estructura de un programa en C++.

2.5.2 Tipos de datos en C++

Un **tipo de dato** es la propiedad de un valor que determina su dominio, esto es, qué valores puede tomar, por lo que todas las variables que aparecen en un programa deberán tener un tipo de dato específico. Los tipos de datos se clasifican en primitivos y derivados. Los tipos primitivos son los que ya están definidos dentro del lenguaje y los tipos de datos derivados se forman a

partir de los tipos primitivos. El formato para definir variables y especificar el tipo de valor que almacenaran es el siguiente:

[Tipo dato] <identificador1>, <identificador2>...;

Tipo de dato numérico entero

Estos tipos de datos son un subconjunto finito de los números enteros del mundo real. Los valores pueden ser positivos o negativos, por ejemplo -3, -2, -1, 0, 1, 2, 3,

Tipo de Dato	Descripción	Número de bytes	Rango
Short	Entero corto	2	-32768 a 32767
int	Entero	4	-2147483648 a +2147483647
Long	Entero largo	4	-2147483648 a +2147483647
Char	Carácter	1	-128 a 127

Tabla 2.1 Datos tipo entero en C++.

También existen ciertos modificadores, que permiten ajustar ligeramente ciertas propiedades de cada tipo; los modificadores pueden ser: **short**, **long**, **signed** y **unsigned**. Observe los siguientes ejemplos de declaración de variables de tipo entero:

```
int prt;  
int num1=15;  
unsigned int a;  
int b=23, d=14, j=56;
```

Tipos numéricos reales

Es un subconjunto finito de los números reales, y pueden ser positivos o negativos. Normalmente se identifican con la palabra **float** que viene del término punto flotante, que es la manera en el que el computador representa internamente los números reales. Literalmente, los números reales se escriben separando la parte entera de la parte decimal con un punto. Además, las partes entera y decimal pueden ser omitidas si alguna de ellas es cero. En C++ los tipos de datos reales son:

Tipo de Dato	Descripción	Número de bytes	Rango
Float	Real (Número en coma flotante)	4	Positivos: 3.4E-38 a 3.4E38 Negativos: -3.4E-38 a -3.4E38
Double	Real doble (Número en coma flotante de doble precisión)	8	Positivos: 1.7E-308 a 1.7E308 Negativos: -1.7E-308 a -1.7E308
long double	Real doble largo	10	Positivos: 3.4E-4932 a 1.1E4932 Negativos: -3.4E-4932 a -1.1E4932

Tabla 2.2 Datos tipo real en C++.

Ejemplos de declaración de variables de tipo flotante:

```
float prueba=58.25;  
float no1=12.57;  
float a=23.33, d=14.63, j=5.667;  
double mayor = 15255.345788;
```

Tipo de dato char

Es de tipo alfanumérico, es decir que puede almacenar un caracter, un dígito numérico o un signo de puntuación. Este tipo contiene un único carácter del código ASCII. El tamaño de memoria es de 1 byte. Ejemplos de declaración de variables de tipo char:

```
char Letra;  
char car2, car3, car4='a';  
char Let='5';
```

Tipo de dato lógico

Los valores lógicos son de tipo **bool**. Las variables de este tipo sólo pueden tomar dos valores **true** (verdadero) o **false** (falso). Sirven para evaluar expresiones lógicas, donde el valor true toma el valor 1 y false el valor 0. Por lo tanto, los únicos valores que pueden contener son 0 o 1. Algunos ejemplos de declaración de variables de tipo booleano son:

```
bool mayor;  
bool resp;
```

Vamos a crear los siguientes ejemplos básicos y de tipos de datos:

```
// Ejemplo 2.7: Mostrar mensajes en pantalla  
#include<iostream>  
using namespace std;  
int main()  
{  
    cout<<"Vamos a echarle todas las ganas "<<endl;  
    cout<<"por eso estamos aqui \n ";  
    system("pause");  
    return 0;
```

```
}
```

```
// Ejemplo 2.8: Mostrar mensajes en pantalla
#include<iostream>
using namespace std;
int main()
{
    cout<<"Escribe lo que quieras "<<endl;
    cout<<"Para mostrar mensajes en pantalla \n"<<endl;
    system("pause");
    return 0;
}
```

```
/* Ejemplo 2.9: Se definen distintos tipos de datos y
se les asigna un valor */
#include<iostream>
using namespace std;
int main()
{
    int entero=18;
    float flotante=39.98;
    double mayor=189.4586;
    char letra='k';
    cout<<entero<<endl;
    cout<<flotante<<endl;
    cout<<mayor<<endl;
    cout<<letra<<endl;
    system("pause");
}
```

```
    return 0;
}
```

//Ejemplo 2.10:Asignando valores a variables y mostrar

```
#include<iostream>
using namespace std;
int main()
{
    int entero=18;
    float flotante=54.89;
    double mayor=528.89897;
    char caracter='1';
    cout<<"El entero es: "<<entero<<endl;
    cout<<"El flotante es: "<<flotante<<endl;
    cout<<"El mayor valor doble es: "<<mayor<<endl;
    cout<<"El caracter es: "<<caracter<<"\n";
    system("pause");
    return 0;
}
```

Realicemos los siguientes ejercicios para usar la lectura de datos:

/* Ejemplo 2.11: Se define variable, se captura un valor y lo muestra en pantalla */

```
#include<iostream>
using namespace std;
int main() {
    int nume;
    cout<<"Dame un numero: ";
    cin>>nume;
```

```
    cout<<"El numero que escribio es: "<<nume<<endl;
    system("pause");
    return 0;
}
/* Ejemplo 2.12: A una variable se asigna valor y a
otra se captura y los muestra */
#include<iostream>
using namespace std;
int main()
{
    int nume;
    float equis=25.98;
    cout<<"Dame el numero entero: ";
    cin>>nume;
    cout<<"El numero entero que escribio es: "
    <<nume<<endl;
    cout<<"El numero flotante es: " <<equis<<endl;
    system("pause");
    return 0;
}
```

2.5.3 Expresiones con Dev C++

Una **expresión** se forma de una secuencia de operandos y operadores escrita bajo ciertas reglas. Los operadores indican determinadas operaciones a realizar con las variables especificadas.

Un ejemplo de una expresión de suma podría ser la siguiente:

$$A=b*c+2$$

Enseguida se listan distintos operadores que se pueden usar en las expresiones:

Operadores aritméticos binarios:

+	Suma
-	Resta
*	Multiplicación
/	División
%	Modulo o resto

Operadores aritméticos unarios:

++	Incremento en 1 (suma 1)
--	Decremento en 1 (resta 1)
-	Cambio de signo

Operadores de asignación:

=	Asignación simple
+=	Suma
-=	Resta
*=	Multiplicación
/=	División
%=	Módulo o resto

Operadores relacionales:

>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que

== Igual que
!= Distinto de

Operadores lógicos:

&& AND (y lógico)
|| OR (o lógico)
! NOT (el valor contrario)

Revisemos los siguientes ejemplos para ver qué es lo que realizan:

```
/* Ejemplo 2.13: Lee 2 datos enteros y hace suma,
resta, multiplicacion y division */
#include<iostream>
using namespace std;
int main()
{
    int n1,n2, suma=0, resta=0, multip=0;
    float division=0;
    cout<<"Dame el primer numero: "; cin>>n1;
    cout<<"Dame el segundo numero: ";
    cin>>n2;
    suma=n1+n2;
    resta=n1-n2;
    multip=n1*n2;
    division=n1/n2;
    cout<<"\nLa suma es: "<<suma;
    cout<<"\nLa resta es: "<<resta;
    cout<<"\nLa multiplicacion es: "<<multip;
```

```
    cout<<"\nLa division es: "<<división<<endl;
    system("pause");
    return 0;
}
```

// Ejemplo 2.14: Usando operador de decremento en 1

```
#include<iostream>
using namespace std;
int main() {
    int x=0, a=5;
    x--a; // Decrementa en 1 al valor de a
    cout<<"Valor final de x es: "<<x<<endl;
    system("pause");
    return 0;
}
```

// Ejemplo 2.15: Se muestra operador abreviado de suma

```
#include<iostream>
using namespace std;
int main() {
    int suma,a=5;
    a+=4; // Equivale a a=a+4
    cout<<"Valor final: "<<a;
    system("pause");
    return 0;
}
```

2.5.4 Cadenas

A partir de los tipos de datos primitivos o básicos analizados anteriormente se pueden formar los tipos derivados como es el caso de la cadena o string. Recordando que el tipo de dato **char** me puede guardar solamente una letra, número o caracter especial, pero solamente uno, y en ocasiones es necesario trabajar con conjuntos de caracteres a la vez, por ejemplo, el nombre de una persona, la dirección de un negocio o el título de un profesionalista, entre otros. Ahora aparece el concepto de cadena que analizamos enseguida.

Una **cadena** o **string** es un conjunto o arreglo de caracteres, esto es, está conformado de la unión de varios elementos tipo **char**.

Debido a que el tema de arreglos es tratado en el capítulo 3, aquí solo se presentan ejemplos de distintas instrucciones que podemos utilizar para leer una cadena de caracteres,

```
/* Ejemplo 2.16: Lectura del nombre de una persona
usando usando cin */
#include<iostream>
using namespace std;
int main()
{
    char nombre[20]; //Declarando cadena de caracteres
    cout<<"\n\nLee cadena usando cin "<<endl;
    cout<<"Dame tu nombre: ";
    cin>>nombre;    // Lectura de cadena
    cout<<"\n \n Hola  "<<nombre<<endl;
    system("pause");
    return 0;
}
```

Al ejecutar el programa anterior e introducimos un nombre sin espacios en blanco no existe ningún problema, pero cuando se teclea el nombre y aparecen espacios se puede observar que la instrucción de lectura **cin** solo me guarda los caracteres hasta donde aparece el primer espacio. Para solucionar este tipo de problemas existen otras instrucciones de lectura que nos pueden ayudar, por ejemplo: **cin.getline**, **cin.get**, **gets** o **fgets**.

//Ejemplo 2.17: Se lee cadena con **cin.getline**

```
#include<iostream>
using namespace std;
int main()
{
    char nombre[20]; //Se declara cadena llamada nombre
    cout<<"\t\t!! Lee cadena usando cin.getline!!\n\n";
    cout<<"\t \t Dame tu nombre: ";
    cin.getline(nombre,20);
    cout<<endl<<"Hola " <<nombre<<endl;
    system ("pause");
    return 0;
}
```

//Ejemplo 2.18: Se lee cadena usando **cin.get**

```
#include<iostream>
using namespace std;
int main()
{
    char nombre[25];
    cout<<"\t \t !! Lee cadena usando cin.get !!\n\n";
    cout<<"\t \t Dame tu nombre: ";
```

```
cin.get(nombre,25);
cout<<endl<<"Hola " <<nombre<<endl<<endl;
system ("pause");
return 0;
}
```

```
//Ejemplo 2.19: Se lee nombre de persona con gets()
/* La funcion gets() no es aconsejada para algunos
compiladores. Por lo que solo se muestra como ejemplo
en Dev-C++ */
#include<iostream>
using namespace std;
int main()
{
    char nombre[20]; // Declarando cadena de caracteres
    cout<<"\n \nLee cadena usando gets "<<endl;
    cout<<"Dame tu nombre: ";
    gets(nombre); // Lectura de cadena
    cout<<"\n\n Hola " <<nombre<<endl;
    system ("pause");
    return 0;
}
```

La función **gets()** no se aconseja para algunos compiladores, por lo que solo se muestra como ejemplo en Dev-C++ *

El tipo **string** en C++ nos ayuda a trabajar con cadenas de texto, y dependiendo del IDE que se esté utilizando en ocasiones es necesario incluir en los programas ciertas librerías y así poder usar las instrucciones de manera correcta; una de estas librerías puede ser la llamada **string** o **cstring**, aunque algunos IDEs modernos ya incluyen a **<cstring>** dentro de en la librería **<iostream>**.

Para declarar una variable de tipo cadena podría ser la siguiente:

```
string nombre_variable(s);
```

Un ejemplo podría ser: `string nomalumno;`

Si quisiéramos almacenar información en la cadena al momento de definirla, podría ser:

```
string nomalumno="Carlos Jesus";
```

//Ejemplo 2.20: Programa usando tipo string y getline

```
#include<iostream>
using namespace std;
int main()
{
    string nombalum; //Se declara variable tipo string
    cout<<"\t \t !! Ejemplo de cadena !!\n\n";
    cout<<"\t \t Dame tu nombre: ";
    getline(cin,nombalum); //Se lee la cadena
    cout<<endl<<"Hola " <<nombalum<<endl<<endl;
    system ("pause");
    return 0;
}
```

// Ejemplo 2.21: Programa usando string y getline

```
#include<iostream>
using namespace std;
int main() {
    string nombre, direccion;
    int edad;
    float est;
    cout<<"\nDame tu Nombre: ";
```

```
getline(cin,nombre);
cout<<"\nDame tu Direccion: ";
getline(cin,direccion);
cout<<"\nDame tu Edad: ";
cin>>edad;
cout<<"\nDame tu Estatura: ";
cin>>est;
cout<<"\nTus características son:\n";
cout<<"\n\nNombre: "<<nombre;
cout<<"\n\nDireccion: "<<direccion;
cout<<"\n\nEstatura: "<<est;
cout<<"\n\nEdad: "<<edad<<endl<<endl;
system ("pause");
return 0;
}
```

Existen funciones que se pueden aplicar con cadenas de texto, por ejemplo **strlen()** que nos regresa el número de caracteres.

```
/* Ejemplo 2.22: Calcula la longitud de una cadena
(numero de caracteres)*/
#include<iostream>
#include<string.h>
using namespace std;
int main() {
    char texto[30];
    int longitud=0;
    cout<<"Dame tu texto: ";
    cin.getline(texto,30);
    longitud=strlen(texto);
    //strlen() regresa total de caracteres de texto
}
```



```
    cout<<"Numero de caracteres del texto tecleado  
    es: "<<longitud<<endl;  
    system ("pause");  
    return 0; }
```

Enseguida se muestra el ejemplo 2.23 donde se utiliza la funcion llamada **strcpy()** para copiar el contenido de una cadena en otra cadena.

/ Ejemplo 2.23: Usando funcion strcpy() para copiar una cadena de caracteres */*

```
#include<iostream>  
#include<string.h>  
  
using namespace std;  
int main()  
{  
    char palabra[]="PROGRAMANDO";  
    char palabra2[15];  
  
    //Usando funcion para copiar el contenido de cadena:  
    strcpy(palabra2,palabra);  
  
    cout<<"La palabra copiada es: "<<palabra2<< endl;  
    system ("pause");  
    return 0;  
}
```

La función **atoi()** convierte el texto de una cadena a un valor entero. La función **atof()** convierte el texto de una cadena a un valor flotante.

Observar el siguiente ejemplo:

```
//Ejemplo 2.24: Convertir cadena a entero y flotante
#include<iostream>
#include<stdlib.h>
using namespace std;
int main()
{
    char cad1[]="123"; //Se almacena una cadena en cad1
    int numentero;
    numentero=atoi(cad1); //Convierte cadena a entero
    char cad2[]="123.456"; //Almacena en cad2
    float numfloat, resultado;
    numfloat=atof(cad2); //Convierte a flotante
    //Ya convertidos a números se pueden sumar
    resultado=numentero+numfloat;
    cout<<"El resultado es: "<<resultado;
    system ("pause");
    return 0;
}
```

2.6. Complemento: Uso de system

Los sistemas operativos cuentan con una **línea de comandos**, también es llamada **consola de comandos**, la cual nos permite escribir y ejecutar instrucciones sin necesidad de usar el ratón. Esta interfaz de línea de Comandos de Windows es una implementación de la consola MS-DOS para la interfaz gráfica de

las diversas versiones de Microsoft **Windows**. Opciones para abrirla:

- Escribir en Ejecutar: **CMD** y presiona la tecla Enter.
- Abre el menú Win-X (teclas Windows + X) y escoge: "Símbolo del sistema".
- En el menú contextual del ratón: al dar clic derecho con la tecla Shift presionada, selecciona: "Iniciar Ventana de comandos aquí".

Con cualquier opción que se use aparecerá la **línea de comandos** como se muestra en la figura 2.8:

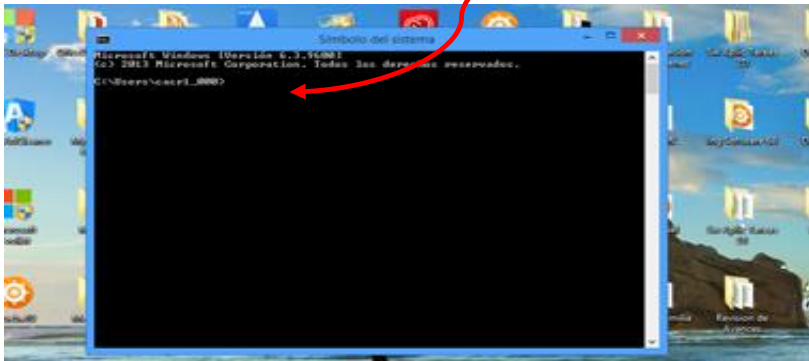


Figura 2.8. Línea de comandos o consola.

Por mencionar solo algunos ejemplos de comandos que se pueden usar aquí, revisa los siguientes:

CD <i>NombCarpeta</i>	Cambia a carpeta <i>NombCarpeta</i> indicada.
CD..	Regresa a carpeta anterior.
DIR	Para ver el contenido de una carpeta.
CLS	Para limpiar el contenido de pantalla.
PAUSE	Se detiene y aparece mensaje: "Presiona tecla para continuar..."

COLOR *Atrib*

Cambia color de letra y fondo. *Atrib* se forma por dos valores hexadecimales (0 a 9 y A a F), el primer valor indica el segundo plano y el segundo valor es el primer plano. Los valores que se pueden usar son:

0 = Negro	8 = Gris
1 = Azul	9 = Azul claro
2 = Verde	A = Verde claro
3 = Aguamarina	B = Aguamarina claro
4 = Rojo	C = Rojo claro
5 = Púrpura	D = Púrpura claro
6 = Amarillo	E = Amarillo claro
7 = Blanco	F = Blanco brillante

DATE *dd-mm-aa*

Para establecer la fecha del sistema.

TIME *hr : mi : sg*

Para establecer la hora del sistema.

EXIT

Para salir de la línea de comandos.

Prueba los comandos mencionados para que observes y entiendas cómo funcionan.

Varios de los comandos que se utilizan en el DOS, también se pueden utilizar desde un programa escrito en C++ mediante la función **system**. Probemos el siguiente programa:

//Ejemplo 2.25: Se prueba el uso system

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string nom1, nom2;
    cout << "Dame primer nombre: ";
    cin >> nom1;
```

```
    system("cls"); // Para limpiar la pantalla
    system("pause"); // Detiene ejecucion
    nom2 = "Otro nombre ";
    //Cambiar colores, verificar numeros y letras:
    system("COLOR 4f");
    cout << nom1;
    system("COLOR 3e"); // Usando comando COLOR
    cout << nom2;
    system("cls");    // Para limpiar la pantalla
    system("pause");
    return 0;
}
```

Esperamos que estas notas te puedan servir para ir dando una mejor vista a la ejecución de tus programas.

Actividad 2.1: Realizar los siguientes programas usando Dev-C++.

1. Mostrar el resultado de la fórmula: **$xyz=4-2+20*3$** (observe que no es necesario solicitar datos de entrada). Dato de salida: **xyz** .
2. Sumar cuatro números dados y al resultado restarle el valor de diez; y a este último resultado multiplicarlo por cinco. Mostrar el resultado final en pantalla.
3. Solicitar el peso de seis estudiantes, calcular el peso promedio y mostrar resultado.
4. Calcular el perímetro de un cuadrado, el cual se obtiene haciendo la suma de la longitud de sus lados iguales, que equivale a **$p=4*longitud$** . Dato de entrada: **$longitud$** . Dato de salida: **p** .
5. Calcular el área de un triángulo rectángulo. La fórmula a utilizar sería: **$\text{Área} = (b * h) / 2$** . Datos de entrada: **b** y **h** . Dato de salida: **Área** .
6. Convertir cantidad de grados ***Fahrenheit*** a grados ***Centígrados***. Solicitar los grados Fahrenheit, restarle 32 a los grados Fahrenheit y el resultado se multiplica por 5/9.
7. Convertir una cantidad dada en ***Euros*** a ***Pesos mexicanos***. Considerando que el Euro equivale a ***22.40 pesos***.
8. Solicitar una cantidad en ***Dólares US*** y convertirla tanto a ***Euros*** como a ***Pesos mexicanos***. Considerando que ***1 Dólar=0.86 euros*** y ***1 Dólar=18.50 pesos***. Los datos de salida son la cantidad de ***Euros*** y ***Pesos*** correspondientes.
9. Diseñe un algoritmo para calcular el ***área de un círculo*** cualquiera y la longitud de su circunferencia. **$\text{Area} = \pi * r^2$** y **$\text{longitud} = 2 * \pi * r$** .
10. Buscar cual es la fórmula para calcular el ***volumen de un cilindro***. Crear programa que solicite datos necesarios y mostrar el resultado obtenido (volumen).

11. Investigar fórmula para calcular el **volumen de una esfera**.
Crear programa que solicite datos necesarios y mostrar el resultado obtenido (volumen).
12. Escribir un programa que lea de la entrada estándar el precio de un producto y muestre en la salida estándar el precio del producto al aplicarle el IVA.
13. Calcular la calificación final de un estudiante de un curso, donde se calificaron tres exámenes equivalentes al 80% de calificación final; y dos prácticas de laboratorio equivalentes al 20%.
14. Realice un programa que lea los siguientes datos de una persona, para posteriormente mostrarlos en pantalla:

Edad	Dato de tipo entero
Sexo	Dato de tipo caracter
Altura en metros	Dato de tipo real.

Rúbrica de evaluación ejercicios del capítulo 2

	Excelente 10-8	Bueno 7-4	Deficiente 3-1	No realizado 0	Total
Estructura básica de un programa	Emplea de forma correcta la estructura de un programa en C++ -Variables y constantes globales -Función main -Variables locales -Instrucciones -Cierre de la función.	Emplea la estructura de un programa en C++ -Variables y constantes globales -Función main -Variables locales -Instrucciones -Cierre de la función. (omite al menos uno de los puntos anteriores)	Emplea parcialmente la estructura de un programa en C++ -Variables y constantes globales -Función main -Variables locales -Instrucciones -Cierre de la función. (omite al menos dos de los puntos anteriores)	No presenta un programa con estructura básica en C++.	
Elementos del lenguaje	Conoce y aplica con razonamiento e interpretación lógica: -Tipos de datos. -Literales. -Constantes. -Variables. - Identificadores. -Parámetros. -Operadores. -Salida de datos.	Conoce y aplica al menos seis de los siguientes elementos: -Tipos de datos. -Literales. -Constantes. -Variables. - Identificadores. -Parámetros. -Operadores. -Salida de datos.	Conoce y aplica al menos cuatro de los siguientes elementos: -Tipos de datos. -Literales. -Constantes. -Variables. - Identificadores. -Parámetros. -Operadores. -Salida de datos.	No aplica los elementos del lenguaje de programación C++	

Capítulo 2. Introducción a la programación.

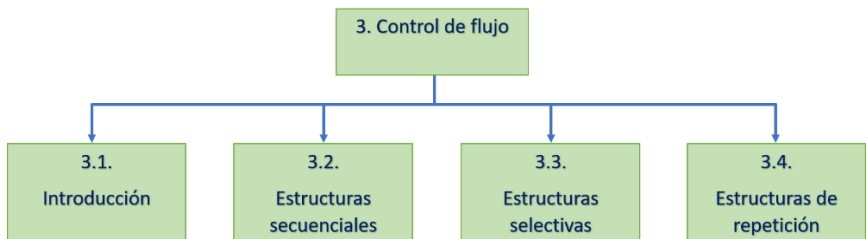
Traducción de un programa	Distingue y aplica los siguientes procesos: -Compilación. -Enlace. -Ejecución. -Errores.	Aplica de manera directa la ejecución de un programa sin conocer el proceso de compilación y enlace. Sin embargo, conoce como identificar errores.	Aplica de manera directa la ejecución de un programa sin conocer el proceso de compilación y enlace. Desconoce el proceso de identificar errores.	Desconoce como traducir un programa.	
Comentarios					

Capítulo 3: Control de flujo

Objetivos de capítulo 3:

- Presentar una introducción al concepto de estructuras de control de flujo, así como sus diferentes tipos y objetivos.
- Conocer las diferentes estructuras de control de flujo de decisión, en dónde las podemos utilizar y como las podemos incluir en nuestros programas.
- Conocer las diferentes estructuras de control de flujo de repetición, en dónde las podemos utilizar y como las podemos incluir en nuestros programas.
- Realizar prácticas (programas) planteadas por el profesor para poder reafirmar cada conocimiento aprendido.

Subtemas de capítulo 3:



3.1 Introducción

Hasta ahora hemos visto solo instrucciones de entrada (cin), salida (cout) y algunas expresiones básicas. Aquí trabajaremos con otro tipo de instrucciones llamadas estructuras de control y sus combinaciones que pueden servir para resolver diferentes tipos de problemas. Las estructuras básicas de control pueden ser:

- **Secuenciales:** Cuando una instrucción del programa sigue a otra instrucción.
- **Selección o decisión:** Utiliza una o varias condiciones para determinar que instrucciones se ejecutaran.
- **Repetición:** Es cuando un conjunto de instrucciones se ejecuta una o varias veces, mientras se cumpla la condición especificada.

3.2 Estructuras secuenciales

En programación, una estructura secuencial es aquella en la que una instrucción (acción) sigue a otra secuencia de instrucciones. Las tareas se realizan de tal modo que la salida de una, puede ser la entrada de la siguiente y así sucesivamente hasta el fin del proceso.

Significa que la ejecución de cada instrucción se lleva a cabo una después de otra, según el orden en que aparecen en el programa. Enseguida se muestra un ejemplo del uso de la estructura **secuencial**.

Ejemplo 3.1: Solicitar cuatro 4 tipo float para resolver la expresión:
 $xyz = b * (x * x) + 4 - a + 20 * 3 - xy$.

*/*Ejemplo 3.1: Se solicitan 4 valores, se realiza operación y muestra resultado */*

```
#include<iostream>
using namespace std;
int main() {
    float xyz, b, x, a, xy;
    cout<<" \n Dame el valor de b: ";
    cin>>b;
    cout<<" \n Dame el valor de x: ";
    cin>>x;
    cout<<" \n Dame el valor de a: ";
    cin>>a;
    cout<<" \n Dame el valor de xy: ";
    cin>>xy;
    xyz=b*(x*x)+4-a+20*3-xy;
    cout<<"\nEl resultado de la operacion es: "
    <<xyz<<endl;
    system("pause");
    return 0;
}
```

Observemos el orden y la secuencia que se sigue para ejecutar una instrucción después de otra, primero debemos capturar los cuatro valores necesarios para después poder realizar la expresión aritmética que se presenta, y posteriormente poder mostrar el resultado. No puedo realizar la operación si no conozco los valores y no puedo mostrar el resultado si no he hecho la operación.

Ejemplo 3.2: Programa que permita conocer el área de un triángulo a partir de la base y la altura.

```
// Ejemplo 3.2: Se obtiene el área de un triángulo
#include<iostream>
using namespace std;
int main()
{
    float base, altura, area;
    cout<<" \n Dame valor de la base: ";
    cin>>base;
    cout<<" \n Dame valor de la altura: ";
    cin>>altura;
    area=(base*altura)/2;
    cout<<"\nEl Area del triángulo es:"<<area<<endl;
    system("pause");
    return 0;
}
```

En el ejemplo 3.2 podemos observar que primero se requiere de los valores de base y altura para poder obtener el área del triángulo y posteriormente mostrar el resultado obtenido.

3.3 Estructuras selectivas

El concepto de estructuras selectivas o condicionales es el mismo que se trabajó en la primera unidad con PSeInt, pero ahora lo enfocaremos a un lenguaje de programación específico. Las instrucciones selectivas o condicionales nos ayudan a determinar si se cumple o no alguna condición. Si la condición se cumple se realizará un grupo de instrucciones, y si no se cumple se realizará otro grupo de instrucciones indicado. Por ejemplo, puede ser útil cuando se le pide al usuario una contraseña para ingresar al sistema de contabilidad. Con la condición se prueba que los datos

ingresados por el usuario son correctos, si lo son permitir ingresar, y si no solo muestra un mensaje de error y sale.

En **C++** se utiliza la instrucción **if**, que es una estructura que permite tomar decisiones dentro de un programa, es decir, le especificamos que acciones se van a realizar si se cumple o no una condición. Enseguida se muestra el formato para poder hacer uso de la instrucción **if**:

```
if (condición a evaluar)
{
    // Instrucciones si SI se cumple la condición
}
else {
    // Instrucciones si NO se cumple la condición
}
```

La parte correspondiente al **else** no es obligatoria, esto depende del problema a resolver o de la forma a usar por el programador.

Ejemplo 3.3: Programa que solicite un número y nos diga si es menor que cero.

```
//Ejemplo 3.3: Dice si un número es menor que cero
#include<iostream>
using namespace std;
int main() {
    int num;
    cout<<"Dame un valor entero: ";
    cin>>num;
    if(num<0){           // Uso de instrucción if()
        cout<<"\n Me diste el numero "<<num;
```

```
        cout<<"\n Y si es menor que 0\n";
    }    //Fin de if
    system ("pause");
    return 0;
}
```

En el ejemplo anterior aparece la instrucción **if(num<0)** que evalúa si el valor de **num** es menor que cero, si es menor muestra dos mensajes y si no lo es no realiza nada y se termina el programa.

Ejemplo 3.4: Programa que solicite 2 números y nos diga si la suma de estos es mayor a 50.

```
//Ejemplo 3.4: Dice si la suma es mayor a 50
#include<iostream>
using namespace std;
int main() {
    float num1, num2, suma=0;
    cout<<"Dame numero 1: ";
    cin>>num1;
    cout<<"Dame numero 2: ";
    cin>>num2;
    suma = num1 + num2;
    if(suma>50)        // Uso de instrucción if()
        cout<<"\n La suma obtenida "<<suma<<" si es
            mayor a 50"<<endl;
    system ("pause");
    return 0;
}
```

En el ejemplo anterior la condición es `if(suma>50)` evalúa si el valor de `suma` es mayor a cincuenta, si es mayor muestra un mensaje, y si no lo es no realiza nada y se termina el programa. También observamos que cuando la condición es verdadera aparece sólo una instrucción, por lo que no es necesario el uso de los caracteres { y }. Aunque si se agregan funcionaría de la misma manera.

Ejemplo 3.5: Programa que nos dice si eres mayor de edad.

// Ejemplo 3.5: Nos dice si eres mayor de edad

```
#include<iostream>
using namespace std;

int main()
{
    int edad;
    cout<<"Dame tu edad: ";
    cin>>edad;
    if(edad>=18)
    {
        cout<<"\n Ya eres mayor de edad ";
    }
    cout<<endl;
    system ("pause");
    return 0;
}
```


Ejemplo 3.6: Programa que nos dice si aprobaste la materia según el promedio obtenido.

```
//Ejemplo 3.6: Nos dice si aprobaste según promedio
#include<iostream>
using namespace std;
int main()
{
    float califexam, califtar, final;
    cout<<"Dame calificacion de examen: ";
    cin>>califexam;
    cout<<"Dame tu calificacion de tarea: ";
    cin>>califtar;
    final=(califexam+califtar)/2;
    if(final>=70)
    {
        cout<<"\n Felicidades si aprobaste con: "<<final;
    }
    cout<<endl;
    system ("pause");
    return 0;
}
```

Como mencionamos, dentro de una instrucción **if**, puede existir la parte **else** (si no) que es donde aparecen las instrucciones que se van a ejecutar si no se cumple la condición. En ejemplo 3.7 se captura el valor de un número entero que es utilizado en la condición, si se cumple la condición nos muestra un mensaje, si no se cumple (**else**) nos mostrará otro mensaje diferente.

/ Ejemplo 3.7: Dice si número es mayor o igual a
cero, o si es menor que cero */*

```
#include<iostream>
using namespace std;

int main()
{
    int valor;
    cout<<"Dame un valor entero: ";
    cin>>valor;
    if(valor>=0)
    {
        cout<<"\nValor dado si es mayor o igual 0\n";
    }
    else
    {
        cout<<"Valor dado es menor que cero \n";
    }
    system ("pause");
    return 0;
}
```

Ejemplo 3.8: Se quiere aumentar el 10% a los trabajadores que ganan entre 5000 y 8000. Y a los que ganan menos de 5000 se les aumenta un 15% su sueldo.

La solución a este problema se presenta a continuación:

*/*Ejemplo 3.8: Se aumenta porcentaje de sueldo según el sueldo actual */*

```
#include<iostream>
using namespace std;
int main() {
    float sueldo;
    cout<<"Dame tu sueldo: ";
    cin>>sueldo;
    if(sueldo<5000) {
        sueldo=sueldo+(sueldo*0.15);
        cout<<"\n Su nuevo sueldo será: "<<sueldo;
    }
    else
    {
        if (sueldo>=5000 && sueldo<=8000)
        {
            sueldo=sueldo+(sueldo*0.10);
            cout<<"\n Su nuevo sueldo será: "<<sueldo;
        }
    }
    cout<<endl;
    system ("pause");
    return 0;
}
```

Los operadores lógicos AND, OR y NOT nos permiten combinar varias expresiones condicionales. En C++ estos operadores son representados como && (AND lógico), || (OR lógico) y ! (Negación).

En el ejemplo 3.8 se hace uso del operador AND lógico dentro de la condición, la cual evalúa si el sueldo es mayor o igual a 5000 y además si el sueldo es menor o igual a 8000, observe la instrucción `if(sueldo>=5000 && sueldo<=8000)`. Si se cumplen

las dos condiciones de este **if** entonces nos arroja un verdadero, por lo tanto, se ejecutan las instrucciones especificadas.

Observar que al usar AND lógico tienen que cumplirse ambas condiciones. Y cuando se utiliza el **OR lógico** representado por **||**, si se cumple alguna de las condiciones nos devolverá verdadero.

Ejemplo 3.9: Se quiere realizar un aumento de salario según el valor determinado de este. Si su salario es 250 o 400 se aumenta el 12%, si su salario es diferente se aumenta el 8%.

```
/* Ejemplo 3.9: Si su salario es 250 o 400 se aumenta
el 12%, si es diferente se aumenta el 8% */
#include<iostream>
using namespace std;
int main(){
    float sueldo;
    cout<<"Dame tu sueldo: ";
    cin>>sueldo;
    if(sueldo==250 || sueldo==400) {
        sueldo=sueldo+(sueldo*0.12);
        cout<<"\n Su nuevo sueldo será: "<<sueldo;
    }
    else {
        sueldo=sueldo+(sueldo*0.08);
        cout<<"\n Su nuevo sueldo será: "<<sueldo;
    }
    cout<<endl;
    system ("pause");
    return 0;
}
```

Ejercicio 3.1: Vamos a resolver en clase ejercicios que requieren de instrucciones de decisión.

1. Programa que solicite dos números enteros y nos diga cuál es el mayor de estos números.
2. Programa que solicite un número entero y nos diga si este es par o impar.
3. Pedir altura de dos estudiantes y nos diga si tienen la misma altura, sino que nos diga quién es más alto.
4. Capturar tres valores numéricos de tipo real, y nos diga cuál es mayor. Además, mostrar la suma de los dos menores.
5. Solicitar tres números y nos diga si el tercer número capturado es igual a la suma del primer y segundo números.
6. Programa que solicite un número entero y si este es mayor o igual a 10 devuelva el triple de este, de lo contrario la mitad del mismo.
7. Solicitar que se capture una letra y nos diga si esta es vocal o no.
8. Solicitar una clave, **si la clave es igual a “casa”**: imprimir “Tienes Acceso” y posteriormente preguntar la edad y si es mayor o igual a 30 sumarle a la edad 5 e imprimirla y si la edad es menor a 30 sumarle 2 e imprimirla. En el caso de que la clave no sea válida imprimir “Nooo se puede”.

Instrucción switch

En ocasiones el uso de varias instrucciones de decisión **if** anidadas (if dentro de otro if) puede volverse un poco tedioso para el programador. La existencia de instrucciones de decisión múltiple nos puede ayudar a mejorar los programas que usan condicionales. En **C++** la instrucción se llama **switch** y se utiliza cuando existen varias opciones a ser seleccionadas por una variable, a cada opción se le llama **case** y cuando una de estas coincide con el valor de la variable especificada se realizan las instrucciones correspondientes. El formato para escribir la instrucción switch es:

```
switch (variable) //variable se compara con los case
{
    case valor1:
        instrucciones; //instrucciones de valor1
        break;
    case valor2:
        instrucciones; //instrucciones de valor2
        break;
        ...
        break;
    default:
        instrucciones;
}
```

La opción **default** se aplica cuando el valor de la variable no corresponde a ninguno de los casos, significa que es posible especificar las instrucciones a realizar si no se coincide con ninguna de las opciones. También, al final de las instrucciones de cada uno de los casos se aparece **break**, esta instrucción nos dice que se salga de la estructura switch y continúe con la instrucción que continúa después de ésta.

Ejemplo 3.10: Se quiere presentar un menú formado por tres opciones (1, 2, 3), el usuario seleccionara cualquier opción y el programa nos mostrara un texto que nos diga cuál fue la opción escogida. En caso de que no corresponda a ninguno del menú escribir en pantalla “Seleccionaste un número no válido”.

```
/* Ejemplo 3.10: Se muestra un menú de 3 opciones y  
nos dice cual fue seleccionada */  
#include<iostream>
```

```
using namespace std;
int main()
{
    int num;
    cout<<"1. Es el menor \n ";
    cout<<"2. Es el intermedio \n ";
    cout<<"Es el mayor \n";
    cout<<"selecciona una opcion: ";
    cin>>num;
    switch(num)
    {
        case 1:
            cout<<"Seleccionaste numero uno: menor";
            break;
        case 2:
            cout<<"Seleccionaste numero dos: intermedio";
            break;
        case 3:
            cout<<"Seleccionaste numero tres: mayor";
            break;
        default:
            cout<<"Seleccionaste numero no valido";
            } //Fin de switch
        cout<<endl;
        system ("pause");
        return 0;
    }
```

Ejemplo 3.11: Se pide número entre 1 y 7 correspondientes al día de la semana (Lunes, Martes, ..., Domingo) y muestra comentario según el día. De lo contrario muestra: "Día no existente".

```
/* Ejemplo 3.11: Nos muestra comentario segun el dia
seleccionado */
#include<iostream>
using namespace std;
int main() {
int num;
    cout<<"Dame un numero del 1 al 7: ";
    cin>>num;
    switch(num) {
        case 1: cout<<"Es lunes, iniciando semana";
            break;
        case 2: cout<<"Es martes, mucho trabajo";
            break;
        case 3: cout<<"Es miercoles y con mas trabajo";
            break;
        case 4: cout<<"Ya es jueves, mitad de semana";
            break;
        case 5: cout<<"Que rapido ya estamos viernes";
            break;
        case 6: cout<<"Hoy sabado se necesita descanso";
            break;
        case 7: cout<<"Por fin ya es domingo";
            break;
        default: cout<<"Dia inexistente";
    }
    cout<<endl;
    return 0;
}
```

Ejemplo 3.12: Mostrar 3 opciones del menú de un restaurant y en cada opción presentar: número, comida y costo. Al seleccionar alguna opción, el programa me mostrara el total a pagar agregando el impuesto correspondiente (13% en opción uno, 18% en la dos y 17% en la tres).

/ Ejemplo 3.12: Se muestra un menu para seleccionar comida a consumir */*

```
#include<iostream>
using namespace std;
int main() // Inicio del programa
{
    int opc;
    float total1=75.00, total2=82.50, total3=80.00,
    total=0;
    cout<<"1. Caldo de pollo (75.00) "<<endl;
    cout<<"2. Bistec (82.50) "<<endl;
    cout<<"3. Ensalada de camarón (80.00) "<<endl;
    cout<<"Que deseas comer: ";
    cin>>opc;

    switch(opc) // Inicio del switch
    {
        case 1:
            total = total1+(75.00*0.13);
            cout<<"El total a pagar es: "<<total;
            break;
        case 2:
            total = total2+(82.50*0.18);
            cout<<"El total a pagar es: "<<total;
            break;
        case 3:
            total = total3+(80.00*0.17);
            cout<<"El total a pagar es: "<<total;
            break;
        default: cout<<"Comida inexistente gracias";
    }
}
```

```
    }           // Fin del switch
    cout<<endl;
    system ("pause");
    return 0;
}           // Fin del programa
```

Ejemplo 3.13: Programa que nos pide una letra y nos dice si es una vocal minúscula o no.

/ Ejemplo 3.13: Se pide una letra y nos dice si esta es una vocal minúscula */*

```
#include<iostream>
using namespace std;
int main()
{
    char caract;
    cout<<"Dame una letra: ";
    cin>>caract;
    switch(caract)
    {           // Inicio del switch
        case 'a':
            cout<<"Es una vocal, tecleaste "<<caract;
            break;
        case 'e':
            cout<<"Es una vocal, tecleaste "<<caract;
            break;
        case 'i':
            cout<<"Es una vocal, tecleaste "<<caract;
            break;
        case 'o':
            cout<<"Es una vocal, tecleaste "<<caract;
```

```
        break;
    case 'u':
        cout<<"Es una vocal, tecleaste "<<caract;
        break;
    default:
        cout<<"Lo ingresado no es vocal minuscula";
    }           // Fin del switch
    cout<<endl;
    system ("pause");
    return 0;
}           // Fin del programa
```

La instrucción **switch** tiene ciertas ventajas sobre la instrucción **if**, y es muy utilizada cuando se requiere seleccionar una de varias opciones disponibles, por ejemplo, cuando se quiere mostrar al usuario un menú de actividades para él pueda seleccionar alguna de ellas.

Actividad 3.1: Crear los siguientes programas usando Dev C++:

1. Programa que solicite dos números y nos diga cuál es el mayor de estos números, o bien, que nos diga si son iguales.
2. Pedir altura de dos estudiantes y nos diga si tienen la misma altura, sino que nos diga quién es más alto.

3. Capturar tres valores y nos diga cuál es mayor. Además, mostrar la suma de los otros dos menores.
4. Solicitar tres números y nos diga si el tercer número capturado es igual a la suma del primer y segundo números.
5. Solicitar tres números y los muestre en pantalla ordenados de menor a mayor.
6. Leer un número entero y determinar si tiene tres cifras. Esto es: $n \geq 100$ y $n < 1000$.
7. Leer un número entero menor a 1000 y determinar cuántos dígitos tiene. Ejemplo: Si $n < 10$ y $n > 0$ entonces imprimir el mensaje "Tiene un dígito".
8. Pedir la calificación de examen de un estudiante, así como su calificación de asistencia, y mostrar en pantalla si es aprobado o no. Aprobará siempre y cuando tenga mínimo el 80% de Asistencia y la calificación de Examen sea mayor o igual al 80%.
9. Una empresa aumentará el salario a empleados de acuerdo con la escala salarial en que se encuentren: Aumentará 5% a los que ganan entre 5000 y 7000, 10% a los que ganan más de 3000 y menos que 5000, y 15% a los que ganan menos de 3000. Diseña programa para calcular el nuevo salario del empleado según su escala.
10. Desarrolle programa que solicite Nombre, Edad, Sexo y Estado civil de una persona y muestre el nombre de la persona si corresponde a un hombre casado mayor de 40 o a una mujer soltera menor de 50 años. Datos de entrada: nombre, edad, sexo y estado civil. Sugerencia: 1 significa casado y 0 soltero; 'm' significa mujer y 'h' hombre. Dato de salida es: Nombre.
11. Que se solicite un número del uno al doce y nos diga el nombre del mes correspondiente (Enero es 1, Febrero es 2, ..., Diciembre es 12).
12. Solicitar dos valores y el operador que se quiere aplicar: + representa la suma, - es resta, / es división y * la multiplicación. Realizar el cálculo usando los valores y el operador seleccionado para mostrar el resultado.

13. Programa que presente un menú de opciones y que pregunte cual se desea: 1. Área de un Cuadrado, 2. Área de Triángulo y 3. Área de Círculo. Solicitar los valores necesarios según la opción y mostrar en pantalla el resultado.
14. Programa que presente 3 opciones y que pregunte cual se desea (1. Número, 2. Departamento y 3. Nombre), una vez seleccionada la opción pedir el dato correspondiente y mostrar un mensaje que nos diga: “ACEPTADO” siempre y cuando los números sean 2, 4 o 6, en el caso de Departamento solo “Gerente” o “Jefe”, y en el caso de nombre solo: “Carlos” o “Diana”.

3.4 Estructuras de repetición

Una repetición significa que queremos ejecutar dos o más veces a un conjunto de instrucciones; y la actividad que nos permite realizarlo se le conoce como ciclo, bucle o repetición. En programación existen diferentes tipos de instrucciones cíclicas o repetitivas, por ejemplo, en C++ contamos con: `for`, `while` y `do-while` en donde cada una de estas tiene su utilidad para algunos casos específicos.

3.4.1 Instrucción `for`

El **`for`** nos permite ejecutar de manera repetitiva un conjunto de instrucciones, y se usa cuando se conoce el número de veces que queremos repetir al grupo de instrucciones, en otras palabras, es necesario conocer el valor de inicio, el valor de término y el valor del incremento. Por ejemplo, si se quiere mostrar los números pares entre el 2 y 40, que nos llevaría a obtener: 2, 4, 6, 8, ... hasta el 40. Aquí ya contamos con el valor inicial y valor final (2 y 40), además el valor de incremento es 2.

El formato general para la instrucción **for** en lenguaje de programación C++ es el siguiente:

```
for (int Ident1=valor_inicial; Ident1 <= valor_final;  
      Ident1 = Ident1+ paso)  
{  
  
    instrucciones;  
  
}
```

Aparece la variable de control **Ident1** de tipo entero (int) a la cual se le asigna un **valor inicial**. Donde aparece **valor_final** se nos permite evaluar la condición, la cual nos ayuda a definir el número de veces que se realizará. Y en **paso** se especifica como se irá incrementando. Esto se va a realizar hasta que deje de cumplirse la **condición**.

Notas para el uso de for () en C++:

- Los componentes del **for ()** están separados por punto y coma (;).
- La **variable de control** va a estar cambiando de acuerdo al **paso** especificado.
- Al **paso** se le asigna un valor específico, que puede ir incrementando o disminuyendo.
- Las llaves { y } encierran al bloque de instrucciones a realizarse **n** número de veces.

- Si la instrucción a realizar dentro del ciclo es solamente una, no son necesario las llaves.

Ejemplo 3.14: A continuación, se presenta un programa que nos ayuda a mostrar ocho veces nuestro nombre en la pantalla de salida.

// Ejemplo 3.14: Muestra ocho veces mi nombre

```
#include<iostream>
using namespace std;
int main()
{
    int x;
    for (x=1; x<=8; x=x+1)
    {
        cout<<endl<<"Carlos Alonso Camacho";
        cout<<endl;
    }
    system ("pause");
    return 0;
}
```

En ejemplo 3.14 la variable es **x** se inicializa en 1 y va a ejecutar las instrucciones hasta que la variable tome el valor de 8 (condición). Además, los incrementos son de uno en uno (**x=x+1**) que también podría escribirse como **x++**.

Ejemplo 3.15: Programa que pida 15 valores enteros y nos muestre en pantalla cuántos de ellos son negativos y cuantos positivos.

A continuación, se muestra una solución al problema usando la instrucción for:

/ Ejemplo 3.15: Solicita 15 valores y muestra el total de negativos y positivos, sin contar el cero */*

```
#include<iostream>
using namespace std;

int main()
{
    int num, i, ctaneg=0, ctaposit=0;
    for(i=1; i<=15; i++)
    {
        cout<<"Dame un numero entero: ";
        cin>>num;
        if(num<0)
            ctaneg=ctaneg+1;
        if(num>0)
            ctaposit=ctaposit+1;
    }          // Termina el ciclo for
    cout<< "\nPositivos: " << ctaposit <<
    " Negativos:"<< ctaneg;
    system ("pause");
    return 0;
}
```

Ejemplo 3.16: Se quiere mostrar en pantalla los números pares entre el 2 y 40.

La solución a este problema se muestra a continuación:

/ Ejemplo 3.16: Muestra los números pares entre el 2 y el 40 */*

```
#include<iostream>
using namespace std;
int main()
{
    int i;
    for(i=2; i<=40; i=i+2)
    {
        cout<<endl<<"El numero es: "<<i;
    }    //Fin del ciclo for
    cout<<endl<<"Ya terminaste"<<endl;
    system ("pause");
    return 0;
}
```

3.4.2 Instrucción while

Al igual que la instrucción **for**, la estructura cíclica **while** o **bucle mientras** es un ciclo repetitivo basado en los resultados de una expresión lógica (condición). En pocas palabras, el objetivo de esta estructura es repetir un conjunto de instrucciones mientras una condición sea verdadera. El formato de la instrucción **while** en lenguaje de programación C++ es el siguiente:

```
while (condición)
{
    instrucciones;
}
```

En la parte de **condición** se especifica una comparación que nos devuelva un valor lógico, esto es, que nos regrese un resultado **verdadero** (cuando se cumple la condición) o uno **falso** (cuando no se cumple la condición). Además, en la condición también puede aparecer una variable donde el resultado depende de su contenido (0 equivale a falso y diferente de 0 es verdadero).

Ya sabemos que los operadores del lenguaje C++ que pueden aparecer en una condición pueden ser: < (menor), > (mayor), == (igual), != (diferente), <= (menor o igual) o >= (mayor o igual).

Ejemplo 3.17: Se pedirán valores numéricos hasta que se capture un valor mayor a 100.

```
/* Ejemplo 3.17: Se lee valor hasta que uno de ellos
sea mayor a 100 */
#include<iostream>
using namespace std;
int main()
{
    int num=0;        // Damos valor inicial a num
    while(num<=100) // Usamos num en el condicional
    {
        cout<<"Dame un valor: ";
        cin>>num;    //La continuacion depende de num
        cout<<endl;
    }
    system ("pause");
    return 0;
}
```

Ejemplo 3.18: Escribir programa que esté solicitando un valor numérico entero hasta que se capture un valor negativo y que nos diga cuantos valores positivos fueron.

```
/*Ejemplo 3.18: Pide valor hasta que sea negativo y
muestra total de positivos */
#include<iostream>
using namespace std;

int main()
{
    int valor=0, contad=0; //Inicializa variable
    while(valor>=0)
    {
        cout<<"\n Dame un numero: ";
        cin>>valor; //Pide valor que ayuda a condición
        contad++;    //Variable para contar positivos
    }
    contad--;
    cout<<"Los números positivos capturados fueron: "
    <<contad<<endl;
    system ("pause");
    return 0;
}
```

Ejemplo 3.19: Escribir programa que muestre en pantalla 8 veces mi nombre usando while. Observe que estamos utilizando la variable de control i, la cual se inicializa antes de while y se va incrementando dentro de las instrucciones de este.

```
/* Ejemplo 3.19: Muestra 8 veces su nombre usando  
while */
```

```
#include<iostream>  
using namespace std;  
int main()  
{  
    int i=1;    // Damos valor inicial a i  
    while(i<=8)  
    {  
        cout<<"\n Carlos Alonso ";  
        i++;  
    }  
    cout<<endl;  
    system ("pause");  
    return 0;  
}
```

3.4.3 Instrucción do-while

Esta estructura cíclica nos permite ejecutar un bloque de instrucciones de forma repetitiva sin necesidad de conocer un valor inicial e incluso sin saber valor final. La diferencia con las otras es que **do-while** nos permite ejecutar primero las instrucciones antes de evaluar la condición, lo que significa que el conjunto de instrucciones se ejecuta como mínimo una vez y después se evalúa la condición. El formato de la instrucción **do-while** es el siguiente:

```
do
{
    instrucciones;
}
while (condición);
```

Ejemplo 3.20: Hacer programa que pida números enteros hasta que se capture valor mayor a 100.

```
/* Ejemplo 3.20: Usando do-while pide valores hasta
que sea mayor que 100 */
#include<iostream>
using namespace std;

int i;
int main()
{
    do
    {
        cout<<"\n Dame un valor: ";
        cin>>i;
    }
    while(i<=100); //La condicion se escribe al final
    system ("pause");
    return 0;
}
```

Ejemplo 3.21: Escribir programa que este solicitando un valor numérico entero hasta que se capture un valor negativo y que nos diga cuantos valores positivos fueron.

*/*Ejemplo 3.21: Pide valor hasta que sea negativo, y muestre total de positivos */*

```
#include<iostream>
using namespace std;
/*Se usa num para pedir valor y contad para contar los
numeros positivos */
int main()
{
    int num, contad=0;
    do                // Hacer
    {
        cout<<"\n Dame un valor: ";
        cin>>num;
        contad++;
    }
    while(num>=0); //Mientras sea mayor o igual a 0
    contad--;
    cout<<endl<<"La cantidad de numeros positivos
fueron: "<<contad<<endl;
    system ("pause");
    return 0;
}
```

Ejemplo 3.22: Escribir programa que muestre un menú conformado por 4 opciones y que se esté ejecutando hasta que se seleccione la opción Salir. El menú es: 1. Suma, 2. Multiplica, 3. Resta y 4. Salir. En cada opción se debe mostrar un mensaje relacionado con la opción seleccionada.

*/*Ejemplo 3.22: Se muestran opciones y solicita cual quiere realizar. Se va a ejecutar hasta que se seleccione opcion 4 que es Salir */*

```
#include<iostream>
using namespace std;
int main()
{
    int opc;
    do // Hacer lo siguiente
    {
        cout<<"\n 1. Suma ";
        cout<<"\n 2. Multiplica ";
        cout<<"\n 3. Resta ";
        cout<<"\n 4. Salir ";
        cout<<"\n Opcion deseada: ";
        cin>>opc; // Se captura el valor de opc
        switch(opc)
        {
            case 1:
                cout<<"\n Seleccionaste opcion 1 de Suma \n";
                break;
            case 2:
                cout<<"\n Seleccionaste opción 2 Multiplica \n";
                break;
```

```
case 3:
    cout<<"\nSeleccionaste opcion 3 Resta\n";
    break;
case 4:
    break;    //Permite salirnos del switch()
default:
    cout<<"\n Opcion no valida \n";
}            // Fin del switch
}            // Fin de while
while(opc!=4); // Mientras opc sea distinto de 4
system ("pause");
return 0;
}
```

3.5. Complemento: Instrucciones de salto en estructuras de control

Anteriormente se mencionó que la manera en que se ejecutan las instrucciones en C++ es secuencial, es decir, en el mismo orden en que están escritas en el programa. Pero existen instrucciones que permiten realizar saltos de líneas de código. Por ejemplo, las instrucciones **break**, **continue** y **goto**.

break

Recordemos el uso de la instrucción **break** dentro del **switch**, donde se escribe un break al final de cada opción para poder salir del bloque y no ejecutar las siguientes opciones. Pero también algunas instrucciones de este tipo pueden ser aplicadas en las estructuras de control que hemos visto hasta ahora, como son las condicionales y los ciclos, por lo tanto, nos pueden ayudar a alterar

el flujo normal de un programa. Por ejemplo, se puede utilizar para salir de un ciclo (for, while y do-while) antes de que este llegue a su finalización, terminando la ejecución del ciclo más interno en que se encuentra ubicada. Para poder entender, analicemos el siguiente ejemplo:

En **ejemplo 3.23**, se leerán números enteros hasta que se introduzca un número par. Observe que se utiliza un ciclo infinito **for**, donde las instrucciones se estarán ejecutando hasta que se encuentre una instrucción **break** para salir del ciclo. Al momento de introducir un número par se muestra mensaje y enseguida se sale del ciclo. Si el número es impar, se mostrará mensaje y el valor del número incrementado en 1. Al salir del ciclo sólo mostrará mensaje indicando que ya terminó.

/ Ejemplo 3.23: Solicita número hasta que sea par y sale del ciclo usando break */*

```
#include<iostream>
using namespace std;
int main() {
    int num;
    for(;;)    //Ciclo infinito
    {
        cout << "\nDame un entero: ";
        cin >> num;
        if(num %2 == 0 )    //Si el numero es par o cero:
        {
            cout<<"\n El numero es par o cero, adios \n";
            break; // Se sale del ciclo
        }
        else                //Si el numero es impar:
        {
```

```
        cout<< "\nEl numero es impar ..." << endl;
        cout << "... y al sumarle 1 quedaria: "
        << num+1 << endl;
    }
}
cout<<"\n.....Programa terminado.....\n";
system ("pause");
return 0;
}
```

En los **ejemplos 3.24 y 3.25** que se presentan a continuación se solicita el valor de variable de **a**, y el ciclo se ejecutara mientras el valor no sea mayor que 100. Sin embargo, se saldrá del bucle si se captura un valor negativo.

```
/* Ejemplo 3.24: Ejemplo que usa break para salir del
ciclo for */
#include<iostream>
using namespace std;
int main()
{
    float a, total=0;
    int c; //Nos ayuda como variable de control en for
    for(c=1; a<=100; c++)
    {
        cout<<"Dame valor de a: ";
        cin>>a;
        if (a<0) {
            //Si se cumple muestra mensaje y sale con break
            cout<<("\nUltimo valor capturado es negativo");
            break;
        }
    }
}
```

```
    }
    total = total+a;
}
cout<<"\n\nTotal es: " << total;
return 0;
}    // Fin de programa
```

//Ejemplo 3.25: Se usa break para salir del ciclo while

```
#include<iostream>
using namespace std;
int main(){
    float a=1, total=0;
    int c;    // c es variable de control en while
    while (a<=100)
    {
        cout<<"Dame valor de a: ";
        cin>>a;
        if (a<0)
        {    //Si es negativo, muestra mensaje y sale
            cout<<("\nUltimo valor capturado negativo");
            break;
        }
        total = total+a;
    }    //Fin de while
    cout<<"\n\nTotal es: " << total;
    system ("pause");
    return 0;
}
```

continue

La instrucción **continue** tiene un uso menos frecuente que el **break** y debe escribirse dentro del cuerpo de un ciclo, sin embargo, aquí no se sale del bucle, sino que se saltan todas las instrucciones del bucle desde que la encontramos hasta la siguiente repetición.

El efecto de la instrucción **continue** en un ciclo **while** o **do while** es, que se dejan de ejecutar el resto de las instrucciones del cuerpo del ciclo y se pasa a evaluar la condición lógica que controla el ciclo. El efecto en una sentencia **for** es que se dejan de ejecutar el resto de las instrucciones del cuerpo del ciclo y se pasa a evaluar la expresión de incremento del ciclo y enseguida evaluar la condición del ciclo.

Esta instrucción también puede ser utilizada dentro de un condicional para que afecte sólo a algunas instrucciones del bucle. Por ejemplo, en **ejemplo 3.26** si queremos mostrar en pantalla sólo los números pares contenidos entre el 1 y el 40, utilizando la instrucción **continue** dentro de un condicional. Una posible solución a este problema se mostrará a continuación en el programa.

Podremos observar que en programa del **ejemplo 3.26** sólo se escribirá el número cuando éste sea par, ya que la sentencia **continue** afecta a las repeticiones impares, y hace que en éstas se ejecute un salto hasta la siguiente repetición.

/ Ejemplo 3.26: Muestra números pares, si es impar se
brinca al siguiente ciclo */*

```
#include<iostream>
using namespace std;
int main() {
    int c;
    for (int c=1; c<=40; c++)
    {
        if (c%2==1)
            continue;
        cout << c << " -- ";
    }
    cout << "\n\n\t\t..... Programa terminado ..... \n";
    system ("pause");
    return 0;
}
```

goto

Esta instrucción puede ponerse en cualquier punto del programa, y da un salto hacia otro punto llamado referencia. Lo que significa que en el programa escribiríamos **goto** seguido de una referencia. El nombre de la referencia puede ser cualquier palabra siempre y cuando no sea reservada, ni use caracteres raros. En otro punto del programa escribiremos la referencia seguida de dos puntos (:), por ejemplo: **refer1:** y enseguida las instrucciones que se ejecutarán. Entonces para pasar el control a esa referencia se debería escribir: **goto refer1;**

La referencia puede estar antes o después del **goto**. Cuidado cuando el salto es hacia atrás, para no crear un bucle infinito, ya que el programa volverá otra vez a la sentencia goto.

Nota de goto:

La instrucción **goto** solo se presenta para que el programador la conozca, ya que esta sentencia va en contra de la programación estructurada, por lo que no se aconseja hacer uso de esta.

Enseguida se muestra un ejemplo usando **goto**:

*/*Ejemplo 3.27: Uso del goto para ejecutar otro bloque de instrucciones. Para salir del bloque se pulsa "s"*/*

```
#include<iostream>
using namespace std;
int main()
{
    cout << "Dame una letra (s para salir): " << endl;
    char c='a';
salto1:                //Referencia a salto1
    cin >> c;
    if (c=='s')
        goto salto2;    //Se brinca a salto2
    cout << "No es una s. Dame una letra: " << endl;
    goto salto1;        // Se brinca a salto1
salto2:                //Referencia a Salto2
    cout << "Escribiste una s. Programa termino \n";
    system ("pause");
    return 0;
}
```

3.6. Complemento: Buffer en C++

El **buffer** es un área de almacenamiento temporal en memoria que guarda los datos que se introduce por el teclado, y esa información es enviada a la computadora hasta que se pulsa la tecla Enter. También se utiliza en los archivos reteniendo los datos en tránsito desde y hacia el archivo, y tienen la finalidad de hacer más eficiente las operaciones de entrada/salida en los archivos de disco, provocando menor cantidad de accesos, los cuales son más lentos que la memoria.

Al momento de crear programas de **consola** con C++ muchas veces necesitamos ingresar datos de diferentes tipos a través de esta, sin embargo, en ocasiones nos encontramos con algunos comportamientos inesperados. Por eso es importante que el programador tenga un buen manejo del buffer y limpiarlo antes de querer ingresar datos de tipo carácter o cadena de caracteres (string).

Veamos el siguiente **ejemplo 3.28** que consiste en solicitar primero el nombre de una persona y después su edad, para después mostrar los datos en pantalla.

/ Ejemplo 3.28: Solicita primero el nombre y luego la edad, después los muestra */*

```
#include <iostream>
using namespace std;
int main ()
{
    string nombre;
    int edad;
```

```
cout<<"\nIngrese su Nombre: ";
getline(cin,nombre);
cout<<endl<<"\nIngrese su Edad: ";
cin>>edad;
cout<<"\nTu nombre: "<<nombre<<" y edad: " << edad
<< endl;
system ("pause");
return 0;
}
```

Como podemos observar el comportamiento del programa 3.28 es normal, ya que al ejecutarlo no presenta ningún problema y realizará lo que queremos. Pero qué pasa en el **ejemplo 3.29** donde invertimos el orden en que se solicitan los datos, es decir, primero la edad y después el nombre.

```
/* Ejemplo 3.29: Programa con error, solicita edad y
no permite capturar el nombre */
#include <iostream>
using namespace std;
int main () {
    string nombre;
    int edad;
    cout<<endl<<"Dame tu edad: ";
    cin>>edad;
    cout<<"Dame tu nombre: ";
    getline(cin,nombre);
    cout<<"\nNombre: "<<nombre<<" Edad: "<<edad<<endl;
    system ("pause");
    return 0; }
```


Podemos ver que el ejemplo 3.29 no funciona correctamente, ya que después de capturar la edad (numero) no se permitió ingresar el nombre y por lo tanto al momento de mostrar los datos no se hará de la manera adecuada. Lo que ocurre aquí es un error del programador al no hacer un buen manejo del buffer y limpiarlo antes de ingresar datos de tipo caracter. Cuando se ingresa por el teclado un valor numérico y se presiona la tecla Enter también se ingresa el carácter de fin de cadena '\n' y como los valores numéricos los tenemos con el método `cin >>edad`, se lee el valor numérico y descarta el carácter '\n' lo que quiere decir que éste se queda en el buffer hasta una nueva lectura, y cuando vayamos a leer los datos de nombre en el programa, la función `getline()` tomara el primer carácter que encuentre, en este caso '\n' y lo que tendremos será una cadena vacía.

Entonces cuando se lee un número y después una cadena en C++ se presentan errores, los cuales se pueden solucionar limpiando el buffer mediante dos maneras, usando el `cin.ignore()` y con `fflush(stdin)`. En el siguiente ejemplo 3.30 se utiliza `cin.ignore()` y en ejemplo 3.31 se usa `fflush()`, para el cual es necesario incluir la librería `stdlib.h`.

/ Ejemplo 3.30: Antes de capturar caracteres se limpia Buffer con cin.ignore() */*

```
#include <iostream>
#include <string>
using namespace std;
int main ()
{
    string nombre;
    int edad;
    cout<<endl<<"Dame tu edad: ";
```

```
    cin>>edad;
    cin.ignore();//Limpia Buffer para continuar
    cout<<"Ingrese su Nombre: ";
    getline(cin,nombre);
    cout << "\nTu Nombre es: " << nombre << " y tu
    edad es: " << edad << endl;
    system ("pause");
    return 0;
}
```

/ Ejemplo 3.31: Antes de capturar caracteres se
limpia el Buffer con flush() */*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main ()
{
    string nombre;
    int edad;
    cout<<endl<<"Dame tu edad: ";
    cin>>edad;
    fflush(stdin);    //Limpia Buffer para continuar
    cout<<"Ingrese su Nombre: ";
    getline(cin,nombre);
    cout << "\nTu Nombre es: " << nombre << " y tu
    edad es: " << edad << endl;
    system ("pause");
    return 0;
}
```

Actividad 3.2: Resolver los siguientes ejercicios de repetición usando for, while y do-while de C++:

1. Programa que lea 8 números flotantes e indicar cuánto es la suma de todos ellos.
2. Programa que lea 10 números e indicar cuál es la suma de los números pares.
3. Programa que solicite cuantos números quiere leer (**n**), capturar esos **n** valores y nos muestre cuantos fueron mayores a 500.
4. Leer una cantidad **n** de números enteros hasta que se ingrese el número 250. Mostrar el número de datos que fueron ingresados.
5. Obtener la factorial de un número **n**, siempre y cuando esté entre los valores 3 y 19. La factorial corresponde a la multiplicación de todos los números enteros positivos desde 1 hasta **n**. Por ejemplo, la factorial de 5 es: $5*4*3*2*1$ que equivale a 120. Tomar en cuenta que la factorial de 1 es 1.
6. Programa que permita ingresar 15 calificaciones que pueden ser entre 0 y 100, y nos muestre finalmente cuántas fueron aprobatorias (de 70 o más) y cuantas reprobadas.
7. Programa que solicite cuantos números quiere leer, capturar esos valores y nos muestre cuantos fueron mayores a 50 y cuántos menores a 100.
8. Programa que nos permita ingresar números, hasta que se lean 3 números pares o 4 números negativos.
9. Programa para leer las edades y los pesos de los alumnos de clase (20). Mostrar cual es la edad promedio y el peso promedio.
10. Pedir un número **n** entre 1 y 10, y escribir la tabla de multiplicar según el número capturado:
Dame número: 4 Salida: $4*1=4$, $4*2=8$, $4*3=12$, $4*10=40$.

Rúbrica de evaluación ejercicios del capítulo 3

	Excelente 10-8	Bueno 7-4	Deficiente 3-1	No realizado 0	Total
Estructuras secuenciales	Distingue y aplica correctamente el orden y la secuencia que se sigue para ejecutar una instrucción después de otra.	Aplica parcialmente el orden y la secuencia que se sigue para ejecutar una instrucción después de otra, sin embargo, llega a los resultados correctos.	Aplica la secuencia de instrucciones, sin embargo, tiene errores que generan el resultado de las operaciones sea incorrecto.	Desconoce el orden y secuencia que deben seguir las instrucciones secuenciales en C++.	
Estructuras selectivas	Conoce y aplica con razonamiento e interpretación lógica los siguientes tipos de estructuras: -Simple. -Doble. -Múltiple.	Conoce y aplica con razonamiento e interpretación lógica al menos dos de los siguientes tipos de estructuras: -Simple. -Doble. -Múltiple.	Conoce y aplica al menos un tipo de estructura selectiva.	No conoce del tema estructuras selectivas.	

Capítulo 3. Control de flujo

Estructuras iterativas	Conoce y aplica con razonamiento e interpretación lógica los siguientes tipos de estructuras: -Repetir mientras. -Hasta. -Desde.	Conoce y aplica con razonamiento e interpretación lógica al menos dos de los siguientes tipos de estructuras: -Repetir mientras. -Hasta. -Desde.	Conoce y aplica al menos un tipo de estructura iterativa.	No conoce del tema estructuras iterativas.	
Comentarios					

Capítulo 4: Organización de datos

Objetivos de capítulo 4:

- Presentar una introducción a la organización de datos dentro de un programa de computadora, conociendo el concepto de arreglo y sus diferentes aplicaciones en la construcción de programas.
- Crear programas que incluyan arreglos unidimensionales y bidimensionales para la solución de problemas.
- Explicar el concepto de estructuras y como las podemos utilizar en programación.

Subtemas de capítulo 4:



4.1 Arreglos

Hasta aquí hemos resuelto problemas utilizando variables que solo almacenan el valor de un dato en un momento dado (el valor puede cambiar, pero sólo es un valor en ese momento). Los cuales se definieron indicando el tipo y el nombre del identificador, por ejemplo:

```
char a;  
float sueldo;  
int valor;
```

En ocasiones es necesario elaborar programas donde se requiere agrupar muchos datos de un mismo tipo, por ejemplo, si quiero almacenar 38 calificaciones de un grupo de estudiantes no sería muy práctico tener que definir las 38 variables. Es aquí donde surge la necesidad de buscar otra forma de estructurar y almacenar los datos.

Un **arreglo** es una colección o grupo de elementos que pertenecen a un mismo tipo de dato, y están organizados de tal manera que se puede acceder a cada uno de ellos de manera independiente. En el ejemplo de almacenar 38 calificaciones, cada una de estas es un elemento el cual tiene una posición o lugar específico dentro del arreglo. Si queremos acceder a alguna de las 38 calificaciones solo debemos indicar la posición o índice que ésta tiene dentro del arreglo.

Para realizar la definición de un arreglo es necesario especificar el tipo de datos que almacenará, el nombre del arreglo y la cantidad de elementos que va a almacenar o guardar.

4.2 Arreglos unidimensionales

Un **arreglo unidimensional** es un conjunto finito de datos contiguos del mismo tipo. Un arreglo unidimensional de **n** elementos se podría representar como en la figura 4.1.

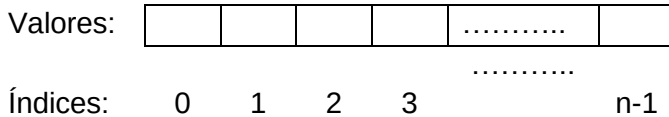


Figura 4.1. Representación de arreglo o vector

Observemos que la primera localidad del arreglo corresponde al índice **0** y la última corresponde al índice **n-1**, donde **n** es el número de elementos que almacenará el arreglo. El formato o la sintaxis para definir un arreglo en C++ se especifica de la siguiente manera:

Tipo Nombre [Tamaño];

Tipo nos indica el tipo de datos que se almacenarán en el arreglo (int, char, float, etc.). **Nombre** se refiere al Identificador del arreglo. Y **Tamaño** es un número entero y define el número máximo de elementos que puede almacenar el arreglo.

Para el ejemplo de guardar las calificaciones de un grupo de 38 alumnos usando C++, nos quedaría de la siguiente manera:

float calificaciones [38];

Al igual que en las variables, también es posible inicializar o dar valores a los arreglos al momento en que se definen, esto se hace de la siguiente manera:

```
int numeros [5] = {8, 6, 3, 9, 22};
```

Al momento en que se declara el arreglo **números** le estamos diciendo que almacenará **5** valores, y al mismo momento le especificamos estos valores (se inicializa). Claro está que también podemos estar solicitando al usuario estos valores.

Ejemplo 4.1: Leer 10 números enteros y almacenarlos en un arreglo. Mostrar la suma total de todos los números.

//Ejemplo 4.1: Guarda 10 números en arreglo y los suma

```
#include<iostream>
using namespace std;
int main()
{
    int i, suma=0, arreg[10];//Declara arreglo tamaño 10
    cout<<"Dame 10 valores enteros: \n";
    for(i=0; i<10; i++)           //Capturar 10 datos
    {
        cin>>arreg[i];           //Captura dato en posición i
        suma=suma+arreg[i]; //Se suma valor capturado
    }                             // Termina for
    cout<<"\n La suma total es: "<<suma<<endl;
    system ("pause");
    return 0;
}
```

Explicación de ejemplo 4.1:

En la línea 5 se define una variable llamada **i** que es nuestra variable de control y nos ayudará para indicar en qué posición almacenaremos cada dato capturado; también se define una variable llamada **suma** que nos servirá para ir acumulando cada dato capturado, por eso se inicializa en cero; además se define un arreglo de tamaño 10 llamado **arreg** que almacenará los diez valores.

Observe que **i**, **suma** y **arreg[]** almacenaran datos de tipo entero. En la línea 6 se muestra el mensaje: "Dame 10 valores enteros:", y después hace un salto de línea.

En la línea 7 utilizamos un ciclo **for()** indicando que las instrucciones dentro de él se ejecuten diez veces, observe que las posiciones dentro del arreglo son de 0 a 9. Y los incrementos serán de uno.

Las líneas 8 y 11 respectivamente nos indican el principio y fin de las instrucciones del **for()**.

La línea 9 nos sirve para capturar un dato y que se almacene en la posición **i** del arreglo **arreg**.

Observe que **i** ira cambiando, lo que nos indicara en qué posición del arreglo se almacenara el dato capturado (**i** empieza en 0 y termina en 9).

En la línea 10 se utiliza una variable llamada **suma**, la cual nos ayudara a ir acumulando cada uno de los datos enteros capturados, observe que al momento de definirla la inicializamos con el valor cero para asegurar que la variable no tenga nada almacenado.

La línea 12 nos ayuda para que se realice un salto de línea y después muestra en pantalla el mensaje: "**La suma total es:** " seguida por el valor de **suma** y terminando con un salto de línea.

Ejemplo 4.2: Programa que permita leer 8 números tipo real y los almacene en un arreglo, después cada elemento se multiplicará por el valor 3 y el resultado lo almacenará en otro arreglo. Al final mostrar el segundo arreglo.

/ Ejemplo 4.2: Lee 8 valores flotantes y los guarda en arreglo, multiplica cada valor por 3 y guarda los resultados en otro arreglo */*

```
#include<iostream>
using namespace std;
int main()
{
    float arr1[8], arr2[8];
    int b;
    cout<<"Dame ocho valores tipo real: "<<endl;
    for(b=0; b<8; b++)
    {
        cout<<"\n Dame valor "<<b+1<<": ";
        cin>>arr1[b];    //Captura dato en primer arreglo
        arr2[b]=arr1[b]*3; //Guarda en segundo arreglo
    }
    cout<<"\nEl resultado del segundo arreglo es: \n";
    for(b=0; b<8; b++)
    {
        cout<<"\n"<<arr2[b];
    }
    cout<<endl<<endl;
    system ("pause");
}
```

```
    return 0;  
}
```

En el ejemplo 4.2 observamos que los dos arreglos son tipo flotante y la variable **b** nos ayuda a ubicar la posición de cada elemento en el arreglo correspondiente. En el primer **for()** se lee el dato y luego se multiplica por el valor 3 y se guarda el resultado en el segundo arreglo.

Observe que usamos la misma variable **b** para las dos instrucciones **for()**, esto es válido ya que al principio de cada **for()** esta variable es inicializada en cero. También es posible solicitar cuantos elementos queremos en un arreglo y después declararlo del tamaño que se ha solicitado.

Ejercicio 4.1: Leer 18 datos tipo real y guardarlos en un arreglo, también leer 15 datos de tipo entero y guardarlos en otro arreglo. Sumar los valores de cada arreglo y mostrar total de las dos sumas, así como la división de la primera suma con la segunda suma.

Ejercicio 4.2: Programa para calcular el promedio de n números solicitando primero el número de elementos que desea almacenar en el arreglo.

4.3 Arreglos Bidimensionales

Un **arreglo bidimensional**, o de dos dimensiones, es un conjunto finito de datos del mismo tipo, en donde los elementos se almacenan en una tabla con filas y columnas. Este tipo de arreglos también pueden ser conocidos con el nombre de **matriz** y se puede representar como en la figura 4.2.

	0	1	2	3	4
0	34	25	14	25	38
1	29	32	16	28	25
2	6	55	60	49	50

Figura 4.2 Representación de arreglo bidimensional

En figura anterior, de manera horizontal se observan las filas y de manera vertical están las columnas. En cada celda se puede guardar un dato, y en total se almacenan 15 datos todos del mismo tipo, lo cual se especifica al momento de declarar la matriz.

En la figura observamos a 15 datos enteros distribuidos en tres filas y cinco columnas. Para acceder a un dato específico es necesario indicar el índice o posición, y en los arreglos bidimensionales se requiere indicar el número de la fila y el número de la columna. Por ejemplo, con la fila 1 y la columna 3 estaríamos accediendo al valor 28.

El formato o la sintaxis para definir un arreglo bidimensional en C++ sería de la siguiente forma:

Tipo Nombre [n] [m];

Donde **Tipo** nos indica el tipo de datos que se almacenaran en el arreglo (int, char, float, etc.). **Nombre** se refiere al Identificador que le damos al arreglo. El valor de **n** representa el número de filas y el valor de **m** el número de columnas en el arreglo.

Si al ejemplo de la figura 4.2 yo lo quisiera identificar con el nombre de **Matrizej**, tendríamos que declararlo de la siguiente manera en C++:

```
int Matrizej [3] [5];
```

Algunos ejemplos de declaración de matrices podrían ser:

```
char Letras[6][4];//Declara matriz para 24 caracteres
```

```
float Matreal [8] [8]; //Matriz para valores flotantes
```

Al igual que los vectores, a las matrices se les puede asignar un valor de inicio de la siguiente manera:

```
int ABC[4][3]={8,6,22},{3,9,22},{9,8,12},{1,9,6,8}};
```

Para acceder a un elemento almacenado en una matriz necesitamos especificar **dos posiciones** donde la primera indica el **renglón**, seguido por el número de **columna**. Ejemplos:

Mat[0][3] Se refiere al elemento ubicado en el primer renglón y cuarta columna.

Mat[3][3] Se refiere al elemento ubicado en el cuarto renglón y cuarta columna.

Mat[2][0] Se refiere al elemento ubicado en el tercer renglón y primera columna.

Para acceder a cada uno de los elementos de una matriz podemos usar cualquier instrucción repetitiva, específicamente necesitamos de dos de estas, una para recorrer los renglones y otra para las columnas. Enseguida se muestran ejemplos usando **for** anidados.

Ejemplo 4.3: Se desea pedir los valores enteros en una matriz de 3 renglones por 4 columnas, para almacenarlos en ésta, y después mostrar en pantalla el primer y último elemento capturado.

/ Ejemplo 4.3: Pide los valores de una matriz y muestra el primer y último valor */*

```
#include<iostream>
using namespace std;
int main()
{
    int mat[3][4],x,y; //Declaración de matriz de 3x4
    cout<<"Dame los valores de la matriz: \n";
    for(x=0; x<3; x++) //Para moverse en cada renglón
    {
        for(y=0; y<4; y++) //Para moverse por la columna
        {
            cout<<"Dame valor: ";
```

```

        cin>>mat[x][y];    //x y y para indicar posición
    }
}
cout<<"\nPrimer valor capturado: "<<mat[0][0];
cout<<"\nUtimo valor capturado: "<<mat[2][3];
system ("pause");
return 0;
}

```

Ejemplo 4.4: Pedir los valores de una matriz de 4 renglones y 2 columnas. Después usar otros ciclos para mostrar en pantalla todos los datos capturados.

```

/* Ejemplo 4.4: Pide valores de una matriz de 4x2 y
mostrar todos los datos */
#include<iostream>
using namespace std;
int main()
{
    int mat[4][2], x, y;
    cout<<"Dame los valores de la matriz: \n";
    for(x=0; x<4; x++)    //Ciclo externo para renglones
    {
        for(y=0; y<2; y++) //Ciclo interno para columnas
        {
            cout<<"Dame valor: "<<x+1<<","<<y+1<<": ";
            cin>>mat[x][y];
        }
        //Fin de ciclo interno
    }
    //Fin de ciclo externo
    cout<<"\nLos valores capturados son: \n";
    for(x=0; x<4; x++)

```



```
{
    cout<<endl;
    for(y=0; y<2; y++)
        cout<<"  "<<mat[x][y];
}
cout<<endl<<endl;
system ("pause");
return 0;
}
```

Ejercicio 4.3: Pedir los valores reales de una matriz de 4 renglones y 3 columnas. Obtener el promedio de los valores almacenados en la matriz y mostrarlo en pantalla.

Ejercicio 4.4: Pedir los valores reales de dos matrices de 5 renglones y 3 columnas. Mostrar el resultado de una tercera matriz que corresponde a la suma de los elementos de las primeras dos matrices.

Ejercicio 4.5: En una matriz queremos almacenar las calificaciones de 6 alumnos que presentaron 3 exámenes. Una vez capturadas las calificaciones, queremos obtener el promedio de cada alumno y guardarlo en otro arreglo para mostrarlos en pantalla y decir cuál promedio es el mayor.

4.4 Estructuras o registros

A diferencia de los arreglos que guardan datos del mismo tipo, la **estructura** es una colección de uno o más elementos que nos permite almacenar un conjunto de datos de diferente tipo, los cuales pueden ser datos simples, datos compuestos como arreglos y listas, o bien, otras estructuras. Aquí, cada uno de los

elementos se define de manera independiente dentro de la estructura ya que puede variar su tipo. A los elementos de la estructura o registro también se les conoce como campos.

Una manera de definir a una estructura en lenguaje C++ es la siguiente:

```
struct
{
    Elemento1;
    Elemento2;
    :
    :
    Elemento n;
} nombre;
```

En la declaración aparece la palabra reservada **struct** que nos indica que es una estructura, enseguida se especifican cada uno de los elementos indicando su tipo y nombre. Y al final se especifica el **nombre** que se la asigna a la variable estructura.

En el siguiente ejemplo se define la variable **persona** que es de tipo estructura, la cual está conformada por los tres campos llamados **nombre**, **edad** y **nota** y cada uno tiene su tipo.

```
struct
{
    string nombre;
    int edad;
    float nota;
} persona;
```

Una vez que tenemos definida la estructura llamada **persona**, podremos acceder a cada uno de los campos escribiendo el nombre de la variable estructura seguida por un punto ".", y luego el nombre del campo. Pudiendo obtener, almacenar o modificar la información contenida en cada campo.

En el **ejemplo 4.5** se almacena información a cada uno de los tres campos y enseguida se muestra la información almacenada en el campo edad.

// Ejemplo 4.5: Declaración y acceso de una estructura

```
#include<iostream>
using namespace std;
int main()
{
    struct
    {
        string nombre;
        int edad;
        float nota;
    } persona; //Variable persona de tipo estructura

    persona.nombre="Carlos";
    persona.edad=51;
    persona.nota=88;
    cout << "La edad es: " << persona.edad;
    system ("pause");
    return 0;
}
```

Otra forma de definir a una estructura es escribir el **nombre** después de la palabra **struct**, es decir, estamos definiendo un nuevo tipo de estructura. Enseguida se definen los campos, y tras el cierre de este bloque debe aparecer un punto y coma.

```
struct correo
{
    unsigned int numpaquete;
    char persona[40];
    char calle[30];
    char Ciudad[25];
    unsigned long int código;
};
```

Observemos que el nombre de la estructura es **correo**, formada por los cinco campos siguientes:

numpaquete es un campo que puede almacenar un dato entero sin signo.

persona [40] se usa para el nombre de la persona que puede almacenar hasta 40 caracteres.

calle [30] se usa para el nombre de calle y puede almacenar hasta 30 caracteres.

Ciudad [25] se usa para el nombre de la ciudad y puede almacenar hasta 25 caracteres.

codigo se usa para almacenar un dato entero largo sin signo.

En el ejemplo de la estructura **correo** solo hemos definido la forma de los datos de la estructura (el tipo), ahora necesitamos declarar

las variables del tipo **correo**, y así poder acceder a los datos. Para declarar a una variable del tipo de estructura definido se escribe de la siguiente manera:

struct Nombre_estructura Nombre_variable

Siguiendo el mismo ejemplo de **correo**, podríamos declarar a una variable que se llama **info_cor** y ésta es de tipo **correo**, aunque en algunos compiladores no es necesario escribir la palabra **struct**.

struct correo info_cor

En el **ejemplo 4.6**, que se muestra enseguida, se define el tipo llamado Datpersona que está formado por tres campos y para poderlos usar es necesario declarar a la variable, en este caso se llama persona. Se guardan valores y muestran en pantalla.

Veamos la solución de este problema por medio del siguiente programa:

```
// Ejemplo 4.6: Declaración y acceso de una estructura
#include<iostream>
#include<string>
using namespace std;
int main() {
    struct Datpersona    //Se define el tipo Datpersona
    {
        string nombre;
        int edad;
```

```
        float nota;
    }; // Termina la definición de la estructura

//Enseguida se declara persona del tipo Datpersona:
    Datpersona persona;
//Enseguida almacena dato en campo nombre:
    persona.nombre="Carlos";
    persona.edad=51;      //Almacena dato en campo edad
    persona.nota=88;      //Almacena dato en campo nota
    cout << "El nombre es: " << persona.nombre << endl;
    cout << "La edad es: " << persona.edad;
    system ("pause");
    return 0;
}
```

En el siguiente **ejemplo 4.7** se definen dos variables (pers1 y pers2) del tipo **Datpersona**. La variable **pers1** es inicializada con datos y después, la información de pers1 se almacena en **pers2**. Enseguida se muestra la información de pers2, después se le hacen modificaciones y se vuelve a mostrar en pantalla.

```
//Ejemplo 4.7: Iniciación y asignación de estructuras
#include<iostream>
#include<string>
using namespace std;
struct Datpersona // Tipo llamado Datpersona
{
    string nombre;
    int edad;
};
```

```
int main()
{
    //Enseguida se inicializa con datos en nombre y edad
    Datpersona pers1 = {"Carlos", 30};
    //Enseguida se copia el contenido de pers1 a pers2
    Datpersona pers2 = pers1;
    cout << "Datos iniciales de persona 2: " << endl;
    cout << "Nombre: " << pers2.nombre << endl;
    cout << "Edad: " << pers2.edad << endl << endl;
    system ("pause");
    pers2.nombre = "Alonso";    //Cambia nombre de pers2
    pers2.edad++; //Aumenta en uno al campo edad de pers2
    cout << "\nDatos ya modificados de persona 2: " <<
    endl;
    cout << "Nombre: " << pers2.nombre << endl;
    cout << "Edad: " << pers2.edad << endl;
    system ("pause");
    return 0;
}
```

4.4.1 Estructuras anidadas

Una estructura anidada es, cuando dentro de una estructura se incluye a un campo que sea del tipo de otra estructura.

En el siguiente **ejemplo 4.8** podemos observar que se define la estructura llamada **Trabajador**, que contiene el campo **nacimiento** que a su vez es del tipo **Fecha**. También podemos ver como se utilizan las llaves para inicializar una estructura anidada especificando cada uno de los campos. Es necesario especificar la ruta para acceder a cada uno de los campos y mostrarlos en

pantalla. Resumiendo, las definiciones que aparecen en el programa podemos observar lo siguiente:

// Ejemplo 4.8: Se utilizan las estructuras anidadas

```
#include<iostream>
#include<string>
using namespace std;

struct Fecha      //Define tipo Fecha y sus campos
{
    int dia, mes, anio;
};

struct Trabajador //Define tipo Trabajador y campos
{
    string nombre;
    int nivel;
    Fecha nacimto;    //nacimto es de tipo Fecha
};

int main()
{
    // Enseguida se almacenn datos en tr
    Trabajador tr = {"Jesus Camacho",2,{10,06,2002}};

    // Enseguida se muestran datos que estan en tr

    cout << "Nombre: " << tr.nombre << endl;
    cout << "Rango de trabajo: " << tr.nivel << endl;
    cout << "Fecha nacimiento: "<<tr.nacimto.dia << "-"
```



```
<<tr.nacimto.mes<<"-"<< tr.nacimto.anio<<endl;
system ("pause");
return 0;
}
```

En **Trabajador** *tr*, *tr* es una variable de tipo **Trabajador**.

tr.nacimto es una variable de tipo **Fecha**

tr.nacimto.día, **tr.nacimto.mes** y **tr.nacimto.anio** son variables de tipo entero.

Observemos el siguiente **ejemplo 4.9**, en el cual se define una estructura llamada **Registro**, la cual hace uso de otras dos estructuras llamadas **Persona** y **Fecha**, para posteriormente definir a la variable **Ingreso** del tipo **Registro**. Se capturan los datos y posteriormente se muestran en pantalla. Observe como se utiliza el carácter punto para acceder a cada uno de los datos.

// Ejemplo 4.9: Utilizando estructuras anidadas

```
#include <iostream>
#include <string>
using namespace std;
struct Persona
{
    string nombre, apellido;
    int edad, Identificacion;
};
```

```
struct Fecha
```

```
{
    int dia, mes, anio;
};

struct Registro
{
    Persona persona;    //persona es del tipo Persona
    Fecha fec;          //fec es del tipo Fecha
};

int main()
{
    Registro Ingreso; //Se define variable tipo Registro
    cout << "\nIngresar nombre: ";
    getline(cin, Ingreso.persona.nombre);
    cout << "\nIngresar apellido: ";
    getline(cin, Ingreso.persona.apellido);
    cout << "\nIngresar edad: ";
    cin >> Ingreso.persona.edad;
    cout<<"\nIngresa Identificacion (3 digitos): ";
    cin >> Ingreso.persona.Identificacion;
    cout << "\nDia de nacimiento: ";
    cin >> Ingreso.fec.dia;
    cout << "\nMes de nacimiento: ";
    cin >> Ingreso.fec.mes;
    cout << "\nAño de nacimiento: ";
    cin >> Ingreso.fec.anio;
    cout<<"\n\n Los datos capturados son: "<<endl;
    cout << "\n\nNombre: " <<
    Ingreso.persona.nombre << endl;
```

```
cout << "\nApellido: " <<
Ingreso.persona.apellido << endl;
cout<<"\nEdad: "<<Ingreso.persona.edad<<endl;
cout<<"\nNo. Identificacion: "
<<Ingreso.persona.Identificacion<< endl;
cout << "\nDia de nacimiento: " <<
Ingreso.fec.dia << endl;
cout << "\nMes de nacimiento: " <<
Ingreso.fec.mes << endl;
cout << "\nAño de nacimiento: " <<
Ingreso.fec.anio << endl;
system ("pause");
return 0;
}
```

4.4.2 Arreglo de estructuras

Ya que conocemos las estructuras de datos, podemos decir que una de las partes donde son utilizadas con frecuencia es en los arreglos de estructuras, que son donde se puede representar a un conjunto de entidades de cualquier tipo, por ejemplo, para llevar el control de los clientes de una tienda, los medicamentos de una farmacia, o bien, los trabajadores de una industria.

En el **ejemplo 4.10** se define un arreglo de estructuras llamado **alu** de tamaño 2 y del tipo **Alumno**. La estructura **Alumno** utiliza a un campo llamado **nac** y es del tipo **Fecha**, también tiene a un campo llamado **nombre** que es de tipo **string** y un tercer campo que es un arreglo llamado **califas** que almacena 3 números **float**. En el programa se guarda información en cada uno de los campos y posteriormente se muestra la información.

```
/* Ejemplo 4.10: Se declara un arreglo y almacena
información */
#include<iostream>
using namespace std;

struct Fecha
{
    int dia, mes, anio;
};
struct Alumno
{
    Fecha nac;
    string nombre;
    float califas[3];
};
int main()
{
    Alumno alu [2] ={{{14,12,1968},"Carlos", {8,7,9}},
    {{10, 6, 1970}, "Paty macias", {10,9,8}} };

    for (int i=0; i<2; i++)
    {
        cout << "\n Nombre: " << alu[i].nombre;
        cout <<" (" << alu[i].nac.dia << "/" <<
        alu[i].nac.mes << "/" <<
        alu[i].nac.anio << ")";
        cout << "\tNotas: "<< alu[i].califas[0]<<" "
        <<alu[i].califas[1] << " "<<
        alu[i].califas[2] << endl;
    }
}
```

```
system ("pause");  
return 0;  
}
```

En el **ejemplo 4.11** aparece un ejemplo de programa donde aparece una estructura llamada **Perro**, la cual contiene a los elementos **nombre** y, **edad** y **peso**. Después se define un arreglo llamado **perros** del tipo Perro y que almacena la información de 4 perros.

Observemos a continuación la solución al problema planteado:

```
/*Ejemplo 4.11: Se usa arreglo de cuatro elementos  
llamado perros del tipo Perro */  
#include<iostream>  
  
#include<string>  
  
#include<stdlib.h>  
  
using namespace std;  
  
struct Perro  
{  
    char nombre[30];  
    int edad;  
    float peso;  
};  
  
int main()
```

```
{
    Perro perros[4];
    cout<<"***** Dame los datos *****\n";
    for(int i=0; i<4; i++)
    {
        cout << endl;
        fflush(stdin);
        cout<<"\tDame el nombre del perro "<<i+1<<": ";
        cin.getline(perros[i].nombre, 30, '\n');
        cout << "\tDame edad de " <<
            perros[i].nombre << ": ";
        cin >> perros[i].edad;
        cout << "\tDame el peso de " <<
            perros[i].nombre << ": ";
        cin >> perros[i].peso;
    }
    cout<<"\nEl nombre del tercer perro capturado
    es: "<< perros[2].nombre;
    cout<<"\nEl nombre del segundo perro capturado
    es: "<< perros[1].nombre;
    system ("pause");
    return 0;
}
```

En el **ejemplo 4.12** aparece un programa que utiliza una estructura la cual contiene a los elementos **nombre** y **código** que representan los datos de un **usuario**. Se define a una variable llamada **v [40]** que es un arreglo de tamaño 40 y del tipo **usuario**. Se pedirán estos datos hasta que se presione una opción diferente a 's'. Al terminar mostrar el tercer registro.

/ Ejemplo 4.12: Captura nombre y codigo de usuario mientras se este presionando s. Al final muestra los datos capturados del tercer elemento */*

```
#include<iostream>
#include<stdlib.h>
using namespace std;
struct usuario // Se declara estructura llamada
{
    char nombre[20];
    int codigo;
};

int main()
{
    usuario v[40]; //Se declara arreglo de tipo usuario
    int cant=0;
    char opcion;
    do
    {
        fflush(stdin);
        cout<<"Dame Nombre: ";
        cin.getline(v[cant].nombre, 20, '\n');
        cout<<"Dame Codigo: ";
```

```

    cin>>v[cant].codigo;
    cout<<"Desea otro registro (s/n): ";
    cin>>opcion;
    cant++;
}
while(opcion=='s'); //Mientras opcion sea igual a s
//Enseguida se muestran datos del registro 3:
cout<<v[2].codigo<<endl;
cout<<v[2].nombre<<endl;
system ("pause");
return 0;
}

```

En el **ejemplo 4.13** se muestra un programa que almacena la información de jugadores de un equipo de futbol. Utiliza un arreglo del tipo JUGADOR que puede almacenar los datos de hasta 20 jugadores. Primero se pide el nombre del equipo y enseguida la información de los integrantes de este. Cada vez pregunta si desea capturar a otro jugador (s/n).

```

/* Ejemplo 4.13: Almacena información de hasta 20
jugadores de un equipo */

```

```

#include<iostream>
#include<string>
#include<stdlib.h>
using namespace std;
struct JUGADOR
{
    char nomJugador[30];
    int edJugador;
    float pesJugador;
}

```



```
};  
int main()  
{  
    JUGADOR jugador[20];  
    int numJugador=0, contador=0, out;  
    double pesProm=0, edadProm=0;  
    char resp;  
    char nomEquipo[25];  
    cout << "\t TORNEO DE FUTBOL DEL PACIFICO" << endl;  
    cout<<"-----"<<endl;  
    cout << "Digite el nombre del equipo: ";  
    cin.getline(nomEquipo, 25, '\n');  
    for (int i=0; i<20; i++)  
    {  
        fflush(stdin);  
        cout << "\nDigite el nombre del jugador # " <<  
        ++numJugador << ": ";  
        cin.getline(jugador[i].nomJugador, 30, '\n');  
        cout << "Digite el peso de " << jugador[i].nomJugador  
        << ": ";  
        cin>>jugador[i].pesJugador;  
        pesProm += jugador[i].pesJugador;  
        cout<<"Digite edad de "<<jugador[i].nomJugador<<": ";  
        cin>>jugador[i].edJugador;  
        edadProm += jugador[i].edJugador;  
        cout << "desea agregar mas jugadores? s/n: ";  
        cin >> resp;  
        if (resp == '\n')  
            break;  
    }  
    system ("cls");
```

```

cout << "\t TORNEO DE FUTBOL DEL PACIFICO" << endl;
cout<<"\t-----" << endl;
cout << "Nombre del equipo: " << nomEquipo << endl;
cout << "-----" << endl;
cout << "Resultados: " << endl;
for (out=0; out<numJugador; out++ ) {
    cout<<"Nombre: "<<jugador[out].nomJugador<<"\tpeso:
    "<<jugador[out].pesJugador<<"kg. "<<"edad: "<<
    jugador[out].edJugador<<endl;
}
    cout<<"-----" <<endl;
    cout << "Peso Promedio del equipo: " << pesProm/out
    << "kg" << endl;
    cout << "Edad Promedio del equipo: " <<
    edadProm/out << " años" << endl;
    system ("pause");
    return 0;
}

```

Ejemplo 4.14: Se necesita almacenar datos de álbumes musicales (máximo 50 registros), que contenga la **categoría**, el **artista** y el **título**. Observe como se define el arreglo de estructuras llamado **álbum** del tipo **música** y tamaño 50. Primero se solicita cuantos registros necesita capturar y enseguida se piden los datos de cada uno. Después de capturados los mostrara en pantalla.

```

/* Ejemplo 4.14: Arreglo de estructuras que almacena
datos de álbum musical */
#include<iostream>

```

```
#include<stdlib.h>

using namespace std;

struct musica //Se define la estructura llamada música
{
    char categoria[15];
    char artista[20];
    char titulo[25];
};

int main() {
    musica album[50]; //Define un arreglo de tipo música
    int i=-1, conta=0;
    cout<<"Dame el numero de álbumes a registrar:";
    cin>>conta;
    while(++i<conta) //Se incrementa i antes de comparar
    {
        cout<<"\nDame la Categoria "<<i+1<<": ";
        fflush(stdin);
        cin.getline(album[i].categoria, 15, '\n');
        cout<<"\nDame el Artista "<<i+1<<": ";
        fflush(stdin);
        cin.getline(album[i].artista, 20, '\n');
        cout<<"\nDame el Titulo "<<i+1<<": ";
        fflush(stdin);
        cin.getline(album[i].titulo, 25, '\n');
    }
    cout<<"\nLOS DATOS CAPTURADOS SON: \n";
    for(i=0; i<conta;i++) {
        cout<<endl<<i+1<<": "<<album[i].categoria;
        cout<<" | "<<album[i].artista;
```

```
        cout<<" | "<<album[i].titulo<<endl;
    }
    system ("pause");
    return 0;
}
```

Dentro de la organización de datos, un tema muy importante es el almacenamiento de información de manera permanente por medio de archivos.

Actividad 4.1: Realizar los siguientes ejercicios utilizando lo aprendido con arreglos y estructuras:

1. Programa que solicite 18 valores tipo real y los guarde en un arreglo, después mostrar cuál de estos es el menor y en qué posición se encuentra dentro del arreglo.
2. Programa que capture 15 valores y los guarde en un primer arreglo. Después copiar los valores del primer arreglo en otro segundo arreglo, pero en orden inverso a como fueron capturados. Mostrar los valores de los dos arreglos.
3. Programa que primero solicite las dimensiones de una matriz, después declarar la matriz según las dimensiones y capturar los datos de la matriz. Enseguida mostrar los datos de la matriz, así como el número menor de todos y su posición dentro de la matriz.
4. Programa que se utilice para almacenar el número de clientes atendidos por 15 agentes de ventas, se solicitará el número del agente y la cantidad atendida. Después el programa pedirá el número de agente y mostrará en pantalla el número clientes.
5. Programa que trabaje con una estructura, donde estará formada por seis elementos: número de paquete, nombre de persona, nombre de calle, número de casa, ciudad y código postal. Solicitar los datos de 9 paquetes y mostrar cuales tienen el mismo código postal.

Rúbrica de evaluación ejercicios del capítulo 4

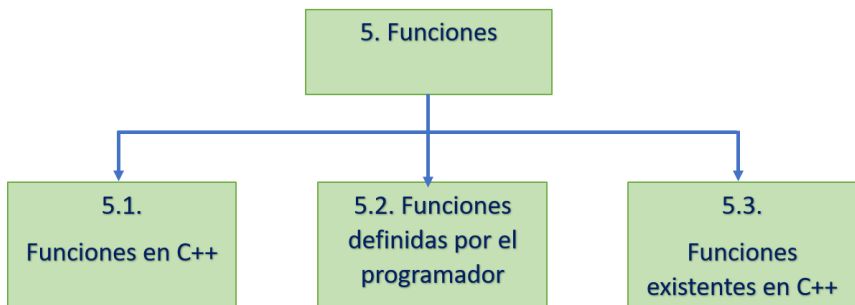
	Excelente 10-8	Bueno 7-4	Deficiente 3-1	No realizado 0	Total
Arreglos unidimensionales	Distingue y aplica correctamente la creación de arreglos unidimensionales: -Declaración -Tipos de datos -Asignación de valores.	Crea arreglos unidimensionales, sin embargo desconoce cómo hacer la asignación de valores correctamente.	Únicamente declara la creación de arreglos unidimensionales, no domina el tipo de datos de acuerdo a la asignación de valores que utiliza.	Desconoce el tema de creación de arreglos unidimensionales.	
Arreglos multidimensionales	Distingue y aplica correctamente la creación de arreglos multidimensionales: -Declaración -Tipos de datos -Asignación de valores en las diferentes dimensiones	Crea arreglos multidimensionales, sin embargo desconoce cómo hacer la asignación de valores en el total de dimensiones utilizadas.	Únicamente declara la creación de arreglos no domina el tipo de datos de acuerdo a la asignación de valores que utiliza.	Desconoce el tema de creación de arreglos multidimensionales.	
Estructuras o registros.	Distingue y aplica correctamente las instrucciones para crear estructuras. -Declaración -Tipos de datos -Asignación de valores.	Crea la estructura, sin embargo desconoce cómo hacer la asignación correcta de los valores.	Únicamente declara la creación de estructuras, no domina el tipo de datos de acuerdo a la asignación de valores que utiliza.	Desconoce el tema de creación de estructuras.	
Comentarios					

Capítulo 5: Modularidad

Objetivos de capítulo 5:

- Aplicar el concepto y utilidad de los módulos o funciones dentro de la programación, que ayuden a la organización y solución de problemas complejos.
- Conocer y utilizar el paso de parámetros o argumentos.
- Implementación de módulos

Subtemas de capítulo 5:



5.1 Módulos o funciones en C++

Cuando los programas se vuelven más grandes, cosa que lógicamente sucede cuando aumenta la complejidad del problema a resolver, provoca que el número de instrucciones aumente considerablemente, de modo tal que el programador tiene muchas dificultades para controlar ese gran número de instrucciones. Para resolver este problema, los programas se pueden descomponer o dividir en partes más pequeñas llamadas **módulos o funciones**.

El hacer esta división es un concepto o herramienta muy utilizada por los programadores. A estas partes o fragmentos de programa se les conoce como **funciones** (módulos, subprocesos, procedimientos, subprogramas o subrutinas en otros lenguajes de programación). Por lo tanto, una **función** es un subprograma que encapsula código (tiene sus propias instrucciones) dentro del programa total, y de la misma manera que identificamos a variables y estructuras la función debe tener su propio nombre.

El fragmentar nos permite tener un mejor control de todo el programa y además utilizar estas partes de código cuantas veces sea necesario. Cada **función** tiene un propósito bien definido y resuelve una tarea concreta, además puede estar comunicada con otras funciones por medio de los argumentos que puede recibir y por los valores que retorna. Si observamos, hasta aquí hemos utilizando el concepto de función en todos nuestros programas ya que siempre existe una función llamada **main ()** que representa a la función principal.

Clasificación de módulos o funciones en C++

Hablando de funciones dentro del lenguaje de programación C++, estas pueden ser clasificadas de la siguiente manera:

- **Funciones definidas por programador**, son todas las funciones que pueden ser diseñadas y creadas por el mismo programador dentro del programa total. En éstas se define que es lo que la función debe realizar por medio de las instrucciones incluidas.
- **Funciones existentes en el lenguaje**, son aquellas que ya están creadas y almacenadas en librerías o bibliotecas del lenguaje de programación. Al incluir una librería en nuestros programas, podemos tener acceso a un determinado grupo de funciones que ahí existen.

5.2 Funciones definidas por el programador en C++

Las funciones creadas por el programador deben estar formadas por:

- Un **identificador** de función (nombre), se utiliza para llamar o invocar a la función.
- **Parámetros** o argumentos que son variables o estructuras que se van a utilizar dentro de la función, los cuales pueden ser de **cero** a **n** parámetros.
- **Valor de retorno**, después de ejecutar una función comúnmente se puede regresar o retornar un valor (resultado). Es por esto por lo que antes del nombre de la función se especifica un tipo de dato a retornar.

El formato por seguir para construir una función es el siguiente:


```
Tipo_retorno  Nombre_Funcion (parámetros)  
{  
    .....  
Instrucciones_de_funcion;  
    .....  
return val;  
}
```

Algunos puntos importantes para tomar en cuenta para la creación de funciones son:

- En todos los programas elaborados en este documento siempre aparece una función llamada `main ( )`, que es la función principal del programa, y es la primera que se ejecuta al correr el programa.
- Para poder utilizar una función ya existente dentro del programa solo tendremos que "llamarla" o "invocarla" por medio de su nombre, y si es necesario, enviarle valores llamados argumentos o parámetros encerrados entre paréntesis.
- Para declarar los parámetros en la función es necesario indicar el tipo y nombre de cada uno de ellos. Un ejemplo que donde indique que la función va a recibir dos parámetros de tipo entero puede ser: nombre (`int x, int y`)
- El compilador de C++ revisa que las funciones estén definidas o declaradas antes de ser invocadas. Por lo que una buena práctica es declarar todas las funciones al inicio del programa, o bien, especificar con anterioridad el prototipo de la misma, la cual será creada más adelante.

- La diferencia entre un parámetro y un argumento es: un parámetro representa un valor que el procedimiento espera que se pase al llamarlo. La declaración del procedimiento define sus parámetros, mientras que un argumento representa el valor que se pasa a un parámetro de procedimiento al llamar a este último.

Veamos algunos ejemplos sencillos donde podemos encontrar el uso de funciones creadas por el usuario:

Ejemplo 5.1: Programa que capture una distancia en kilómetros y la convierta en metros y muestre el resultado. Se utiliza una función que llamamos **proc1()**.

```
/* Ejemplo 5.1: Pide valor en kilómetros, y llama a  
función proc1 para convertir a metros */
```

```
#include<iostream>  
using namespace std;
```

```
/* Enseguida se define función proc1or que retorna un  
double y recibe un argumento en variable a */
```

```
double proc1(float a)  e  
{  
    double b;          // Variable tipo double  
    b = a * 1000;  
    return b;          //Regresa valor guardado en b  
}
```

```
int main()
```

```
{  
    float km;  
    double metros;  
    cout << "\nDame distancia en kilometros: ";  
    cin >> km;  
    //Se llama a función proc1 y se le envía km:  
    metros = proc1(km);  
    cout<<"\nLa distancia en metros es:"<<metros;  
    system ("pause");  
    return 0;  
}
```

Explicación de ejemplo 5.1:

En el ejemplo anterior observamos que aparecen dos funciones dentro del programa, la función principal **main()** y la función llamada **proc1()**. La función **proc1()** recibe un argumento de tipo flotante llamado **a**, lo que significa que al momento de llamar a esta función es necesario enviarle un valor de este mismo tipo.

En la definición de **proc1()**, antes de escribir el nombre de la función, también estamos indicando que la función nos regresará un valor de tipo **double**, lo cual vemos en instrucción **return b**;

Observamos que el valor recibido en la variable **a** posteriormente es utilizado en una operación aritmética para poder obtener la conversión a metros. Y después de trabajar con el valor recibido simplemente se retorna el valor **b**. El valor que es retornado por **proc1()** se regresa exactamente al lugar donde la función fue invocada o llamada, por lo tanto, en nuestro ejemplo el resultado

es almacenado en la variable llamada **metros**, que es del mismo tipo de retorno de **proc1()**.

Ejemplo 5.2: Aquí se usa una función llamada **mifuncion()** que recibe un argumento para que se realice una operación matemática y el resultado obtenido es retornado a la parte donde se invocó a **mifuncion()**. Dentro de la función **main()** el valor regresado se guarda en la variable **tot**.

```
/* Ejemplo 5.2: Se invoca a función enviándole valor
entero y regresa valor entero */
#include<iostream>
using namespace std;

//Se define función mifuncion() y recibe parametro a:
int mifuncion(int a)
{
    int res;    //Variable res dentro de la función
    cout << "Ejemplo de funcion "<<a;
    res = a + 5;
    return res; //Se retorna un valor tipo int
}
int main(){
    int c, tot;
    cout << "Dame un valor entero: ";
    cin >> c;
    tot = mifuncion(c); //Se llama a mifuncion
    cout << "\nEl valor regresado por mi funcion es:
\n"<< tot;
    system ("pause");
```

```
    return 0;
}
```

Ejemplo 5.3: Programa con una función llamada `mult()` la cual recibe dos argumentos que son almacenados en las variables `a` y `b` para posteriormente utilizarlos en una operación de multiplicación y por ultimo regresar el resultado.

*/*Ejemplo 5.3: La función mult() recibe 2 argumentos y retorna un valor entero */*

```
#include<iostream>
using namespace std;

// Se define funcion mult que recibe 2 valores:
int mult(int a, int b)
{
    int res;
    res = a * b;
    return res;        //La función regresa valor
}

int main()
{
    int c, d, tot;
    cout<<"Dame dos valores enteros para
multiplicar: ";
    cin>>c;
    cin>>d;
    tot = mult(c,d); //Se envían 2 valores a mult()
    cout << "El valor que me regreso mult() es: " <<
tot;
    system ("pause");
}
```

```
    return 0;  
}
```

Explicación de ejemplo 5.3:

Se crea una función llamada **mult()** en donde indicamos que recibirá dos argumentos de tipo entero llamados **a** y **b**, los cuales son solo conocidos dentro de esta función; además se especifica que la función retornará un valor tipo entero: **int mult (int a, int b)**.

Dentro de la función **mult()** se realizan las instrucciones indicadas para obtener un resultado y guardarlo en variable **res** que es de tipo **int**. El valor obtenido es regresado al punto donde se indicó el llamado a la función **mult()**. Después de crear nuestra función **mult ()**, esta puede ser invocada desde cualquier parte del programa.

Dentro de **main()** se capturan dos valores, los cuales son enviados a otra función por medio de: **tot=mult(c, d)**. Los valores son recibidos en función **mult ()**, y una vez realizadas las operaciones se obtiene resultado, el cual es regresado con la instrucción **return**.

Prototipo de una función

Dentro de los puntos importantes mencionados, también dijimos que en un programa se puede escribir el código de una función después de **main()**, siempre y cuando se especifique con anterioridad el prototipo de la función a crear. Veamos el **ejemplo 5.4**, el cual tiene una función llamada **EncMayor()** que recibe dos números y me dice cuál de ellos es el mayor.

/ Ejemplo 5.4: Primero se define el prototipo de la función y después las instrucciones de esta */*

```
#include<iostream>
```

```
using namespace std;
```

```
//Enseguida se define prototipo de función EncMayor:
```

```
int EncMayor(int a, int b);
```

```
int main()
```

```
{
```

```
    int num1, num2, elmayor;
```

```
    cout << "Dame dos numeros:"<<endl;
```

```
    cin >> num1;
```

```
    cin >> num2;
```

```
    elmayor = EncMayor(num1,num2);
```

```
    cout << "El numero mayor es:"<<elmayor<<endl;
```

```
    system ("pause");
```

```
    return 0;
```

```
}
```

```
int EncMayor(int a, int b)    //Funcion EncMayor
```

```
{
```

```
    int nmax;
```

```
    if (a > b)
```

```
        nmax=a;
```

```
    else
```

```
        nmax=b;
```

```
    return nmax;
```

}

Al inicio del **ejemplo 5.4**, se especifica el prototipo de una función llamada **EncMayor()**, esto es, el tipo de valor que retornará, su nombre y los argumentos que recibirá. Aunque el código de la función aparece al final, ya están definidas sus características, por lo que no debería causar problema alguno al momento de ejecutar el programa.

Puntos importantes con el uso de funciones en C++:

- ✓ Cuando se llama a una función y se le envían datos, estos deben ser del mismo tipo que las variables que los reciben en la función.
- ✓ Al llamar a una función, el número de argumentos enviados debe coincidir con el prototipo especificado al momento de declarar a la función.
- ✓ Para la función **main ()** los pasos que realiza cualquier otra función son transparentes, es decir, ésta solo manda llamar a la otra función y espera el valor de retorno.
- ✓ Los valores de retorno de una función pueden variar:
 - **No reciben ni retornan**, son las más sencillas ya que para usarlas solo tenemos que saber el nombre de la función para llamarla sin enviarle argumentos.
 - **Reciben y no retornan**, solo reciben los argumentos, pero no retornan ningún valor.
 - **No reciben y retornan**, es cuando se llama a una función sin enviarle argumentos y después nos retorna un valor como resultado.
 - **Reciben y retornan**, son las que mandamos llamar enviándole argumentos, y enseguida nos retorna un valor como resultado. Los podemos ver en los ejemplos anteriores.

Cuando la función a escribir no regresará ningún valor, el tipo de retorno se define como tipo **void**, por lo que ya no sería necesaria la palabra reservada **return** al final de la función.

Ejemplo 5.5: Crear un programa que contenga una función llamada **suma ()** la cual no debe recibir nada como argumentos (no se le envían datos), y además no debe retornar ningún valor.

```
/* Ejemplo 5.5: Programa con función suma() que no  
retorna ningún dato */
```

```
#include<iostream>
using namespace std;
void suma() //Funcion que no recibe ni regresa valores
{
    int a, b, res;
    cout << "Dame los valores a sumar: ";
    cin >> a;
    cin >> b;
    res = a + b;
    cout << "La suma es: " << res;
}

int main()
{
    suma();//Se invoca a función suma() sin enviar datos
    system ("pause");
    return 0;
}
```

Ejemplo 5.6: Crear un programa que contenga una función llamada **suma ()** la cual no debe recibir nada como argumentos (no se le envían datos), pero si retorna un valor.

/ Ejemplo 5.6: La función suma() no recibe argumentos y si retorna un entero */*

```
#include<iostream>
using namespace std;

int suma()          // Funcion llamada suma()
{
    int a, b, res;
    cout << "Dame los valores a sumar: ";
    cin >> a;
    cin >> b;
    res = a + b;
    return res;
}

int main()
{
    int resum;
    resum = suma(); //Se llama funcion suma()
    cout << "La suma es: " << resum << endl;
    system ("pause");
    return 0;
}
```

5.2.1 Variables locales y globales

El lugar donde las variables son definidas en un programa es muy importante, ya que nos lleva a tomar en cuenta el ámbito o alcance que éstas tendrán. El acceso a cada una de las variables depende del lugar donde éstas son declaradas. Tomando en cuenta que el alcance también es aplicable a constantes y objetos. En C++ existen tres tipos de ámbitos: Global o de programa, Local o de función y De bloque, aunque aquí solo tratamos los dos primeros.

Variables globales

Una **variable global** es aquella que es declarada fuera de cualquier función, normalmente al principio del programa. Estas son accesibles desde su declaración hasta el final del programa por todas las funciones que ahí aparezcan, por lo que desde cualquier función podrán ser modificadas. Estas variables son inicializadas en cero cuando se empieza a ejecutar el programa.

Ejemplo 5.7: Aquí se presenta el uso de una variable global llamada **num** que es utilizada por dos diferentes funciones, y en ambas es modificado su valor.

```
// Ejemplo 5.7: Uso de una variable global
#include <iostream>
using namespace std;
int num = 1;      // Se define variable global

void func1()
{
    num = num * 2; // Se usa variable global
}
192
```

```
int main()
{
    cout << num << endl;      //Se usa variable global
    num = num + 2;
    cout << num << endl;
    func1();
    cout << num << endl;
    system ("pause");
    return 0;
}
```

En el **ejemplo 5.7** al principio se declaró a la variable global llamada **num**, y se le guardó el valor 1. Aquí observamos que tanto **main()** como la función **func1()** tienen acceso a la variable **num** y por tanto pueden modificar su valor.

Ejemplo 5.8: Se declara a una variable global llamada **var_global**, y una variable local llamada **i**. Observe que desde la función **main()** y desde **funcionX()** se puede modificar el valor de la variable global **var_global**.

```
/* Ejemplo 5.8 Programa que usa una variable global en
distintas funciones */
#include<iostream>
using namespace std;
float var_global;      //Variable global
float funcionX()       //Función llamada funcionX
{
    //Enseguida se incrementa en 2 a la variable global:
    var_global=var_global+2;
}
int main()
```

```
{
    float i;           // Variable local llamada i

    cout<<"\nValor variable global:"<<var_global;

    i = funcionX();    //Se llama a funcionX()

    cout<<"\nValor variable local: " << i;
    cout<<"\nVariable global:"<<var_global;
    //Enseguida se incrementa a variable global:
    var_global=var_global+1;
    cout<<"\nVariable global:"<<var_global;
    cout<<endl;

    system ("pause");
    return 0;
}
```

Observemos que en el **ejemplo 5.8** la variable llamada **var_global** fue declarada al principio del programa y fuera de todas las funciones, por lo que esta puede ser utilizada desde cualquier parte del programa.

Variables locales

Las variables locales son las que se declaran dentro de alguna función, y solo serán conocidas dentro del bloque de la función donde fueron declaradas. Estas variables desaparecen cuando la función termina y se devuelve el control del programa a quien la haya invocado, lo cual permite que una vez usadas las variables

desaparecerán de la memoria dejando libre el espacio que ocupaban.

Ya que la variable local solo es conocida en la función donde se declaró, esto nos permite que en un programa puedan existir variables con el mismo nombre, pero definidas en funciones diferentes.

Al momento de definir una función, los parámetros que aparecen en la cabecera de ésta se consideran variables locales de la función. Además, las variables locales no pueden ser inicializadas previamente por el compilador, por lo que se recomienda inicializarlas en el momento de su declaración, de otra manera su valor será indeterminado.

En el siguiente **ejemplo 5.9** se muestra que en `main()` son solicitados 10 números para determinar si cada uno es o no un valor primo, para esto se utiliza una función llamada `f_primo()` que recibe el número y retorna un valor booleano (`true` o `false`). Y sobre todo, aparece la manera de utilizar a las variables locales. Es importante conocer que el parámetro `num` es una variable local de la función `f_primo()`.

/ Ejemplo 5.9: La función llamada f_primo() comprueba si un número es primo */*

```
#include <iostream>
using namespace std;
```

```
bool f_primo(int num) // Definición de función f_primo
{
    int i = 2; // i es una variable local de f_primo()
    if(num == 1)
```

```
        return true;
else
{
    //Mientras el resto de division sea diferente de 0
    while(num%i != 0)
        i++;
    if(i == num)
        return true;
    else
        return false;
}
} // Fin de la funcion f_primo

int main()
{
    int numero, i; // variables locales de main()
    cout<<"--Dame 10 numeros positivos y enteros--\n";
    for(i=1;i<=10;i++)
    {
        do
        {
            cout << "\nDame numero " << i << ": ";
            cin >> numero;

            if(numero < 0)
                cout << "\nNumero no valido";
        }
        while(numero <= 0);
        if (f_primo(numero)) // Se llama a la funcion
            cout << "Es primo" << endl;
        else
            cout << "No es primo" << endl;
    }
    system ("pause");
    return 0;
}
```

Ejemplo 5.10: Se muestra como es utilizada una variable llamadas **x** en dos funciones diferentes, pero observe que cada **x** es solo conocida en la función que fue declarada.

/ Ejemplo 5.10: Uso de una variable llamada x definida en funciones distintas */*

```
#include<iostream>
#include<conio.h>
using namespace std;

int y;          // Se declara la variable global

void Mifuncion()
{
    int x; // x es variable local en Mifuncion()
    x = 3;
    cout << "\nEstamos en funcion Mifuncion(): " << endl;
    cout << "x=" << x << "\ty=" << y << endl;
    getch();
}

int main()
{
    int x;          // Esta variable es local en main()
    y = 100;
    x = 1;
    cout << "La variable global se llama y.";
    cout << "\nLa funcion main() tiene una variable local
    llamada x";
    cout << "\nLa funcion Mifuncion() tiene variable local
```



```
llamada x.";
cout << "\n\nEstamos en funcion main(): " << endl;
cout << "x=" << x << "\ty=" << y << endl;
getch();

Mifuncion();      //Se manda llamar a Mifuncion()
cout << "\nEstamos en funcion main(): " << endl;
cout << "x=" << x << "\ty=" << y << endl;
y = 150;          //Esta variable es global
getch();
cout << "\nEstamos en funcion main(): " << endl;
cout << "x=" << x << "\ty=" << y << endl<< endl;
getch();
system ("pause");
return 0;
}
```

5.3 Funciones existentes en C++

El lenguaje de programación C++ nos ofrece un conjunto de funciones de uso general, las cuales están definidas y agrupadas según el tipo de actividad o proceso que realizan, y a cada grupo de instrucciones se les conoce como archivos de biblioteca, de cabecera (header) o simplemente librería.

Las librerías son un tipo de archivo que podemos agregar o incluir en nuestros programas, y al hacerlo significa que podemos utilizar cualquier función perteneciente a estas librerías. Por eso, cuando escribimos nuestros programas empezamos incluyendo los archivos de cabecera o librerías necesarias, por ejemplo:

```
#include<iostream>
#include<conio.h>
#include<cmath>
```

Al escribir los `#include< >`, le estamos indicando al compilador las librerías que deberá utilizar, y así nos permitirá hacer uso de instrucciones o funciones existentes dentro de ellas.

Ejemplo 5.11: Programa que utilice una función para elevar un número dado (base) a una potencia determinada (exponente). Sabemos que ya existe una función que realiza esto y para poder utilizarla es necesario incluir la librería a la cual pertenece. En este ejemplo vamos a utilizar una del compilador de Dev C++ que se llama `pow()`, la cual pertenece a la librería `<cmath>`. Es importante que antes de usar una función conozcamos cuál es su prototipo, para poder enviarle los argumentos de manera correcta, y así nos retorne el valor esperado.

```
/* Ejemplo 5.11 Programa que utiliza una funcion ya
definida llamada pow */
```

```
#include<iostream>
```

```
/* Enseguida se agrega libreria de funciones
matematicas: */
```

```
#include<cmath>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int base, exponente;
```

```
double resultado;  
cout << "\nDame la base: ";  
cin >> base;  
cout<<"\nDame el exponente: ";  
cin>>exponente;  
resultado=pow(base,exponente); //Se llama a funcion  
cout << "\nEl resultado es: " << resultado << endl;  
return 0;  
}
```

Enseguida se mencionan sólo de funciones ya existentes en librerías de C++, las cuales podremos utilizar en nuestros programas, pero existen muchas más.

- ✓ **Funciones numéricas de la librería `cmath`.** Nos ayudan a trabajar con operaciones matemáticas. Y para usarlas es necesario incluir la librería **`cmath`**.

`ceil (x)` Redondea al entero más cercano, el prototipo es: **`double ceil(double x)`**.

`fabs (x)` Regresa el valor absoluto de x, el prototipo es: **`double fabs(double x)`**.

`floor (x)` Redondea de manera predeterminada al entero próximo más cercano, el prototipo es: **`double floor(double x)`**.

`pow (x, y)` Eleva a la potencia y el valor de x (si x es menor o igual a cero y tiene que ser entero, o si x es igual a cero y no

puede ser negativo), el prototipo es: **double pow (double x, double y).**

sqrt (x) Regresa la raíz cuadrada de x, donde x debe ser mayor o igual a cero. El prototipo es:

double ceil(double x).

- ✓ **Funciones trigonométricas.** Sirven para realizar cálculos trigonométricos y para esto es necesario incluir la librería **cmath**.

cos (x) Calcula el coseno del ángulo x, donde x debe expresarse en radianes, el prototipo de la función es: **double cos (double x).**

acos (x) Calcula el arco coseno de x, donde x debe estar entre -1 y 1. El prototipo de la función es: **double acos (double x).**

sin (x) Calcula el seno de un ángulo x, donde x debe expresarse en radianes. El prototipo de función es: **double sin (double x).**

tan(x) Devuelve la tangente del ángulo x, donde x se expresa en radianes. El prototipo de la función es: **double tan (double x).**

Algo que nos puede ayudar para convertir un ángulo expresado en grados a radianes y así poderlo utilizar con las funciones trigonométricas, se hace de la siguiente forma: Multiplicar los grados por **pi/180**, donde pi es 3.14159.

- ✓ **Funciones logarítmicas y exponenciales.** Estas son utilizadas en las matemáticas y en el mundo de las

empresas y los negocios. Para utilizarlas requieren de la librería llamada **cmath**.

exp (x) Calcula el exponencial de una base e de logaritmos naturales (2.718282), el prototipo es el siguiente: **double exp (double x)**.

log (x) Calcula el logaritmo natural (logaritmo en base e) del valor x, el prototipo es el siguiente: **double log (double x)**.

- ✓ **Funciones aleatorias.** Se utilizan en diferentes aplicaciones donde se requiere generar números aleatorios. Estas funciones están en la librería **stdlib.h**.

rand(void) Genera un número aleatorio que varía en el rango entero de 0 a RAND_MAX que es una constante que se define en **stdlib.h**. El prototipo de la función es: **int rand (void)**.

random(x) Genera un número aleatorio dentro de un rango especificado que es entre 0 y el valor de **x** especificado (límite superior). El prototipo de la función es: **int random (void)**.

- ✓ **Funciones de fecha y hora.** En las computadoras viene instalado un sistema para el control de las fechas y horas, la fecha se guarda de acuerdo con el formato gregoriano (mm/dd/aa). Y para usar funciones de este tipo debemos incluir la librería **time.h**.

time (hora) Devuelve el número de segundos transcurridos desde la hora base.

clock (void) Devuelve el tiempo de CPU empleado por el programa en curso. Esto es, tiempo de procesador transcurrido desde el principio de la ejecución del programa.

- ✓ **Funciones de caracter.** Son funciones y macros para la manipulación de caracteres. Y estas funciones requieren del archivo de cabecera **ctype.h**. La mayoría de las funciones retornan falso o verdadero.

int isalpha(int c) Nos sirve para indicar si **c** es letra mayúscula o minúscula.

int toupper(int c) Convierte la letra **c** a mayúscula.

int isdigit(int c) Nos indica si es un dígito decimal entre 0 y 9.

Ejemplo 5.12: Programa que usa función aleatoria llamada **rand()**, aquí nos sirve para obtener diez números aleatorios y mostrarlos en pantalla. Esta función pertenece a la librería **stdlib.h**.

/ Ejemplo 5.12 Programa que usa función ya definida que se llama rand */*

```
#include<iostream>
#include<stdlib.h>      //Libreria para funcion rand()
using namespace std;
int main()
{
    int va, Numero;
    for (va = 1; va <= 10; va++)
    {
        Numero=rand();      // Uso de funcion rand()
        cout << "\n El numero aleatorio generado es: " <<
```

```
    Numero;  
}  
system ("pause");  
return 0;  
}
```

Ejemplo 5.13: Programa que utiliza una función para elevar al cubo a un número flotante. La función se llama **potencia3()** y recibe el argumento, **a** que representa la base.

```
/* Ejemplo 5.13 Programa que calcula el cubo de un  
numero flotante */
```

```
#include<iostream>  
#include<cmath>  
using namespace std;
```

```
void potencia3(float a) //Funcion de programador  
{  
    float b;  
    b = pow(a,3); //Llama a funcion pow()  
    cout << "\n El numero al cubo es: " << b;  
}
```

```
int main()  
{  
    float a;  
    cout << "\n Dame un numero flotante: ";  
    cin >> a;  
    potencia3(a); //Se llama a funcion potencia3()
```

```
    return 0;  
}
```

Ejemplo 5.14: Programa que busca la posición de una letra dentro de una cadena. Se muestra una función creada por el programador llamada **buscaletra()**, la cual recibe dos argumentos, el carácter a buscar y cadena donde se buscara. Además se hace uso de la función llamada **strlen()** que nos regresa el tamaño de una cadena y se encuentra en la biblioteca llamada **string.h**.

```
/* Ejemplo 5.14 Encuentra la posicion de una letra  
dentro de una cadena */
```

```
#include<iostream>  
#include<string.h>      // Libreria para usar strlen()  
#define MAX 15          //Se define constante  
using namespace std;  
//La siguiente funcion regresa un valor entero  
int buscaletra(char caracter, char texto[])  
{  
    int valor=0;  
    while((caracter != texto[valor]) &&  
          (texto[valor] != '\0'))  
        valor++;  
    return valor;  
}  
int main()  
{  
    char cadena[MAX], letra;
```



```
int num, ind, tam;
cout << "\nDame una cadena: ";
cin.getline(cadena, MAX, '\n'); //Se lee la cadena
cout << "Que caracter quieres encontrar: ";
cin>>letra; //Se lee la letra a encontrar en cadena
tam = strlen(cadena); /* Funcion que regresa
                        longitud de la cadena */

num = -1;
num = buscaletra(letra, cadena);
if((num >= 0) && (num < tam))
    cout << "El caracter " << letra << " esta en
    posicion: " << num+1 << endl;
else
    cout<<"Caracter no aparece en la cadena"<<endl;
system ("pause");
return 0;
}
```

Actividad 5.1: Realizar los siguientes ejercicios utilizando lo aprendido con módulos.

1. Programa que permita al usuario elegir calcular el área de figuras geométricas como: círculo, cuadrado, rectángulo, triángulo.
2. Programa que permita calcular el salario de un trabajador dadas las horas trabajadas y la tarifa por hora. Si la cantidad de horas trabajadas es mayor a 40 horas, la tarifa por hora se incrementa en un 50% para las horas extras, calcular el salario del trabajador.
3. Programa que permita ingresar tres números, obtener su promedio y visualizar “APROBADO”, si su promedio es mayor a 7, caso contrario visualizara “DEBES MEJORAR”. Utilice función de la librería de C++.
4. Programa que solicite ingresar una cantidad (número entero) en segundos y lo muestre en minutos y horas.
5. Programa que solicite la temperatura promedio diaria, durante cinco días y muestre el mensaje que corresponde de acuerdo a la siguiente tabla:

Menor a 15 grados centígrados	Baja
Entre 15 y 27 grados centígrados	Templada
Mayor a 27 grados centígrados	Alta

Rúbrica de evaluación ejercicios del capítulo 5

	Excelente 10-8	Bueno 7-4	Deficiente 3-1	No realizado 0	Tot al
Declaración de módulos o funciones	Conoce y aplica la modularidad en el desarrollo de programas para la optimización de los mismos y reutilización de código.	Conoce y aplica la modularidad en el desarrollo de programas, sin embargo no optimiza la reutilización del código.	Tiene conocimientos básicos de modularidad, pero desconoce la aplicación de éstos.	Desconoce el tema de modularidad,	
El paso de parámetros y argumentos	Distingue y aplica correctamente la diferencia entre el paso de parámetros vs. argumentos	Aplica la creación de módulos con uso de parámetros sin embargo confunde la diferencia entre el paso de parámetros vs. argumentos	Tiene deficientes nociones de la diferencia entre el paso de parámetros vs. argumentos	Desconoce el tema módulos con parámetros y argumentos.	
Funciones existentes en c++	Distingue y aplica correctamente el uso de funciones en c++	Conoce algunas funciones precargadas en c++ sin embargo la aplicación del paso de argumentos no lo hace correctamente,	Conoce unas pocas funciones precargadas en c++ sin embargo la aplicación del paso de argumentos no lo hace correctamente,	Desconoce la aplicación de funciones precargadas en c++	
Comentarios					

Referencias

- Bratley, P., & Brassard, G. (1996). *Fundamentos de algoritmia: Una perspectiva de la ciencia de los computadores*. Pearson, Prentice Hall.
- Cairó Battistutti, O. (2003). *Metodología de la programación: algoritmos, diagramas de flujo y programas*. Alfaomega.
- Deitel, P., & Deitel, H. (2015). *Cómo programar en C++*. Pearson.
- Farrell, J. (2001). *Introducción a la programación: lógica y diseño*. International Thomson Editores, S. A. de C. V.
- García Carballeira, F. (2001). *El lenguaje de programación C, diseño e implementación de programas*. Pearson Education.
- García Hernández, E. (s.f.). *Programación C++*. Obtenido de Programación C++.
- García Molina, J. (2005). *Una introducción a la programación: un enfoque algorítmico*. Thomson.
- Joyanes Aguilar, L. (2003). *Fundamentos de programación, libro de problemas*. McGraw-Hill/Interamericana de España, S. A. U.
- Joyanes Aguilar, L. (2020). *Fundamentos de programación. Algoritmos, estructura de datos y objetos*. Mcgraw-Hil.
- Joyanes Aguilar, L., & Zahonero Martínez, I. (2014). *Programación en C, C++, Java y UML*. McGraw-Hill.
- López, L. (2011). *Programación estructurada y orientada a objetos, un enfoque algorítmico*. Alfaomega.

- Marquez Frausto, T. G., Osorio Ángel, S., & Olvera Pérez, E. (2011). *Introducción a la programación estructurada en C*. México: Pearson Education.
- Martinez del Rio, F. (s.f.). *Fundamentos básicos de programación en C++*. Obtenido de Universidad de Jaén.
- Programación ATS. (2016). *Programación en C++ || Primer programa en C++* [Archivo de Vídeo]. Youtube.
<https://www.youtube.com/watch?v=ld4nzao5XAc>
- Rodríguez Almeida, M. A. (1991). *Metodología de la programación a través de pseudocódigo*. McGraw-Hill Interamericana de España.
- Solórzano, P. J., Millán Najera, A., Solórzano, S. C., & Sánchez Baca, A. (2012). *Fundamentos de computación, Panorama histórico y programación*. México, D.F.: Universidad Nacional Autónoma de México.
- Urrutia Landa, G. (2012). *Curso de programación en C para principiantes: Aprende a programar en C desde cero*.
- Zavala López, R., & Llamas Avalos, R. (2013). *Fundamentos de programación para principiantes*. Edición Kindle.

Resumen

El libro de “Fundamentos de programación” está conformado por 209 páginas y el contenido está distribuido en cinco capítulos, los cuales ayudarán a entender los conceptos básicos necesarios para poder construir algoritmos y programas de computadora.

Se presenta de una forma teórico-práctico ya que se enfoca en la creación de ejemplos y ejercicios prácticos, pero sin dejar a un lado los conocimientos teóricos necesarios para poderlo lograr.

Se enfoca a todas aquellas personas principiantes de cualquier edad y que buscan la manera de introducirse en el área de la programación.

Lo que vas a lograr con este libro:

- Los conceptos básicos de algoritmos
- Las bases necesarias para la programación estructurada
- Crear algoritmos en diagrama de flujo con DFD
- Crear algoritmos en pseudocódigo con PSeInt
- Crear programas para la solución de problemas con el lenguaje de programación Dev-C++
- Estar preparado para aprender otros lenguajes de programación

Este libro está escrito para desarrollar todas tus habilidades como programador empezando desde cero.

En el apasionante mundo de la programación de computadoras, el conocimiento es la clave para abrir las puertas a un futuro lleno de posibilidades. En "Fundamentos de Programación: Una Guía Teórico-Práctica", los lectores encontrarán una valiosa herramienta para iniciarse en este emocionante campo. Desde el diseño algorítmico hasta la modularidad, este libro ofrece una estructura ordenada y gradual que guiará a los estudiantes hacia un dominio efectivo de los conceptos fundamentales.

Con un enfoque didáctico y ameno, este libro presenta una combinación única de teoría y práctica.

Los autores, han diseñado cuidadosamente cada capítulo para facilitar la comprensión y el aprendizaje. Con la ayuda de ejemplos claros y ejercicios desafiantes, los estudiantes podrán fortalecer sus habilidades en la creación de algoritmos, el uso de diagramas de flujo DFD y pseudocódigos con PSEint, la programación en Dev-C++, y mucho más.

Ya sea que estés dando tus primeros pasos en el mundo de la programación o buscando mejorar tus habilidades, "Fundamentos de Programación" te guiará en un emocionante viaje hacia el dominio de esta poderosa herramienta del siglo XXI.

ISBN 978-607598724-8



9 786075 987248

